WNDRVR
SUPPORT NETWORK

VXWORKS 7 DHCP PROGRAMMER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 DHCP Programmer's Guide, SR0650

16 September 2020

1. OVERVIEW

1. Introduction

The Dynamic Host Configuration Protocol (DHCP) is an Internet protocol that automates the configuration of computers that use TCP/IP. You can use DHCP to automatically assign IP addresses, deliver TCP/IP stack configuration parameters such as the subnet mask and default router, and provide other configuration information, such as the addresses for printer, time, and news servers. DHCP is specified in RFC 2131: *Dynamic Host Configuration Protocol* and RFC 2132: *DHCP Options and BOOTP Vendor Extensions*.

Further Reading

For further general information on DHCP, see the following sources:

- For an overview of DHCP, see DHCP Overview on page 3.
- For information on RFC conformance details for DHCP, see Supported RFCs on page 2.
- For the DHCPv4 client, see RFC 2131 and RFC 2132.
- For the DHCPv4 server, see RFC 2131, RFC 2132, and RFC 2242.
- For the DHCPv4 relay agent, see RFC 951, RFC 1542, RFC 2131, RFC 2132, RFC 3046, and RFC 4030.
- For the DHCPv6 client, see RFC 3315, RFC 3319, RFC 3646, RFC 3736, RFC 4075, and RFC 4242.
- For the DHCPv6 relay agent, see RFC 3315.
- For the DHCPv6 server, see RFC 2462, RFC 3315, RFC 3319, RFC 3633, RFC 3646, and RFC 3736.

For detailed information on the following related topics, refer to the appropriate chapters:

- Configurating and Building DHCP on page 4
- Wind River DHCPv4: Client on page 12
- Wind River DHCPv4: Server on page 32
- Wind River DHCPv4: Relay Agent on page 56
- Wind River DHCPv6: Client on page 66
- Wind River DHCPv6: Server and Relay Agent on page 88

Source Code Files

The DHCP components are in the following base directory:

installDir **/vxworks-7/pkgs_v2/net/ipnet/app/dhcp-x.x.x.x**

The source code files for DHCP components are in the following sub-directories:

- DHCPv4 client (DHCPC): **/dhcpc-***x.x.x.x***/**
- DHCPv4 server (DHCPS): **/dhcps-***x.x.x.x***/**
- DHCPv4 relay agent (DHCPR): **/dhcpr-***x.x.x.x***/**
- DHCPv6 client (DHCPC6): **/dhcpc6-***x.x.x.x***/**
- DHCPv6 server (DHCPS6): **/dhcps6-***x.x.x.x***/**

WNDRVR

2. Supported RFCs

The RFC conformance details for the Wind River DHCP modules are as follows:

- RFC 951: *Clarifications and Extensions for the Bootstrap Protocol*
- RFC 1542: *Clarifications and Extensions for the Bootstrap Protocol*
- RFC 2131: *Dynamic Host Configuration Protocol*
- RFC 2132: *DHCP Options and BOOTP Vendor Extensions*

  The server implements all options and vendor extensions in RFC 2132, with the exception of the following options:

  - 66 – TFTP server
  - 67 – Bootfile name

  There is a Wind River-specific option for setting a bootfile name; see the entry for boot-file in Wind River-Specific DHCPv4 Server Configuration Options table on page 54.

  For more information about the Wind River DHCP client options, see DHCP Options Implemented in the DHCPv4 Client table on page 22 and Macros for DHCP Options Not Implemented in the DHCPv4 Client table on page 25.
- RFC 2242: *NetWare/IP Domain Name and Information*

  RFC 2242 adds two option codes for the NetWare/IP product: Option code 62 (NetWare/IP Domain Name) and option code 63 (Netware/IP Information.

  The Wind River DHCP server implements option code 62, but does not implement option code 63.
- RFC 2462: *IPv6 Stateless Address Autoconfiguration*
- RFC 3046, *DHCP Relay Agent Information Option*

  RFC 3046 adds an option code for the DHCP Relay Agent Information Option: Option code 82 (Relay Agent Information Option). RFC 3046 defines two suboptions for option 82: the Agent Circuit ID suboption (suboption 1) and the Agent Remote ID suboption (suboption 2).
- RFC 3315: *Dynamic Host Configuration Protocol for IPv6*

  Reconfigure messages:

  The Wind River DHCPv6 client does not include Reconfigure messages. Reconfigure messages are described in RFC 3315, section 5.3, "DHCP Message Types." This means the Reconfiguration options **OPTION_RECONF_MSG** and **OPTION_RECONF_ACCEPT** do not work.

  Vendor or user-defined options:

  The Wind River DHCPv6 client does not allow vendor- or user-defined configuration options. This means the following options do not work: **OPTION_USER_CLASS, OPTION_VENDOR_CLASS**, and **OPTION_VENDOR_OPTS**.

  Temporary address assignments: The Wind River DHCPv6 client does not include the assignment of temporary addresses, as described in RFC 3041: *Privacy Extensions for Stateless Address Autoconfiguration in IPv6*.
- RFC 3319: *Dynamic Host Configuration Protocol (DHCPv6) Options for Session Initiation Protocol (SIP) Servers*
- RFC 3633: *IPv6 Prefix Options for Dynamic Host Configuration Protocol (DHCP) version 6*
- RFC 3646: *DNS Configuration options for Dynamic Host Configuration Protocol for IPv6*
- RFC 3736: *Stateless Dynamic Host Configuration Protocol (DHCP) Service for IPv6*
- RFC 4030, *The Authentication Suboption for the DHCP Relay Agent Option*

  RFC 4030 defines a suboption for option 82 (RFC 3046, DHCP Relay Agent Information Option). The suboption is named Authentication Suboption (suboption 8).
- RFC 4075: Simple Network Time Protocol (SNTP) Configuration Option for DHCPv6
- RFC 4242: Information Refresh Time Option for Dynamic Host Configuration Protocol for IPv6

The Wind River DHCPv6 client does not include vendor-specific and user-defined configuration options or configuration options contained in RFCs that are not listed here.

3. DHCP Overview

The Wind River Network Stack includes a server and client for both DHCPv4 and DHCPv6, and a relay agent. The chapters that follow this overview describe these implementations.

DHCP extends an earlier protocol, Bootstrap Protocol (BOOTP, see RFCs 951 and 1542), and you can configure most DHCP servers, including the Wind River DHCP server, to allow requests from BOOTP clients.

When a DHCP server assigns an IP address to a client, it gives the client a lease on that address for a specific length of time. Clients can renew their leases before they expire. When a lease expires, the server can reassign the IP address to a different computer. This allows the server to maintain a pool of IP addresses that it reuses as computers join and leave the network.

Although the primary purpose of DHCP is dynamic configuration, it can also assign a client a static, permanent IP address and configuration. In such a case, the client is given a lease of infinite duration.

For a list of implemented RFCs, see Supported RFCs on page 2.

Architectural Overview: Client, Server, and Relay Agent

DHCP uses a client-server model. DHCP clients obtain configuration information from a DHCP server. DHCP servers maintain databases with lease and configuration information and provide this information to DHCP clients.

A DHCP client obtains configuration information by broadcasting a message to locate one or more DHCP servers. Routers are generally configured to block broadcast messages, which would prevent a client's broadcast message from reaching a DHCP server that is not on the same subnet as the client. DHCP overcomes this problem through the use of a relay agent. The relay agent relays messages from the client's subnet to servers on other subnets. Typically the relay agent resides on a router, but this is not a requirement.

A DHCP client may receive lease offers from multiple servers, at which point it may choose the offer that best meets its requirements.

DHCPv6

DHCPv6 is DHCP for IPv6 (RFC 3315). In its functionality and architecture, DHCPv6 is similar to DHCP for IPv4, but there are important differences.

Many of these reflect the change in addressing from IPv4 to IPv6, but there are also changes in other areas. For example, DHCPv6 introduces changes in messaging and message formats and you configure it with a different set of configuration options.

DHCPv6 supports the DHCPv6 Client Fully Qualified Domain Name (FQDN) option (RFC 4704), which can be used by DHCPv6 clients and servers to exchange information about the client's fully qualified domain name and about which service has the responsibility for updating the DNS with the associated AAAA resource record (RR). The server always updates the pointer (PTR) RR.

DHCPv6 supports the Session Initiation Protocol (SIP) that allows SIP clients to locate SIP servers for outbound SIP requests (see RFC 3319). SIP is an application-layer control protocol that can establish, modify, and terminate multimedia sessions such as Internet telephony calls.

2. CONFIGURING AND BUILDING DHCP

1. Configuration and Build Workflow

Your Wind River VxWorks 7 installation includes source code but does not include pre-built kernel libraries. Before beginning any VxWorks development, you must build the VxWorks kernel libraries (VSB project).

You must also build the VxWorks run-time kernel image based on the kernel libraries (VIP). For specific instructions on how to configure and build a VSB and a VIP, and how to create a downloadable kernel module (DKM) or a real-time process (RTP) application, see the *VxWorks 7 Configuration and Build Guide*.

For a list of source build configuration options for DHCP, see Source Build Configuration Options (VSB) on page 4.

For a list of kernel configuration components for DHCP, see Kernel Configuration Components (VIP) on page 4.

2. Source Build Configuration Options (VSB)

The DHCP VSB Options table on page 4 lists the DHCP VSB options.

Table 1. DHCP VSB Options

Service	Option	Description
DHCPv4 Client	IPNET_DHCPC	Include the DHCPv4 client in the VSB.
DHCPv6 Client	IPNET_DHCPC6	Include the DHCPv6 client in the VSB.
DHCP Relay Agent	IPNET_DHCPR	Include the DHCPv4 relay agent in the VSB.
DHCPv4 Server	IPNET_DHCPS	Include the DHCPv4 server in the VSB.
DHCPv6 Server	IPNET_DHCPS6	Include the DHCPv6 server in the VSB

3. Kernel Configuration Components (VIP)

The DHCP Components table on page 4 lists the DHCP VIP components that are required for DHCP functionality.

Table 1. DHCP Components

Component	Description
INCLUDE_IPDHCPC	Include DHCPv4 client functionality to interface with DHCPv4 servers using the Dynamic Host Configuration Protocol.

Component	Description
	See Wind River DHCPv4: Client on page 12 for information on the DHCPv4 client.
INCLUDE_IPDHCPS	Include DHCPv4 server functionality to interface with DHCPv4 clients. See Wind River DHCPv4: Server on page 32 for information on the DHCPv4 server.
INCLUDE_IPDHCPR	Include DHCP relay agent functionality to pass DHCP messages between DHCP clients and DHCP servers. See Wind River DHCPv6: Server and Relay Agent on page 88 for information on the DHCP relay agent.
INCLUDE_IPDHCPC6	Include DHCPv6 client functionality to interface with DHCPv6 servers using the Dynamic Host Configuration Protocol. To add the DHCPv6 client shell command, see Optional Kernel Configuration Components on page 5. See Wind River DHCPv6: Client on page 66 for information on the DHCPv6 client.
INCLUDE_IPDHCPS6	Include DHCPv6 server functionality to interface with DHCPv6 clients. To add the DHCPv6 server shell command, see Optional Kernel Configuration Components on page 5. See Wind River DHCPv6: Server and Relay Agent on page 88 for information on the DHCPv6 server and DHCPv6 relay agent.

4. Optional Kernel Configuration Components

The Optional Components table on page 5 lists optional DHCP and shell command components.

Table 1. Optional Components

Component	Description
INCLUDE_BOOT_DHCPC	Enable DHCP in a boot ROM. See Enabling DHCP in a Boot ROM on page 12.
INCLUDE_IPNET_IFCONFIG_1	Interface configuration component for interface number 1. See Enabling DHCP in a Boot ROM on page 12.
INCLUDE_IPCOM_SHELL_CMD	Include the IPCOM shell command interface to access the dhcpc, dhcps, and dhcpr shell commands.
INCLUDE_IPDHCPC6_CMD	Include the DHCPv6 client shell command (dhcpc6).
INCLUDE_IPDHCPS6_CMD	Include the DHCPv6 server shell command (dhcps6).

Component	Description
INCLUDE_IPIFCONFIG_CMD	Include the interface configuration shell command (ifconfig).
INCLUDE_IPD_CMD	Include the daemon control shell command (ipd).
INCLUDE_IPCOM_SYSLOGD_CMD	Include the system logging shell command (syslog).
INCLUDE_IPCOM_SYSVAR_CMD	Include the system variables shell command (sysvar).
INCLUDE_STARTUP_SCRIPT	Include the startup script facility to run kernel shell commands at VxWorks startup. For example usage, see DHCPv4 Relay Agent Information Option Tutorial on page 96.

5. Setting DHCP Parameters at Boot Time

At boot time, you can set DHCP parameters from a startup script or by implementing a routine that reads a configuration file at startup.

Startup Script

To run kernel shell commands automatically upon startup, include commands in a startup script and run the script at boot time. You must also include the **kernel shell startup script (INCLUDE_STARTUP_SCRIPT)** in your VIP

For example usage of a startup script to set DHCP parameters as part of the DHCPv4 Relay Agent Information Option Tutorial on page 96 see Creating the Target Startup Scripts on page 101.

Configuration File

For example usage of a routine that reads a configuration file at startup to set DHCP parameters, see Implementing Hook Routines for Initialization and Shutdown on page 42 and Reading Configuration Values from a File on page 44.

6. Setting DHCP Parameters at Run Time

At run time, you can set parameters with DHCP shell commands or by modifying system variables (sysvars).

To run shell commands and modify system variables, you must include specific VIP shell command components in your project, see Optional Kernel Configuration Components on page 5.

Shell Commands

You can run shell commands from the kernel shell or execute shell commands from an application using public API routines.

To view component-specific DHCP shell commands and display help for each command, access the kernel shell and execute the following commands:

```
-> cmd
# dhcpc
```

```
# dhcps
# dhcpr
# dhcpc6
# dhcps6
```

To return to the kernel shell from the command-interpreter mode, enter the following command:

```
# C
->
```

sysvars

To view all DHCP **sysvars**, access the command-interpreter mode from the kernel shell and execute the following command:

```
# sysvar list ipdhcp
```

To list component-specific DHCP sysvars, execute the following commands:

```
# sysvar list ipdhcpc
# sysvar list ipdhcps
# sysvar list ipdhcpr
# sysvar list ipdhcpc6
# sysvar list ipdhcps6
```

To change DHCP parameters with **sysvar**, use the sysvar set command.

3. FILE-BASED CONFIGURATION FOR THE DHCP CLIENTS

1. Introduction

By default, DHCPv4 and DHCPv6 client parameter values are set at VIP configuration time. The file-based configuration functionality allows you to reconfigure DHCPv4 and DHCPv6 client parameter values at boot time.

You reconfigure service parameter values with a list of system variables stored in a text-based configuration file. The file path and file name are specified by a VIP parameter. This configuration method provides an easy way to adapt a system to new networking environments by simply updating the configuration file contents and then transferring the file to the target before starting the service.

2. Enabling and Disabling File-Based Configuration

DHCPv4 Client

To enable the configuration of the DHCPv4 client from a configuration file, you must specify the full path to the configuration file as part of the DHCPC_CFGFILE VIP parameter.

To disable the configuration of the DHCPv4 client from a configuration file, set the DHCPC_CFGFILE VIP parameter to NULL (" "). The DHCPv4 client configuration is then based on the VIP configuration.

For information on configuring the DHCPC_CFGFILE VIP parameter, see Sample DHCP Client Configuration Files on page 8.

For information on operational requirements for the target, see Optional Kernel Configuration Components on page 5.

DHCPv6 Client

To enable the configuration of the DHCPv6 client from a configuration file, you must specify the full path to the configuration file as part of the DHCPC6_CFGFILE VIP parameter

To disable the configuration of the DHCPv6 client from a configuration file, set the DHCPC6_CFGFILE VIP parameter to NULL (" "). The DHCPv6 client configuration is then based on the VIP configuration.

For information on configuring the DHCPC6_CFGFILE VIP parameter, see DHCPv6 Client Configuration Parameters on page 66.

For information on operational requirements for the target, see Operational Requirements on page 11.

3. Sample DHCP Client Configuration Files

DHCPv4 Client

Wind River provides a sample DHCPv4 client configuration file that contains the list of valid system variables that can be configured from a configuration file.

The sample configuration file for the DHCPv4 client is at:

installDir **/vxworks-7/pkgs_v2/net/ipnet/app/dhcpc/sample_cfg/etc/dhcpc.conf**

WNDRVR

Note: Only the specified system variables in the sample file are configurable from a configuration file.

To correlate DHCPv4 client VIP parameter names to DHCPv4 client system variables listed in the sample configuration file, see DHCPv4 Client Configuration Parameters on page 13.

Note: To prevent unnecessary reconfiguration of the VIP parameters, the configuration file should only contain system variables that must be changed. The remaining parameter values will be taken from the VIP configuration.

DHCPv6 Client

Wind River provides a sample DHCPv6 client configuration file that contains the list of valid system variables that can be configured from a configuration file

The sample configuration file for the DHCPv6 client is at:

installDir **/vxworks-7/pkgs_v2/net/ipnet/app/dhcpc6/sample_cfg/etc/dhcpc6.conf**

Note: Only the specified system variables in the sample file are configurable from a configuration file.

To correlate DHCPv6 client VIP parameter names to DHCPv6 client system variables listed in the sample configuration file, see DHCPv6 Client Configuration Parameters on page 66.

Note: To prevent unnecessary reconfiguration of the VIP parameters, the configuration file should only contain system variables that must be changed. The remaining parameter values will be taken from the VIP configuration.

4. Configuration File Format

The configuration file is a text-based file you can edit with a standard text editor.

To configure a parameter from a configuration file, a valid system variable name must be followed by a valid value (system variable / value pair) and be on a single line.

A configuration file can also contain comments.

Note: The configuration file does not accept Escape characters (\).

System Variable and Value Pair on page 9A system variable/value pair consists of a system variable name and a value. Valid system variables are provided in the sample configuration file.
Comment Delimiter on page 10 The comment delimiter for the configuration file is the pound (**#**) character. For example:

4.1. System Variable and Value Pair

A system variable/value pair consists of a system variable name and a value. Valid system variables are provided in the sample configuration file.

A system variable/value pair can start in any column of a given line. A system variable and name must be separated by least one space or tab.

Each line can only contain a single system variable/value pair.

Boolean, integer, floating point, and string values can be listed with or without double quotation marks. Example valid line content follows:

```
systemVariableName_A TRUE
systemVariableName_B 2
systemVariableName_C 192.168.1.1
systemVariableName_D String
```

```
systemVariableName_A "TRUE"
systemVariableName_B "2"
systemVariableName_C "192.168.1.1"
systemVariableName_D "String"
```

Parent topic: Configuration File Format on page 9

4.2. Comment Delimiter

The comment delimiter for the configuration file is the pound (**#**) character. For example:

```
# This is a comment line.
```

You can add a comment on the same line as a system variable/name pair. For example:

```
systemVariableName_A FALSE       # This is a boolean value.
systemVariableName_B 22          # This is an integer value.
systemVariableName_C 192.168.2.2 # This is a floating point value.
systemVariableName_D String      # This is a string value.
```

Parent topic: Configuration File Format on page 9

5. Configuration File Processing

All empty lines and comment lines are not processed.

If a configuration file is not found, no processing is done and the system variables retain their values from the VIP configuration.

If a configuration file is located, the system processes the configuration file line-by-line. The values in the configuration file overwrite the values set at VIP configuration time.

Lines with errors such as misspelled system variable names and missing parameter values are ignored (skipped). Hence, a system variable associated with a faulty line entry retains the value set at VIP configuration time.

Note: If a system variable/value pair is listed more than once in a file, the last valid occurrence is used to configure the system variable. No error is reported as part of the diagnostic output.

WNDRVR

6. Diagnostics

You enable and disable diagnostics from within the configuration file. When enabled, the configuration status of the system variables is sent to the console, if a console is available.

To enable diagnostics, set the **cfgfile.diag** variable in the configuration file to TRUE. To disable, set to **FALSE**. For example:

```
cfgfile.diags FALSE
```

When enabled, the diagnostics process generates output and provides status on the action performed on each system variable.

An updated system variable is labelled as **Updated**. The output format is as follows:

```
Variable: systemVariableName Value: value Result: Updated
```

A skipped update for a system variable is labelled as **Unrecognized variable (Skipping)**. The output format is as follows:

```
Variable: systemVariableName Value: value Result: Unrecognized variable (Skipping)
```

7. Verifying System Variables

To verify the configuration of the system variables, you can display the system variables and their content from the command shell.

To view system variables, use the following command:

```
vxWorks# sysvar list serviceName
```

Note: To access the sysvar shell command, you must add the **INCLUDE_IPCOM_SYSVAR_CMD** component to your VIP.

8. Operational Requirements

To enable parameter configuration from a file, the target hosting the a DHCPv4 or DHCPv6 client must include read/write mass storage capability.

A configuration file can be modified by an external host and transferred to the target using one of the following methods:

- By transferring the file from a FTP server to the target.
- By mounting the target's mass storage device as a USB disk and using desktop system facilities to update the configuration file.
- By using a removable mass storage device supported by the target.

4. WIND RIVER DHCPV4: CLIENT

1. Introduction

This chapter describes the Wind River DHCPv4 client. For a general overview of DHCP, see DHCP Overview on page 3. For a list of implemented RFCs, see Supported RFCs on page 2.

A Wind River DHCPv4 client obtains an IP address and additional configuration information from a DHCPv4 server.

When a DHCPv4 client starts, it scans through all available interfaces and checks the interface flags on each one. If the interface is up (**IP_IFF_UP** flag is set) and its DHCP flag (**IP_IFF_X_DHCPRUNNING**) is set, the client opens a DHCP session over it.

The client initiates a DHCP session on an interface by broadcasting a **DHCPDISCOVER** message over the interface. It broadcasts the message repeatedly until it gets a response from a DHCPv4 server or until the preset number of attempts has exhausted. The server's response message contains an IP address for the client and a lease time that determines how long the address can be used. The client configures the interface with the address and keeps track of its lease time. The client contacts the server for lease renewal before the lease expires.

The response message from a DHCPv4 server typically contains a number of configuration parameters (DHCP options) for the client, such as an interface subnet mask, an interface broadcast address, a default gateway, DNS servers, a DNS domain, and vendor-specific options. For each option, before it does anything else, the client calls ipdhcpc_option_callback( ), a callback routine that you can implement to perform operations on DHCP options before any further processing by the client (see Implementing ipdhcpc_option_callback( ) on page 23).

The client also calls the ipdhcpc_reply_callback( ) callback routine, which you can implement to notify a user application about messages from the server, such as **ACK, NACK, OFFER**, and so on (see Implementing ipdhcpc_reply_callback( ) on page 28).

2. Configuring VxWorks for the DHCPv4 Client

To add the DHCPv4 client to your VSB project, see Source Build Configuration Options (VSB) on page 4.

To add the DHCPv4 client to your VIP, see Kernel Configuration Components (VIP) on page 4.

The **INCLUDE_IPDHCPC** component includes DHCPv4 client functionality to interface with DHCPv4 servers using the Dynamic Host Configuration Protocol. The component also includes the DHCPv4 client shell command (dhcpc).

To add optional components to your DHCPv4 client VIP environment, see Optional Kernel Configuration Components on page 5.

The DHCPv4 Client Configuration Parameters table on page 13 lists the DHCP client parameters and their default values.

DHCPv4 Client API Routines table on page 22 lists the API routines available to DHCP client application.

DHCP Options Implemented in the DHCPv4 Client table on page 22 lists the options that the DHCP client handles directly.

Macros for DHCP Options Not Implemented in the DHCPv4 Client table on page 25 lists macro names of DHCP option codes that are not implemented in the Wind River DHCP client, but that you can implement through the ipdhcpc_option_callback( ) API function.

Enabling DHCP in a Boot ROM

To enable DHCP in a boot ROM, include the following components:

- **INCLUDE_BOOT_DHCPC**
- **INCLUDE_IPNET_IFCONFIG_1**

  The **INCLUDE_IPNET_IFCONFIG_1** component has a default parameter of **IFCONFIG_1**. To enable DHCP, set the default parameter to the following string:

"ifname","devname driver","inet dhcp","gateway dhcp","inet6 3ffe:1:2:3::4/64"

For information on the bootDhcpcLoadInit( ) function, see *installDir* **/vxworks-7/pkgs_v2/os/utils/bootapp-**x.x.x.x **/src/bootDhcpcLoad.c**.

3. DHCPv4 Client Configuration Parameters

The DHCPv4 Client Configuration Parameters table on page 13 lists the DHCPv4 client configuration parameters and system variables.

Table 1. DHCPv4 Client Configuration Parameters

Parameter Name, System Variable Name, and Description	Default and Type
DHCPC_CFGFILE The full path of the configuration file that VxWorks will use to configure the DHCPv4 client parameters. When no file is specified (default setting), the DHCPv4 client parameter values are based on the VIP configuration. For more information, see File-Based Configuration for the DHCP Clients on page 8.	**""** char *
DHCPC_BOOTP_MODE **ipdhcpc.bootp_mode** Operate as a BOOTP client only. That is, do not append a DHCP "message type" option to requests. "1" to enable, "0" to disable.	**"0"** char *
DHCPC_AUTO_DISABLE **ipdhcpc.auto_disable** Disable DHCP on the interface if another address is configured on the interface, manually, after DHCP starts.	**"0"** char *
DHCPC_BOOT_FILENAME **ipdhcpc.boot_filename** Specify an optional filename to use in dhcp/bootp requests.	**""** char *

Parameter Name, System Variable Name, and Description	Default and Type
DHCPC_CLIENT_PORT **ipdhcpc.client_port** The port on which a DHCP client receives messages from the DHCP server. Enter the value as a quoted string.	**"68"** char *
DHCPC_DISCOVER_RETRIES **ipdhcpc.discover_retries** If the DHCP client sends a **DHCPDISCOVER** message to find a DHCP server and does not get a response, this parameter determines the maximum number of times the client retries.	**"4"** char *
DHCPC_FLAGS_BIT_BROADCAST **ipdhcpc.flags.bit.broadcast** Set the value of broadcast bit. A client that cannot receive unicast IP datagrams until its protocol software has been configured with an IP address should set the broadcast bit in the 'flags' field to 1 in any DHCPDISCOVER or DHCPREQUEST messages that the client sends. The broadcast bit will provide a hint to the DHCP server and BOOTP relay agent to broadcast any messages to the client on the client's subnet. A client that can receive unicast IP datagrams before its protocol software has been configured should clear the BROADCAST bit to 0.	**"0"** char *
DHCPC_OFFER_TIMEOUT **ipdhcpc.offer_timeout** The length of time, in milliseconds, that the DHCP client waits for a response to a **DHCPDISCOVER** message before it resends the message.	**"2000"** char *
DHCPC_SERVER_HOSTNAME **ipdhcpc.server_hostname** Specify an optional server hostname to use in dhcp/bootp requests.	**""** char *
DHCPC_SERVER_PORT **ipdhcpc.server_port** The DHCP server port to which the client sends messages. Enter the value as a quoted string.	**"67"** char *
DHCPC_TTL ipdhcpc.ttl Used to set time to live (TTL) for DHCP packets. The value is in seconds.	**"128"** char *

Parameter Name, System Variable Name, and Description	Default and Type

DHCPC_IF_INFORMATION_ONLY_LIST
ipdhcpc.if.interfaceName.information_only
Specifies whether an interface only requests information. For each interface, enter 'enable' or 'disable.' If it is enabled, the DHCPC_REMOVE_EXISTING_ADDRESSES parameter must be zero.
The following is an example using the configuration parameter:
`"eth0=enable"`
The following is an equivalent example using the system variable:
`sysvar set ipdhcpc.if.eth0.information_only enable`
Note: Set this parameter before enabling DHCP.
	""
	char *

DHCPC_CLIENT_UUID **ipdhcpc.option.client_identifier_uuid** Specifies the data type of the DHCPC_CLIENT_ID parameter (a string or a character array). When set to "**false**" , DHCPC_CLIENT_ID will be treated as a string. Use this setting, for example, when DHCPC_CLIENT_ID is set to "**phantastica**". When set to "**true**", DHCPC_CLIENT_ID will be treated as a character array. Use this setting, for example, when DHCPC_CLIENT_ID is set to "**4C:43:5F:52:41:43:4B:5F:44:00:22:11:5F:01:5F:00**".	**false** char *
DHCPC_CLIENT_ID **ipdhcpc.option.client_identifier** The name that the client wants the DHCP server to use to identify it. This parameter can be used with two different data types (a string or a character array). When it is used with a quoted string (for example: "**phantastica**"), the DHCPC_CLIENT_UUID parameter must be set to "**false**" (the default). When it is used with a character array holding the UUID itself (for example, "4C:43:5F:52:41:43:4B:5F:44:00:22:11:5F:01:5F:00"), the DHCPC_CLIENT_UUID parameter must be set to "**true**" (the default).	**NULL** char *
DHCPC_OPTION_HOSTNAME **ipdhcpc.option.hostname** Defines the hostname option to use in requests.	"" char *
DHCPC_OPTION_MAX_MESSAGE_SIZE **ipdhcpc.option.max_message_size** Maximum message size this client is prepared to accept. Must be at least 576 as per RFC 2131.	**"576"** char *

Parameter Name, System Variable Name, and Description	Default and Type
If the whole length of message size is larger than **DHCPC_OPTION_MAX_MESSAGE_SIZE**, some user configured options will be omitted in packets sent by the dhcp client. The user configured options are those that can be configured by the client with sysvars that take the form **ipdhcpc.option.***option*, such as ipdhcpc.option.netmask.	
DHCPC_OPTION_NETMASK **ipdhcpc.option.netmask** Defines the subnet mask option to use in requests.	" " char *
DHCPC_REQ_OPTS **ipdhcpc.option.requested_options** Specifies a comma-delimited list of DHCP option numbers (see RFC 2132) that the DHCP client wants the DHCP server to provide. For example, the following entry asks the server to provide a subnet mask, IP addresses for domain name servers, a broadcast address, and the IP address of an NTP server: "1,6,28,42".	**NULL** char *
DHCPC_OPTION_USER_SPECIFIED **ipdhcpc.option_user_specified** Defines a list of options and associated data not supported explicitly by other option configuration parameters. Expected format: *option_number=option_data*. Each pair of *option_number=option_data* is semicolon-separated. In addition, *option_data* must be specified in hex with network byte order—e.g., **"1=FF000000;12=7678486f7374"** specifies a netmask option of 255.0.0.0 and a hostname option of vxHost. Note that if an option specified in the **DHCP_OPTION_USER_SPECIFIED** is also defined in an explicitly supported configuration option, the explicitly supported configuration option takes precedence. Note: Do not use **DHCPC_OPTION_USER_SPECIFIED** to set the DHCP Message Type (53), DHCP Client Identifier (61), or DHCP Max Message Size (57) options. These three options are restricted. Instead, use separate, fully supported sysvars to set these options.	**""** char *
DHCPC_OPTION_VENDOR_CLASS_ID **ipdhcpc.option.vendor_class_id** Defines the vendor class ID option.	**""** char *
DHCPC_OPTION_VENDOR_VI_CLASS **ipdhcpc.option.vendor_identifying_vendor_class** Defines the vendor identifying vendor class option. Use the following format for the configuration parameter: *enterpriseNumber=data1;enterpriseNumber=data1; . . .*	**""** char *

Parameter Name, System Variable Name, and Description	Default and Type
DHCPC_OPTION_IPADDRESS_LEASE_TIME **ipdhcpc.option.lease_time** Specify how long you want to keep the client IP address active by entering the lease time in seconds. Otherwise, use the default value of 0 to have the server define the lease time based on what is available.	**"0"** char *
DHCPC_INSTALL_OPTION_CALLBACK_HOOK If set to TRUE, you can implement the **ipdhcpc_option_callback( )** function and include it in the build. If set to FALSE, the default version of the callback, which does not perform any operations, is called. For information on **ipdhcpc_option_callback( )**, (see Implementing ipdhcpc_option_callback( ) on page 23.	**FALSE** Bool
DHCPC_INSTALL_REPLY_CALLBACK_HOOK If set to **TRUE**, you can implement the **ipdhcpc_reply_callback( )** function and include it in the build. If set to **FALSE**, the default version of the callback is called. For information on **ipdhcpc_reply_callback( )**, see Implementing ipdhcpc_reply_callback( ) on page 28.	**FALSE** Bool

DHCPC_IF_CLIENT_ID_LIST
ipdhcpc.interfaceName.option.client_identifier
The name or names the client wants the DHCP server to use to identify it on an interface or interfaces.
Use the following format for the configuration parameter:
interfaceName=clinetID;interfaceName=clientID;interfaceName=clientID; . . .
where interfaceName is the name of an interface and clientID is a character string identifying the client on an interface. For example:
`"eth0=id1;eth1=id2;eth2=id3"`
For the sysvar, use the following format:
`sysvar set ipdhcpc.`*interfaceName*`.option.client_identifier clientID`
For example:
`sysvar set ipdhcpc.eth0.client_identifier id1`
	""
	char *

DHCPC_IF_REQ_OPTS_LIST **ipdhcpc.interfaceName.option.requested_options** The DHCP options that the client wants the server to provide to an interface or interfaces. Use the following format for the configuration parameter: *interfaceName=options;interfaceName=options;interfaceName=options; . . .* where interfaceName is the name of an interface and options is a comma separated list of DHCP options (see RFC 2132). For example:	**""** char *

Parameter Name, System Variable Name, and Description	Default and Type

`"eth0=1,6,28,42;eth1=1,3,5,40,41"`
For the sysvar, use the following format:
`sysvar set ipdhcpc.`*interfaceName* `.option.requested_options options`
For example:
`sysvar set ipdhcpc.eth0.requested_options 1,6,28,42`	

DHCPC_OPTION_CALLBACK_HOOK If the Install dhcp client option callback routine (DHCPC_INSTALL_OPTION_CALLBACK_HOOK) parameter is set to TRUE, you must set this parameter to point to your implementation of the ipdhcpc_option_callback routine. With the ipdhcpc_option_callback function, you can define your own operations on individual DHCP options (see Implementing ipdhcpc_option_callback( ) on page 23.	**ipdhcpc_option_default_callback** funcptr
DHCPC_RFC2131_EXP_BACKOFF **ipdhcpc.rfc2131_exponential_backoff** Either "1" or "0". If set to "1", the DHCP client waits a random number of seconds before resending **DHCPREQUEST** messages. If set to "0" there is no delay.	**"1"** char *
DHCPC_RFC2131_INIT_DELAY **ipdhcpc.rfc2131_init_delay** Either "1" or "0". If set to "1", the DHCP client waits a random number of seconds before resending **DHCPDISCOVER** messages to locate DHCP servers. If set to "0" there is no delay.	**"1"** char *
DHCPC_REMOVE_EXISTING_ADDRESSES **ipdhcpc.remove_existing_addresses** Remove all unicast addresses from the interface before a **DHCP negotiation** begins. A value of "1" removes the addresses, a value of "0" retains them.	**"1"** char *
DHCPC_REMOVE_LEASED_ADDRESS **ipdhcpc.remove_leased_address** Remove any address from the interface if the lease expires. A value of "1" removes them, a value of "0" retains them. If set to "1" then also disable DHCP and remove any negotiated address from the interface.	**"1"** char *

Parameter Name, System Variable Name, and Description	Default and Type
DHCPC_REPLY_CALLBACK_HOOK If the Install dhcp client reply callback routine (DHCPC_INSTALL_REPLY_CALLBACK_HOOK) parameter is set to TRUE, you must set this parameter to point to your implementation of the **ipdhcpc_reply_callback( )** function. With the **ipdhcpc_reply_callback( )** function, you can define your own operations on individual DHCP reply messages. (see Implementing ipdhcpc_reply_callback( ) on page 28).	**ipdhcpc_reply_default_callback** funcptr

4. Shell Commands

You can use the dhcpc shell command to renew a lease, obtain information on an interface or IP address, and list the current lease and interface information for a DHCPv4 client.

You can use the ipd shell command to start and stop the DHCPv4 client daemon.

You can use the ifconfig shell command to enable or disable DHCP on an interface and to display the current status of an interface.

You can use the syslog shell command to set the priority for DHCPv4 client syslog debug messages.

To run shell commands, you must access the command-interpreter mode. To access the command-interpreter mode from the kernel shell, enter the following command:

```
-> cmd
[vxWorks *} #
```

To return to the kernel shell, enter the following command:

```
# C
->
```

Using DHCPv4 Client Shell Commands

To obtain a list of dhcpc shell commands and help for the commands, enter the following command:

```
# dhcpc
```

The available commands are as follows:

```
dhcpc renew {name }
dhcpc inform {name } [addr ]
dhcpc lease list
dhcpc interface list
```

Stopping and Starting the DHCPv4 Client Daemon

Use the ipd command to stop or start the DHCPv4 client daemon (**ipdhcpc**). For example:

```
# ipd list
Services:
ipdhcpc started
```

```
# ipd kill ipdhcpc
ipd: kill ipdhcpc ok
```

```
# ipd list
Services:
ipdhcpc killed
```

```
# ipd start ipdhcpc
ipd: start ipdhcpc ok
```

To view the task information for the DHCPv4 client, run the task command and locate the **ipdhcpc** task. For example:

```
# task
NAME ENTRY TID PRI STATUS PC SP ERRNO DELAY
...
ipdhcpc 10023f30 1049b4a0 50 PEND+T 102b87eb 1096fd60 3d0004 56
```

Disabling and Enabling DHCPv4 on an Interface

To disable DHCPv4 on an individual interface, use the following command syntax:

```
# ifconfig interfaceName -dhcp
```

Closes the current session on interface interfaceName, disables DHCP auto negotiation on it, and removes any negotiated address from the interface.

The interface is no longer flagged as a dhcp interface, and no renewal attempt is performed. However, the IP address is fully functional, as if it had been statically configured.

To enable DHCPv4 on an individual interface, use the following command syntax:

```
# ifconfig interfaceName dhcp
```

Enables the specified interface for DHCPv4 and starts a DHCP session on it (auto negotiation).

To view the status of a DHCPv4 interface, use the following command syntax:

```
# ifconfig interfaceName
```

For example:

```
# ifconfig eth0

eth0 Link type:6 HWaddr 00:0a:01:02:44:02
inet 10.1.1.231 broadcast 10.1.255.255 mask 255.255.0.0
inet6 unicast FEC0::1:20A:1FF:FE02:4402
inet6 unicast FEC0::A9:20A:1FF:FE02:4402
inet6 unicast FEC0::1:0:0:0:82
inet6 multicast FF02::1%eth0
```

```
inet6 unicast FE80::20A:1FF:FE02:4402%eth0
UP RUNNING BROADCAST MULTICAST DHCP
MTU:1500 metric:1 rtab:0
RX packets:62 mcast:1 errors:0 dropped:2
TX packets:11 mcast:3 errors:0
collisions:0 unsupported proto:45
RX bytes:4989 TX bytes:1562
```

Specifying Use Options

If you have already started a DHCP negotiation and want to include some additional options with sysvar, you must first disable DHCPv4 auto negotiation on the interface. To do so, use the following command syntax:

```
# ifconfig interfaceName -dhcp
```

Then set the desired option, using a sysvar command. For example:

```
# sysvar set -c ipdhcpc.option.user_specified "1=FF000000;12=7678486f7374"
```

Then verify the setting with the following command:

```
# sysvar get ipdhcpc.option.user_specified
```

When you have finished setting user options, restart DHCPv4 auto negotiation with the following command syntax:

```
# ifconfig interfaceName dhcp
```

Existing, supported parameters, whether set by a configuration component or a sysvar, override any duplicate options specified with **DHCPC_OPTION_USER_SPECIFIED**. For example, if you define **DHCPC_OPTION_NETMASK**, but also specify this option in the list of **DHCPC_OPTION_USER_SPECIFIED** options, the **DHCPC_OPTION_NETMASK** option and associated data will be the ones used in the DHCP client packet.

Setting the DHCPv4 Client System Log Prioriy

Run the **syslog list** command to get the current priority level for all modules:

```
# syslog list

syslog facility priority
...
ipdhcpc Error
```

To list the available syslog priority levels, use the following command:

```
# syslog priority *

syslog: missing <prio>
priorities: Emerg Crit Error Warning Notice Info Debug Debug2
```

Use the following command syntax to change the priority level for the DHCPv4 client:

```
# syslog priority ipdhcpc priorityLevel
```

For more information on the syslog shell command, see the *VxWorks 7 Network Stack Programmer's Guide*.

5. API Routines

For more information, see the reference entries for the DHCPv4 client (**ipdhcpc**).

The DHCPv4 Client API Routines table on page 22 lists the API routines available to DHCPv4 client applications and gives a brief description of each routine.The DHCPv4 client public API routines are defined in **ipdhcpc.c**.

Table 1. DHCPv4 Client API Routines

Routine	Description
ipdhcpc_inform_get( )	Send an INFORM message to the DHCP server.
ipdhcpc_option_callback( )	Apply user-defined actions based on a DHCP option.
ipdhcpc_renew_get( )	Send a RENEW message before lease expiration.
ipdhcpc_reply_callback( )	Apply user-defined actions based on a DHCP reply.

6. DHCPv4 Client Options

The Wind River DHCPv4 client implements client portions of RFC 2131: *Dynamic Host Configuration Protocol*, and recognizes options and vendor extensions in RFC 2132: *DHCP Options and BOOTP Vendor Extensions*, and other specifications. In some cases, the client explicitly handles an option. In other cases, the client has no preexisting implementation for handling an option, but allows you to implement operations for handling the option in the ipdhcpc_option_callback( )(routine (see Implementing ipdhcpc_option_callback( ) on page 23. If you want the DHCPv4 client to handle options that are not listed in DHCP Options Implemented in the DHCPv4 Client table on page 22, you must handle them in the ipdhcpc_option_callback( ) routine (see Implementing ipdhcpc_option_callback( ) on page 23).

Implementation of DHCP Options

The DHCP Options Implemented in the DHCPv4 Client table on page 22 lists the DHCPv4 client options that the client handles directly.

Table 1. DHCP Options Implemented in the DHCPv4 Client

Option Code	Macro Name
0	**IPDHCPC_OPTCODE_PAD**
1	**IPDHCPC_OPTCODE_SUBNET_MASK**
3	**IPDHCPC_OPTCODE_ROUTERS**
6	**IPDHCPC_OPTCODE_DOMAIN_NAME_SERVERS**

Option Code	Macro Name
12	**IPDHCPC_OPTCODE_HOST_NAME**
15	**IPDHCPC_OPTCODE_DOMAIN_NAME**
42	**IPDHCPC_OPTCODE_NTP_SERVERS**
51	**IPDHCPC_OPTCODE_DHCP_LEASE_TIME**
53	**IPDHCPC_OPTCODE_DHCP_MESSAGE_TYPE**
54	**IPDHCPC_OPTCODE_DHCP_SERVER_IDENTIFIER**
55	**IPDHCPC_OPTCODE_DHCP_PARAMETER_REQUEST_LIST**
56	**IPDHCPC_OPTCODE_DHCP_MESSAGE**
57	**IPDHCPC_OPTCODE_DHCP_MAX_MESSAGE_SIZE**
58	**IPDHCPC_OPTCODE_DHCP_RENEWAL_TIME**
59	**IPDHCPC_OPTCODE_DHCP_REBINDING_TIME**
60	**IPDHCPC_OPTCODE_VENDOR_CLASS_IDENTIFIER**
61	**IPDHCPC_OPTCODE_DHCP_CLIENT_IDENTIFIER**
124	**IPDHCPC_OPTCODE_VI_VENDOR_IDENTIFIER**

Note:

The **IPDHCPC_MAX_USER_OPT_SIZE** value defines the amount of memory to reserve for the options field in an outgoing DHCP message (see RFC2131). The default value is 100 octets and may be insufficient for some applications. You can change **IPDHCPC_OPTION_MAX_MESSAGE_SIZE** at build time from the **ipdhcpc_config.h** file or at run time with the **ipdhcpc.option.max_message_size**sysvar.

If you set **IPDHCPC_MAX_USER_OPT_SIZE** to a value greater than 312 octets, you must also increase the value of **IPDHCPC_OPTION_MAX_MESSAGE_SIZE** accordingly.

The default value of 576 octets (the minimum IP datagram size an IP host must be prepared to accept based on RFC 1122) allows for up to 312 octets for the **IPDHCPC_MAX_USER_OPT_SIZE, 236** octets for the remaining fields in a minimal DHCP message, 20 octets for a minimal IP header, and 8 octets for a UDP header.

Implementing ipdhcpc_option_callback( )

There are many DHCP options that the DHCPv4 client does not handle directly (see DHCP Options Not Initially Implemented in the DHCPv4 Client on page 25, Macros for DHCP Options Not Implemented in the DHCPv4 Client table on page 25). You can implement your own handler for these options through the ipdhcpc_option_callback( ) routine. You can also implement this routine to provide special treatment of any of the DHCP options that the client already handles. For a list of those options, see DHCPv4 Client Options on page 22, DHCP Options Implemented in the DHCPv4 Client table on page 22.

For each option in a DHCP server message, the client calls ipdhcpc_option_callback( ) before it performs any other action involving the option. The signature of ipdhcpc_option_callback( ) is defined in **ipdhcpc.h**.

The handle parameter points to a descriptor, for the DHCP session, which contains information such as the index of the interface the option arrived on, the client address, the server address, and so on. The **Ipdhcpc_handle** structure is defined in **ipdhcpc.h**.

The option parameter points to an option in a DHCP server message, where:

- option[0] is a DHCP option code.
- option[1] is the length of option, in bytes.
- option[2] is the first element containing data for the option.

Write your ipdhcpc_option_callback( ) function to return one of the following:

TRUE

Indicates that the DHCP client should continue to process the option.

FALSE

Indicates that the DHCP client should do nothing further with the option.

If you do not implement the ipdhcpc_option_callback( ) function, the client uses a default version of the routine that performs no operations on any options and returns **TRUE**. See the Option Callback Routine (DHCPC_OPTION_CALLBACK_HOOK) parameter in DHCPv4 Client Configuration Parameters table on page 13 and ipdhcpc_option_default_callback( ) in **ipdhcpc_example.c**.

Example Implementation of ipdhcpc_option_callback( )

The following implementation of the ipdhcpc_option_callback( ) function checks the option parameter to see if it contains the DHCP option code for specifying a host name—**IPDHCPC_OPTCODE_HOST_NAME**. If it does, the routine assigns the host name specified in the option parameter to the client.

```
/*
 * DHCPC_INSTALL_OPTION_CALLBACK_HOOK = TRUE
 * DHCPC_OPTION_CALLBACK_HOOK = myDhcpClientHook
 */

#include <ipcom_type.h>
#include <ipdhcpc_config.h>
#include <ipdhcpc.h>

Ip_err myDhcpClientHook (Ipdhcpc_handle * handle, Ip_u8 * option)
  {
   char myHostName[20];
   unsigned char length;
   (void)handle;

switch (*option)
    {
    case IPDHCPC_OPTCODE_HOST_NAME:
         gethostname (myHostName, 20);
         printf ("Host name before change is %s\n", myHostName);
         option++;
         length = *option;
         option++;
         strcpy (myHostName, (char *)option);
         myHostName[length] = '\0'; /* Insert \0 at the end */
         sethostname (myHostName, length);
         gethostname (myHostName, 20);
         printf ("Host name changed to %s\n", myHostName);
```

```
        break;
    default:
        printf ("Got option %d. Do nothing with it\n", *option);
        break;
    }
}
```

7. DHCP Options Not Initially Implemented in the DHCPv4 Client

Macros for DHCP Options Not Implemented in the DHCPv4 Client table on page 25 lists macro names of DHCP option codes that are not implemented in the Wind River DHCPv4 client. For information on how to set these options, see Setting DHCP Client Options Not Explicitly Supported on page 27.

Table 1. Macros for DHCP Options Not Implemented in the DHCPv4 Client

Macro for DHCP Option Code	Code	Option Value Type
IPDHCPC_OPTCODE_ALL_SUBNETS_LOCAL	27	boolean
IPDHCPC_OPTCODE_ARP_CACHE_TIMEOUT	35	32-bits
IPDHCPC_OPTCODE_BOOT_SIZE	13	16-bits
IPDHCPC_OPTCODE_BROADCAST_ADDRESS	28	IPv4 address
IPDHCPC_OPTCODE_CLIENT_IDENTIFIER	61	(varies)
IPDHCPC_OPTCODE_COOKIE_SERVERS	8	list of IPv4 addresses
IPDHCPC_OPTCODE_DEFAULT_IP_TTL	23	8-bits
IPDHCPC_OPTCODE_DEFAULT_TCP_TTL	37	8-bits
IPDHCPC_OPTCODE_DHCP_OPTION_OVERLOAD	52	8-bits
IPDHCPC_OPTCODE_DHCP_REQUESTED_ADDRESS	50	IPv4 address
IPDHCPC_OPTCODE_EXTENSIONS_PATH	18	string
IPDHCPC_OPTCODE_FINGER_SERVERS	73	list of IPv4 addresses
IPDHCPC_OPTCODE_FONT_SERVERS	48	list of IPv4 addresses
IPDHCPC_OPTCODE_HOME_AGENTS	68	list of IPv4 addresses
IPDHCPC_OPTCODE_IEEE802_3_ENCAPSULATION	36	boolean
IPDHCPC_OPTCODE_IMPRESS_SERVERS	10	list of IPv4 addresses

Macro for DHCP Option Code	Code	Option Value Type
IPDHCPC_OPTCODE_INTERFACE_MTU	26	16-bits
IPDHCPC_OPTCODE_IP_FORWARDING	19	boolean
IPDHCPC_OPTCODE_IRC_SERVERS	74	list of IPv4 addresses
IPDHCPC_OPTCODE_LOG_SERVERS	7	list of IPv4 addresses
IPDHCPC_OPTCODE_LPR_SERVERS	9	list of IPv4 addresses
IPDHCPC_OPTCODE_MASK_SUPPLIER	30	boolean
IPDHCPC_OPTCODE_MAX_DGRAM_REASSEMBLY	22	16-bits
IPDHCPC_OPTCODE_MERIT_DUMP	14	string
IPDHCPC_OPTCODE_NAME_SERVERS	5	list of IPv4 addresses
IPDHCPC_OPTCODE_NETBIOS_DD_SERVER	45	list of IPv4 addresses
IPDHCPC_OPTCODE_NETBIOS_NAME_SERVERS	44	list of IPv4 addresses
IPDHCPC_OPTCODE_NETBIOS_NODE_TYPE	46	8-bits
IPDHCPC_OPTCODE_NETBIOS_SCOPE	47	string
IPDHCPC_OPTCODE_NIS_DOMAIN	40	string
IPDHCPC_OPTCODE_NIS_SERVERS	41	list of IPv4 addresses
IPDHCPC_OPTCODE_NON_LOCAL_SOURCE_ROUTING	20	boolean
IPDHCPC_OPTCODE_PATH_MTU_AGING_TIMEOUT	24	32-bits
IPDHCPC_OPTCODE_PATH_MTU_PLATEAU_TABLE	25	list of 16-bit values
IPDHCPC_OPTCODE_PERFORM_MASK_DISCOVERY	29	boolean
IPDHCPC_OPTCODE_POLICY_FILTER	21	list of IPv4 address pairs
IPDHCPC_OPTCODE_RESOURCE_LOCATION_SERVERS	11	list of IPv4 addresses
IPDHCPC_OPTCODE_ROOT_PATH	17	string
IPDHCPC_OPTCODE_ROUTER_DISCOVERY	31	boolean
IPDHCPC_OPTCODE_ROUTER_SOLICITATION_ADDRESS	32	IPv4 address

Macro for DHCP Option Code	Code	Option Value Type
IPDHCPC_OPTCODE_STATIC_ROUTES	33	list of IPv4 address pairs
IPDHCPC_OPTCODE_SWAP_SERVER	16	IPv4 address
IPDHCPC_OPTCODE_TCP_KEEPALIVE_GARBAGE	39	boolean
IPDHCPC_OPTCODE_TCP_KEEPALIVE_INTERVAL	38	32-bits
IPDHCPC_OPTCODE_TIME_OFFSET	2	32-bits
IPDHCPC_OPTCODE_TIME_SERVERS	4	list of IPv4 addresses
IPDHCPC_OPTCODE_TRAILER_ENCAPSULATION	34	boolean
IPDHCPC_OPTCODE_VENDOR_ENCAPSULATED_OPTIONS	43	string
IPDHCPC_OPTCODE_X_DISPLAY_MANAGER	49	list of IPv4 addresses

Setting DHCP Client Options Not Explicitly Supported on page 27DHCP client options not supported explicitly by configuration parameters can be set at VIP configuration time with a DHCP Client parameter, or at run time, either with the sysvar shell command or programmatically with a function imported into your VIP.

7.1. Setting DHCP Client Options Not Explicitly Supported

DHCP client options not supported explicitly by configuration parameters can be set at VIP configuration time with a DHCP Client parameter, or at run time, either with the sysvar shell command or programmatically with a function imported into your VIP.

Setting the Options at VIP Configuration Time

At VIP configuration time, you can set these options from the DHCP Client component with the **Global Option: User Specified List** (DHCPC_OPTION_USER_SPECIFIED) parameter.

The parameter format is *optionNumber*=*optionValue* where each listed option and value pair is separated by a semicolon. The *optionValue* must be specified in hexadecimal.

For example, to configure option IPDHCPC_OPTCODE_DHCP_REQUESTED_ADDRESS (option 50) with address **192.168.1.1** and option IPDHCPC_OPTCODE_TIME_SERVERS (option 4) with address **192.168.1.2**, enter the following string as the **Global Option: User Specified List** (DHCPC_OPTION_USER_SPECIFIED) parameter:

"50=C0A80101;4=C0A80102"

Setting the Options at Run Time with a System Variable

At run time, you can configure these options with the ipdhcpc.option.user_specified system variable.

For example, to configure option 50 with address **192.168.1.1** and option 4 with address **192.168.1.2**, use the following command:

```
# sysvar set -co ipdhcpc.option.user_specified 50=C0A80101, 4=C0A80102
```

To view the configuration, use the following command:

```
# sysvar list ipdhcpc
…
ipdhcpc.option.user_specified=50=C0A80101,4=C0A80102
…
```

Setting the Options Programmatically

You can also configure these options programmatically by importing the following function into your VIP. You then execute the function from the kernel shell or from your application.

```
#include <vxworks.h>
#include "ipcom_cmd.h"
#include "ipcom_cstyle.h"
#include "ipcom_type.h"
#include <ipcom_sysvar.h>

STATUS setOptions (const char *name,const char *value, int flags)
{
     char buf[256];
     int stat;

     ipcom_snprintf(buf, sizeof(buf), "ipdhcpc.option.%s", name);
     stat=ipcom_sysvar_set(buf, value, flags);

     return stat;
}
```

For example, to configure option 50 with address **192.168.1.1** and option 4 with address **192.168.1.2**, use the following command from the kernel shell:

```
-> sp setOptions,"user_specified","50=C0A8010, 4=C0A80102",1
```

To view the configuration from the kernel shell, use the following command:

```
-> cmd sysvar list ipdhcpc
…
ipdhcpc.option.user_specified=50=C0A80101,4=C0A80102
…
```

Parent topic: DHCP Options Not Initially Implemented in the DHCPv4 Client on page 25

8. Implementing ipdhcpc_reply_callback( )

You can implement your own handler for server messages, through the ipdhcpc_reply_callback( ) function, to provide special treatment of any of the DHCP replies that the client already handles.

For each DHCPv4 server message, the DHCPv4 client calls ipdhcpc_reply_callback( ) before it performs any other action involving the message. The signature of ipdhcpc_reply_callback( ) is defined in **ipdhcpc.h.**

Event Parameter

The event parameter contains the message type of a DHCP server message, and can be one of the following values:

IPDHCPC_REPLY_CB_EVENT_OFFER

When this callback is issued for an **IPDHCPC_REPLY_CB_EVENT_OFFER** event, the return value **IPDHCPC_REPLY_CB_ERROR** does not affect the state machine. The state machine progresses to the next state, **REQUESTING**, and a **DHCPREQUEST** is sent. This can be used to examine the offered address against a specific user criteria.

IPDHCPC_REPLY_CB_EVENT_BOOTREPLY

When this callback is issued for an **IPDHCPC_REPLY_CB_EVENT_BOOTREPLY** event, if **IPDHCPC_REPLY_CB_ERROR** is returned, the state machine will progress to INIT state.

Return values **IPDHCPC_REPLY_CB_ERROR**, **IPDHCPC_REPLY_CB_SUCCESS** do not affect the state machine.

IPDHCPC_REPLY_CB_EVENT_ACK

When this callback is issued for an **IPDHCPC_REPLY_CB_EVENT_ACK** event, if **IPDHCPC_REPLY_CB_ERROR** is returned, the state machine progresses to **INIT** state and a **DHCPDELINE** message is sent.

IPDHCPC_REPLY_CB_EVENT_NAK

Return values **IPDHCPC_REPLY_CB_ERROR**, **IPDHCPC_REPLY_CB_SUCCESS** do not affect the state machine.

IPDHCPC_REPLY_CB_EVENT_EXPIRE or **IPDHCPC_REPLY_CB_EVENT_FAIL_IFLOST**

Return values **IPDHCPC_REPLY_CB_ERROR**, **IPDHCPC_REPLY_CB_SUCCESS** do not affect the state machine.

Return Value

IPDHCPC_REPLY_CB_ERROR indicates that the user application has problems with some of the parameters in the reply. If **IPDHCPC_REPLY_CB_ERROR** is returned, the state machine may change depending on the option, as noted in Event Parameter on page 29.

If you do not implement the ipdhcpc_reply_callback( ) fuction, the client uses a default version of the routine that performs no operations on any options and returns **IPDHCPC_REPLY_CB_SUCCESS**. See the Reply Callback Routine (DHCPC_REPLY_CALLBACK_HOOK) parameter in the DHCPv4 Client Configuration Parameters on page 13 table and ipdhcpc_reply_default_callback( ) in **ipdhcpc_example.c**.

A Sample Implementation of ipdhcpc_reply_callback( )

The following sample implementation of the ipdhcpc_reply_callback( ) routine checks the event parameter to see if it contains an **ACK** or **BOOTREPLY** and prints the interface it arrived on and the offered address.

```
 /*
 * DHCPC_INSTALL_REPLY_CALLBACK_HOOK = TRUE
 * DHCPC_REPLY_CALLBACK_HOOK = myDhcpClientReplyHook
 */

#include <ipdhcpc.h>
#include <ipcom_err.h>

Ip_err myDhcpClientReplyHook(Ipdhcpc_handle *handle, int event)
{
```

```
    char yiaddr[16];

    Ip_err ret = IPDHCPC_REPLY_CB_SUCCESS;
    if (handle == NULL)
    {
        ret = IPCOM_ERR_FAILED;
    }
    else
    {
        ret = IPCOM_SUCCESS;


        switch(event)
        {
            case IPDHCPC_REPLY_CB_EVENT_OFFER:
                /* Informational only in this implementation. */
                printf("ipdhcpc_reply_cb :: received OFFER on interface '%d'.\n", handle->ifindex)
;
                printf("server port is %d\n", handle->serverport_n);
                printf("magic cookie is 0x%lx\n", handle->magic_cookie);
                ipcom_inet_ntop(AF_INET, &handle->info.ipaddr_n, yiaddr, sizeof(yiaddr));
                printf("IP addr offered is %s\n", yiaddr);
                printf("\n");
                break;

            case IPDHCPC_REPLY_CB_EVENT_ACK:
            case IPDHCPC_REPLY_CB_EVENT_BOOTREPLY:
                printf("%s: ACK on interface '%d'\n", __FUNCTION__, handle->ifindex);
                printf("server port is %d\n", handle->serverport_n);
                printf("magic cookie is 0x%lx\n", handle->magic_cookie);
                ipcom_inet_ntop(AF_INET, &handle->info.ipaddr_n, yiaddr, sizeof(yiaddr));
                printf("IP addr offered is %s\n", yiaddr);
                printf("\n");
                break;

            case IPDHCPC_REPLY_CB_EVENT_NAK:
                /* Informational only in this implementation. */
                printf("ipdhcpc_reply_cb :: received NAK on interface '%d'.\n", handle->ifindex);
                break;

            case IPDHCPC_REPLY_CB_EVENT_EXPIRE:
                /* Informational only in this implementation. */
                printf("ipdhcpc_reply_cb :: Lease expired on interface '%d'.\n", handle->ifindex);
                break;

            case IPDHCPC_REPLY_CB_EVENT_FAIL_RETRY:
                printf("ipdhcpc_reply_cb :: Retries expired on interface '%d'.\n", handle->ifindex
);
                break;

            case IPDHCPC_REPLY_CB_EVENT_FAIL_IFLOST:
                printf("ipdhcpc_reply_cb :: Interface '%d' lost.\n", handle->ifindex);
                break;
            default:
                break;
        }
    }
```

```
    return ret;
}
```

5. WIND RIVER DHCPV4: SERVER

1. Introduction

This chapter describes the Wind River DHCPv4 server implementation.

For a general overview of DHCP, see DHCP Overview on page 3. For a list of implemented RFCs, see Supported RFCs on page 2.

The Wind River DHCPv4 server provides IP addresses and option settings to DHCPv4 clients. The option settings are of two types:

- DHCP options, as defined in RFC 2132 and RFC 2242, see Standard DHCPv4 Server Options table on page 47.
- Wind River-specific configuration options, see Wind River-Specific DHCPv4 Server Configuration Options table on page 54.

You can configure the information the DHCPv4 server provides to clients in three ways:

Set global defaults

You can set defaults for all options. You can also specify a default set of IP addresses if your DHCPv4 server provides addresses only to clients on a single subnet.

Assign IP addresses and options on a subnet-by-subnet basis, dynamically

For each subnet your DHCPv4 server covers, you can specify options and a set of addresses for clients on that subnet. The option settings you specify for a subnet override any global default settings for the same options.

Assign IP addresses and options to individual hosts, statically

You can also assign a static IP address and an individualized set of options to individual clients. The option settings you specify for an individual host override any global default settings or subnet settings for the same options.

You can configure information about address pools, options, subnets, and hosts, both statically at build time and dynamically at run time, see Setting Up Addresses, Options, Subnets and Hosts on page 41.

2. Server Components

The Wind River DHCPv4 server includes a configuration database, a lease database, a single process, and the DHCPS server daemon.

DHCPS Daemon

The DHCPS daemon handles client requests in the following way:

1. It checks each packet received to ensure that it is a DHCP or BOOTP packet.
2. It looks for a subnet in its configuration database that matches the client's subnet. If the request came directly from the client, the server interprets the subnet of the interface on which the request was received as the client's subnet. If the server receives the request through a relay agent, the server interprets the relay agent's subnet as the client's subnet.

3. It scans its lease database for a lease matching the client's identification key (which is either the client's link-layer address or a client identifier). If the server finds such a lease, it responds to the client's request based on the lease. If the server does not find a lease, it offers the client a new lease based on the client's subnet and the client's request.

Configuration Database

The configuration database stores the information you configure for global default values, subnets, individual hosts, and options.

Lease Database

The lease database contains all leases currently in effect and all leases previously used by clients that are now either expired or released. The server uses this database to keep track of the state of clients with active leases and to give clients whose leases are no longer in effect the same addresses and options they had in the past.

You can save and restore this lease database so that the server does not have to rebuild it from scratch each time it starts up. The ipdhcps_lease_db_dump( ) function dumps the database to a buffer (you need to implement additional operations to write this database buffer to a file). The ipdhcps_lease_db_restore( ) function restores the database from a buffer (you need to implement whatever operations are necessary to get the database data into the buffer in the first place).

To restore the database at startup, implement the ipdhcps_start_hook( ) function to call ipdhcps_lease_db_restore( ). See Example: Restoring the Lease Database at Startup on page 43. The server calls ipdhcps_start_hook( ) when it starts.

To dump the database when the server shuts down, implement the ipdhcps_stop_hook( ) function to call ipdhcps_lease_db_dump( ). See Example: Dumping the Lease Database at Shutdown on page 46. The server calls ipdhcps_stop_hook( ) when you shut the server down.

3. Relay Agent Information Option

The relay agent information option (option 82) feature enables DHCPv4 servers to implement security and IP address assignment policies based on information received by a relay agent or a bridge. The DHCPv4 server can also use this information to perform access control and set quality of service (QoS) for each client. For more information on the relay agent information option (option 82) feature, see DHCPv4 Relay Agent Information Option (option 82) on page 56.

4. Configuring VxWorks for the DHCPv4 Server

To add the DHCPv4 server to your VSB project, see Source Build Configuration Options (VSB) on page 4.

To add the DHCPv4 server to your VIP, see Kernel Configuration Components (VIP) on page 4.

The **INCLUDE_IPDHCPS** component includes DHCPv4 server functionality to interface with DHCPv4 clients using the Dynamic Host Configuration Protocol. The component also includes the DHCPv4 server shell command (**dhcps**).

To add optional components to your DHCPv4 server VIP environment, see Optional Kernel Configuration Components on page 5.

DHCPv4 Server Configuration Parameters table on page 34 lists the DHCPv4 server parameters.

DHCP Server API Routines table on page 40 lists the API routines available to DHCP server application.

Standard DHCPv4 Server Options table on page 47 lists the standard DHCP server options.

Wind River-Specific DHCPv4 Server Configuration Options table on page 54 lists the DHCP server options specific to Wind River.

DHCPv4 Server Configuration Parameters

DHCPv4 Server Configuration Parameters table on page 34 lists the DHCPv4 server configuration parameters and system variables.

Table 1. DHCPv4 Server Configuration Parameters

Parameter Name, System Variable Name, and Description	Default and Type
DHCPS_ABANDONED_STATE_MAX_TIME **ipdhcps.in_abandoned_state_max_time** The length of time, in seconds, that must pass before an abandoned lease can be released and is free for reuse, or "forever" to indicate that the lease must not be released. A lease is considered to be abandoned when a client has explicitly declined it because the address bound to the offered lease is already in use by another host.	**"8640000"** (100 days) char *
DHCPS_ALLOW_BOOTP **ipdhcps.allow_bootp** Either "true" or "false". If "true", the server accepts requests from BOOTP clients. You can also configure this at run time.	**"true"** char *
DHCPS_ALLOW_DECLINE **ipdhcps.allow_decline** Either "true" or "false". If "true", the server accepts DHCP Decline messages. If "false", the server silently ignores such messages. Clients send messages of type **DHCP Decline** when they want to notify the server that an IPv4 address conflict has occurred, and the address offered by the server is already in use by another host. The default server response is to consider the lease containing the address to be abandoned. However, this makes the server vulnerable to a malfunctioning client or a denial of service attack that could empty the server's address pools. For this reason, you can set the server to ignore client declines.	**"true"** char *
DHCPS_ALLOW_DYNAMIC_BOOTP **ipdhcps.allow_dynamic_bootp** Either "true" or "false". If "true", the server may assign dynamic leases to BOOTP clients. If set to "false", the server assigns only static leases to BOOTP clients. Because BOOTP clients are not able to release their leases, Wind River recommends that you always set Allow dynamic bootp to "false". You can also configure this at run time.	**"false"** char *

Parameter Name, System Variable Name, and Description	Default and Type
DHCPS_AUTHORIZED_AGENTS **ipdhcps.authorized_agents** Either "any" or a space-delimited list of IP addresses of DHCP relay agents that are authorized to access the server.	**"any"** char *
DHCPS_DO_ICMP_ADDRESS_CHECK **ipdhcps.do_icmp_address_check** Either "true" or "false". If "true", the server sends an ICMP ping to check an address before offering it to a client.	**"false"** char *
DHCPS_NETCONF_SYSVAR Either "true" or "false". If "true", you can statically configure the server by entering configuration values in the **ipdhcps_netconf_sysvar** array in the **ipdhcps_config.c** file. For information on configuring the server in this way, see Configuring the Server with the ipdhcps_netconf_sysvar Array on page 41	**NULL** void *
DHCPS_START_CALLBACK_HOOK A function pointer to an implementation of the **ipdhcps_start_hook( )** routine.The server calls this hook routine at startup. You can use this routine to perform operations such as restoring the lease database from a file and reading parameter settings from a configuration file (see Implementing ipdhcps_start_hook( ) on page 43). If you set this parameter, you must also set the Install dhcp server callback routines parameter to **TRUE**.	**NULL** funcptr
DHCPS_STOP_CALLBACK_HOOK A function pointer to an implementation of the **ipdhcps_stop_hook( )** routine. The server calls this hook routine just before the DHCP daemon exits. You can use this routine to dump the lease database to a file and perform other cleanup operations (see Implementing ipdhcps_stop_hook( ) on page 46). If you set this parameter, you must also set the Install dhcp server callback routines parameter to **TRUE**.	**NULL** funcptr
DHCPS_DEFAULT_LEASE_TIME **ipdhcps.default_lease_time** The default length of time, in seconds, that a client can hold a dynamic lease before it expires, or "forever" to indicate a permanent lease. You can also configure this at run time.	**"864000"** (10 days) char *

Parameter Name, System Variable Name, and Description	Default and Type
DHCPS_NO_STRICT_SUBNET_MATCH **ipdhcps.no_strict_subnet_match** This Boolean determines whether IPDHCPS should not prefer to issue a lease from a pool with the same subnet that is present on the enabled DHCPS interface.	**FALSE** BOOL
DHCPS_AUTO_ENABLE_INTERFACES **ipdhcps.enable_new_interface_when_found** When set to "true", newly discovered and attached interfaces are automatically enabled. When set to "false", newly discovered and attached interfaces must be enabled manually.	**"true"** char *
DHCPS_EXPIRED_STATE_MAX_TIME **ipdhcps.in_expired_state_max_time** Either "forever" or the length of time, in seconds, that must pass before an expired lease can be assigned to another client. When a lease expires, the server keeps it in the lease database in case the client it belonged to wants to claim it again.	**"forever"** char *
DHCPS_INSTALL_CALLBACK_HOOK If set to TRUE, you can include your versions of the **ipdhcps_start_hook( )** and **ipdhcps_stop_hook( )** routines in the build.	**FALSE** BOOL
DHCPS_LEASE_BOOTPC_MAX_TIME **ipdhcps.in_bootp_state_max_time** Either "forever" or the length of time, in seconds, that must pass before the server can assign a lease held by a BOOTP client to another client.	**"forever"** char *
DHCPS_REBINDING_TIME **ipdhcps.rebinding_time** Either "disabled" or the length of time, in seconds, after which a client must attempt to rebind a lease if it has failed to renew it. If set to "disabled", the server does not send a rebinding time to clients. You can also configure this at run time.	**"disabled"** char *
DHCPS_RENEWAL_TIME **ipdhcps.renewal_time** Either **"disabled"** or the length of time, in seconds, after which a client must request its lease to be extended, if it wants to keep its IP address. If set to **"disabled"**, the server does not send a renewal time to clients. You can also configure this at run time.	**"disabled"** char *

Parameter Name, System Variable Name, and Description	Default and Type
DHCPS_MAX_LEASE_TIME **ipdhcps.max_lease_time** The maximum length of time, in seconds, that a client can hold a dynamic lease before it expires, or **"forever"** to indicate a permanent lease. You can also configure this at run time.	**"8640000"** (100 days) char *
DHCPS_MIN_LEASE_TIME **ipdhcps.min_lease_time** The minimum length of time, in seconds, that a client can hold a dynamic lease before it expires. You can also configure this at run time.	**"60"** char *
DHCPS_OFFERED_STATE_MAX_TIME **ipdhcps.in_offered_state_max_time** The length of time, in seconds, that must pass before the server assigns an a lease that is in the "offered" state to another client, or **"forever"** to indicate that the lease may not be assigned to another client. A lease is in an offered state when the server has offered it to a client but that client has not yet bound to it.	**"10"** char *
DHCPS_PKT_SIZE **ipdhcps.packet_size** The maximum size, in bytes, of packets sent by the DHCP server. The value must be at least 576 bytes. If you set this to a higher value, when a client contacts the server, the server informs the client of the packet size.	**"576"** char *
DHCPS_RELEASED_STATE_MAX_TIME **ipdhcps.in_release_state_max_time** Either **"forever"** or the length of time, in seconds, that must pass before a released lease can be assigned to another client. The server puts the lease in the released state when the client holding it tells the server to release it. The server holds it in the lease database in case the client wants to claim it again.	**"forever"** char *
DHCPS_SERVER_PORT **ipdhcps.server_port** The port on the DHCP server to which a client sends DHCP messages.	**"67"** char *

Parameter Name, System Variable Name, and Description	Default and Type
DHCPS_CLIENT_PORT **ipdhcps.client_port** The port on a DHCP/BOOTP client to which the DHCP server sends DHCP messages.	**"68"** char *
DHCPS_STRICT_VENDOR_CLASS_ID_MATCH **ipdhcps.strict_vender_class_id_match** If true, any parameters specific to this client's class (as identified by the contents of the 'vendor class identifier' option in the DHCPDISCOVER or DHCPREQUEST message) must be identified by an exact match between the client's vendor class identifiers and the client's classes identified in the server.	**"true"** char *

5. Shell Commands

You can use the dhcps shell command to manage the DHCPvv4 server.

You can use the ipd shell command to start and stop the DHCPv4 server daemon.

You can use the syslog shell command to set the priority for the DHCPv4 server syslog debug messages.

To run shell commands, you must access the command-interpreter mode. To access the command-interpreter mode from the kernel shell, enter the following command:

```
-> cmd
[vxWorks *} #
```

To return to the kernel shell, enter the following command:

```
# C
->
```

Using DHCPv4 Server Shell Commands

To obtain a list of dhcps shell commands and help for the commands, enter the following command:

```
# dhcps
```

The available commands are as follows:

```
dhcps subnet    {list | add {addr} {mask} | delete {addr} }
dhcps pool      {list {{subnet} | default} | add {{{{subnet} | default}}
                |{{{subnet} | default} -c {class name}}} {first addr}
                {last addr} delete {{subnet} | default} {first addr}
                {last addr} }
dhcps class     {list {{subnet} | default} |
                add {{subnet} | default} {class name} {type} {{id in string
                format} | -r {id in raw hex format}}
```

```
                delete {{subnet} | default} {class name} }
dhcps host      {list | add { {addr} & { -h {link-layer addr} &|
                -i {client identifier} [uuid]} delete {addr} }
dhcps option    list {{{{subnet} | {host} | default}} | {{{subnet} | default}
                -c {class name}}} add {{{{subnet} | {host} | default}} |
                {{{subnet} | default} -c {class name}}}
                {option code} {option data} } |delete {{{{subnet} |
                {host} | default}} | {{{subnet} | default}
                -c {class name}}} {option code}}
dhcps config    {list {{subnet} | {host} | default} |
                set {{{{subnet} | {host} | default} | {{{subnet} | default}
                -c {class name}}} {config code} {config data}} |
                reset {{subnet} | {host} | default} {config code}}
dhcps lease     {list | flush {-s {binding state} | -i {identifier} [uuid] |
                -h {link-layer address} | -a {IP address}}
dhcps interface {list | enable {name} | disable {name}}
```

Stopping and Starting the DHCPv4 Server Daemon

Use the ipd command to stop or start the DHCPv4 server daemon (ipdhcps). For example:

```
# ipd list
Services:
ipdhcps              started

# ipd kill ipdhcps
ipd: kill ipdhcps ok

# ipd list
Services:
ipdhcps              killed

# ipd start ipdhcps
ipd: start ipdhcps ok
```

To view the task information for the DHCPv4 server, run the task command and locate the **ipdhcps** task. For example:

```
# task
NAME  ENTRY  TID PRI  STATUS PC  SP  ERRNO  DELAY
...
ipdhcps  100376d0  1090f648  50  PEND+T  102b87eb  109afb20  3d0004  55
```

Setting the DHCPv4 Server System Log Priority

Run the syslog list command to get the current priority level for all modules:

```
# syslog list ipdhcps
    syslog facility priority
    ...
    ipdhcps Error
```

To list the available syslog priority levels, use the following command:

```
# syslog priority *
syslog: missing <prio>
  priorities: Emerg Crit Error Warning Notice Info Debug Debug2
```

Use the following shell command syntax to change this setting:

```
# syslog priority ipdhcps priority
```

For more information on the syslog shell command, see the *VxWorks 7 Network Stack Programmer's Guide*.

6. API Functions

DHCP Server API Routines table on page 40 lists the API routines available to DHCPv4 server applications and gives a brief description of each routine. For more information, see the reference entries for the DHCPv4 server (**ipdhcps**).

Most DHCPv4 server public API routines are defined in either **ipdhcps.c**, while some public server API routines, such as ipdhcps_subnet_add( ), are defined in **ipdhcps.h** and **ipdhcps_db.c**.

For an example of how to programmatically call the equivalent IPCOM shell DHCP commands, see the setup example in the final portion of **ipdhcps_cmd_dhcps.c** in the **IPDHCPS_EXAMPLE_API** define.

Table 1. DHCP Server API Routines

Functions	Descriptions
ipdhcps_class_add( )	Add a new client class to a subnet.
ipdhcps_class_delete( )	Delete a client class from a subnet.
ipdhcps_interface_status_set( )	Set the status of an interface.
ipdhcps_host_add( )	Add a new host to the configuration database.
ipdhcps_host_delete( )	Delete a host from the configuration database.
ipdhcps_lease_db_dump( )	Dump entries in the lease database to a buffer.
ipdhcps_lease_db_restore( )	Initialize the lease database.
ipdhcps_option_add( )	Add a set of DHCP options to a subnet or host.
ipdhcps_option_delete( )	Delete a DHCP option.
ipdhcps_pool_add( )	Add a new address pool to an existing subnet.
ipdhcps_pool_delete( )	Delete an existing address pool from a subnet.
ipdhcps_subnet_add( )	Add a new subnet to the configuration database.
ipdhcps_subnet_delete( )	Delete a subnet from the configuration database.

7. Setting Up Addresses, Options, Subnets and Hosts

You can configure the DHCPv4 server's initial state—IP addresses, options, subnets, hosts, and default settings—either statically or dynamically.

To statically configure the DHCPv4 server, enter the configuration settings in the **ipdhcps_netconf_sysvar** array, see Configuring the Server with the ipdhcps_netconf_sysvar Array on page 41.

To dynamically configure the DHCPv4 server, either issue shell commands, sysvars, or implement the ipdhcps_start_hook( ) routine to read a configuration file at startup (see Implementing ipdhcps_start_hook( ) on page 43).

Configuring the Server with the ipdhcps_netconf_sysvar Array

You can statically configure the server by entering commands and parameter values in the **ipdhcps_netconf_sysvar** array. To do this you must enable this arrayby setting the DHCP server network pre-configuration (**DHCPS_NETCONF_SYSVAR**) build parameter to "**true**" (see DHCPv4 Server Configuration Parameters table on page 34).

You can find example usage of the **ipdhcps_netconf_sysvar** array in **ipdhcps_config.c**.

For a list of valid commands, see Commands Available in the ipdhcps_netconf_sysvar Array on page 41.

With the exception of the last entry in the array, entries have the following form:

```
{"ipdhcps.netconf.index", "command arguments"}
```

where

index is an index into the **ipdhcps_netconf_sysvar** array, and

command is a two-word expression, such as the command subnet add, which is used to add a subnet to the configuration database.

The last entry in the array must always be:

```
{IP_NULL, IP_NULL}
```

The following example adds subnet 10.1.0.0 with subnet mask 255.255.0.0 to the configuration database in the first element in the **ipdhcps_netconf_sysvar** array:

```
IP_STATIC IP_CONST Ipcom_sysvar ipdhcps_netconf_sysvar[] =
{"ipdhcps.netconf.00", "subnet add 10.1.0.0 255.255.0.0"},
...
{IP_NULL, IP_NULL}
}
```

Commands Available in the ipdhcps_netconf_sysvar Array

You can use the following commands in the DHCPv4 server static configuration array:

```
subnet add netAddress mask
```

Add a subnet with the specified network address and mask to the configuration database. The following example adds the 10.1*.x.x* subnet:

```
{"ipdhcps.netconf.00", "subnet add 10.1.0.0 255.255.0.0"}
```

```
pool add subnet firstAddress lastAddress
```

Add an address pool to a subnet, for example:

```
{"ipdhcps.netconf.01", "pool add 10.1.0.0 10.1.0.5 10.1.0.99"}
```

```
class add subnet clientClass classDescriptor
```

Add a new client class to an existing subnet, for example:

```
{"ipdhcps.netconf.23", "class add 10.1.0.0 client-class vendor info"}
```

```
host add hostIPaddress [-h linkLayerAddress | -i \0clientID]
```

Add the specified host to the configuration database. To specify the host, use the host's IP address and, optionally, a link-layer address or a client identifier. The following example adds a host with the client identifier of LabSys1 with the IP address 10.0.0.142:

```
{"ipdhcps.netconf.13", "host add 10.0.0.142 -i \"\\0LabSys1\""}
```

```
option add address optionCode value
```

Add a DHCP option, as listed in RFC 2132, to the server's default set of options or to the options associated with a specified host address or subnet address. To add an option to the default set, use **default** as the address.

For a table of the options available on the Wind River DHCPv4 server, see Standard DHCPv4 Server Options table on page 47. The following DHCP options are not available:

- TFTP server (option code 66)
- Bootfile name (option code 67; there is a Wind-River specific option for setting a bootfile name; see the entry for boot-file in Wind River-Specific DHCPv4 Server Configuration Options table on page 54)

The following example adds two routers to subnet 192.168.2*x*. The DHCP option code to add routers is 3.

```
{"ipdhcps.netconf.09", "option add 192.168.2.0 3 192.168.2.1 192.168.2.2"}
```

```
config set address optionCode value
```

The config set command applies to options that are specific to Wind River's DHCP implementation. For information on these options, see Wind River-Specific DHCPv4 Server Configuration Options table on page 54.

This command adds a dhcps config option to the default set of options or to the options associated with a specified host address or subnet address. To add an option to the default set, use **default** as the address.

The following example specifies a boot file for host 10.0.0.141:

```
{"ipdhcps.netconf.18", "config set 10.0.0.141 boot-file /boot/hostZ.boot"}
```

8. Implementing Hook Routines for Initialization and Shutdown

If you want the DHCP server to do any custom processing of your choosing at startup or shutdown, you can implement this through the ipdhcps_start_hook( ) and ipdhcps_stop_hook( ) routines. For instance, you may want to restore the lease database at startup, and store this database to a file at shutdown. These hook routines are described in this section.

To use your own custom hook routines, you must complete the following steps:

1. Write your routine or routines, according to the instructions in Implementing ipdhcps_start_hook( ) on page 43 and Implementing ipdhcps_stop_hook( ) on page 46.
2. Set the configuration parameter **DHCPS_INSTALL_CALLBACK_HOOK** to TRUE.
3. Set either or both of the configuration parameters **DHCPS_START_CALLBACK_HOOK** and **DHCPS_STOP_CALLBACK_HOOK** to point to your custom hook routines.
4. Rebuild your target.

Implementing ipdhcps_start_hook( )

The DHCPS daemon calls the ipdhcps_start_hook( ) function at startup. The signature of ipdhcps_start_hook( ) is defined in **ipdhcps_config.c**.

This routine returns zero to indicate that initialization has succeeded (or one to indicate a failure).

Write ipdhcps_start_hook( ) to do any initialization you need to accomplish, which may include calling any of the DHCP server API routines. You can use this API to restore the lease database and to configure the server with global defaults, subnets, hosts, and options. For a list and brief descriptions of the routines in the API, see API Functions on page 40. For detailed information about individual routines, see the API reference pages.

You can implement ipdhcps_start_hook( ) to configure the server by reading a configuration file at startup. For an example of this, see Reading Configuration Values from a File on page 44. You could also call DHCPv4 server API routines with hard-coded arguments, but this is equivalent to statically configuring the server through entries in the **ipdhcps_netconf_sysvar** array entries in the **ipdhcps_config.c** file, which is simpler and less error-prone.

Restoring the Lease Database at Startup

The following implementation of ipdhcps_start_hook( ) assumes that the data for the lease database is in a file. It reads this file into a buffer and then calls ipdhcps_lease_db_restore( ).

```
IP_GLOBAL int ipdhcps_start_hook (void)
 {
   IP_FILE      fd;
   struct stat statbuf;
   void *      buf;

   /* Get size of dump file */
   stat ("/opt/ipdhcps/dumpfile", &statbuf);

   /* Transfer data from file to a data buffer */
   fd = fopen ("/opt/ipdhcps/dumpfile", "r");
   buf = malloc (statbuf.st_size);
   fread (buf, statbuf.st_size, 1, fd);

   /* Restore lease database from file */
   ipdhcps_lease_db_restore (buf);

   fclose (fd);
   free (buf);
```

```
    return (0);
}
```

Reading Configuration Values from a File

The following sample implementation of the ipdhcps_start_hook( ) function reads a configuration file in which each line is a string that corresponds to a DHCPv4 server shell command (see Using DHCP Server Shell Commands on page 38) without the initial dhcps command identifier, as in the following example:

```
subnet add 10.1.0.0 255.255.0.0
```

The routine parses the command string using the getOptServ( ) routine (for more information, see the getOptServ( ) reference page) and then calls the ipdhcps_cmd_dhcps( ) function, which executes the command (for more information on the ipdhcps_cmd_dhcps( ) function, see ipdhcps_cmd_dhcps( ) on page 46).

```
IP_GLOBAL int ipdhcps_start_hook (void)
  {
    IP_FILE *    fd         = IP_NULL;
    char *       cmdbuf     = IP_NULL;
    char         args[100];
    const char * cfgfile    = "/home/dhcpServer/dhcps.conf";
    const char * cmd        = "dhcps ";
    int          argc;
    char *       tempargv   = [106 + 2];
    char **      argv       = tempargv;
    int          ret        = -1;

    if ((fd = fopen(cfgfile,"r")) == 0)
         {
         logMsg ("DHCPS ERROR: Failed to open configuration file %s\n",
                 cfgfile, 0, 0, 0, 0, 0);
         goto leave;
         }

    /*
     * The config file consists of command line arguments that are
     * understood by the DHCP server's shell tool
     */

    /* Read the command lines one by one and execute */
    while (fgets (args, sizeof (args), fd) != IP_NULL)
        {
        /* Remove the trailing EOLN */
        args [strlen (args) - 1] = 0;

        /* Compose a complete command args */
        cmdbuf = malloc (strlen (args) + strlen (cmd) + 1);

        if (cmdbuf == IP_NULL)
           {
           logMsg ("DHCPS ERROR: failed to allocate buffer\n",
                   0, 0, 0, 0, 0, 0);
           goto leave;
           }
```

```
        strcpy (cmdbuf, cmd);
        strcat (cmdbuf, args);

       if (getOptServ (cmdbuf, "ipdhcps_start_hook", &argc, argv, 106 + 2)
           != OK)
           {
           logMsg ("DHCPS ERROR: failed to parse command '%s'\n",
                   args, 0, 0, 0, 0, 0);
           goto leave;
           }

      /* Let the dhcps command tool execute */
      if (ipdhcps_cmd_dhcps (argc, argv) != OK)
         {
         logMsg ("DHCPS ERROR: failed to execute command '%s'\n",
                 args, 0, 0, 0, 0, 0);
         goto leave;
         }

      free (cmdbuf);
      cmdbuf = NULL;
      }

    ret = OK;

leave:

   if (fd)
      fclose (fd);
   if (cmdbuf)
      free (cmdbuf);

   return ret;
}
```

Sample configuration file:

```
subnet add 10.1.0.0 255.255.0.0
pool add 10.1.0.0 10.1.0.5 10.1.0.99
pool add 10.1.0.0 10.1.0.199 10.1.0.249
option add 10.1.0.0 15 green.net
option add 10.1.0.0 3 10.1.0.1
option add 10.1.0.0 5 10.1.0.2
subnet add 192.168.2.0 255.255.255.0
pool add 192.168.2.0 192.168.2.10 192.168.2.50
option add 192.168.2.0 15 blue.net
option add 192.168.2.0 3 192.168.2.1 192.168.2.2
option add 192.168.2.0 33 10.1.0.0 192.168.2.2
config set 192.168.2.0 max-lease-time 120
config set 192.168.2.0 default-lease-time 60
host add 10.1.0.142 -i \0Jack's lap top
option add 10.1.0.142 12 blue-host
host add 10.1.0.141 -h BE:60:13:F7:F8:0F
option add 10.1.0.141 12 yellow-host
config set 10.1.0.141 boot-file /boot/yellow-host.boot
config set 10.1.0.141 next-server-name black-server
config set default default-lease-time 600
```

```
config set default allow-bootp false
config set 10.1.0.0 do-icmp-address-check true
class add 10.1.0.0 first-class vendor first
class add default second-class vendor second
option add 10.1.0.0 35 10
option add 10.1.0.0 -c first-class 35 20
option add default 35 60
option add default -c second-class 35 40
config set 10.1.0.0 authorized-agents 10.1.0.2 10.1.0.3
```

ipdhcps_cmd_dhcps( )

The ipdhcps_cmd_dhcps( ) function executes a DHCP server shell command. The signature for the ipdhcps_cmd_dhcps( ) function is:

```
IP_GLOBAL int ipdhcps_cmd_dhcps (int argc, char ** argv)
```

where:

argc is the number of strings in the array pointed to by the argv parameter.

argv is an array of strings that constitute the elements of a DHCP server shell command, without the initial dhcps identifier. The required elements in the array are:

command subcommand `{ 0 or more arguments }`

For example, for the subnet add shell command, argv could contain the following strings:

```
"subnet" "add" "10.1.0.0" "255.255.0.0"
```

The ipdhcps_cmd_dhcps( ) function returns **0** on success.

Implementing ipdhcps_stop_hook( )

The DHCPS daemon automatically calls the ipdhcps_stop_hook( ) function immediately before the server exits. The signature of ipdhcps_stop_hook( ) is defined in **ipdhcps_config.c**.

This routine returns zero to indicate that the hook routine has succeeded (or one to indicate a failure)

You can write an implementation of the ipdhcps_stop_hook( ) function that does any shutdown and cleanup processing necessary to your application, including dumping the lease database using the ipdhcps_lease_db_dump( ) API.

Dumping the Lease Database at Shutdown

The following implementation of ipdhcps_stop_hook( ) reads the contents of the lease database into a buffer using the ipdhcps_lease_db_dump( ) API and then writes that buffer to a file:

```
IP_GLOBAL int ipdhcps_stop_hook ()
  {
    void * buf;
    IP_FILE * fd;
    int n;

    /* Dump the lease database */
```

```
    n = ipdhcps_lease_db_dump (&buf);

    /* Write it to a file */
    fd = fopen ("/opt/ipdhcps/dumpfile", "w+");
    fwrite (buf, n, 1, fd);

    fclose (fd);
    free (buf);

    return (0);
}
```

9. Setting Options with Shell Commands and API Routines

You can set standard DHCP options and Wind River-specific options statically in the **ipdhcps_config.c** file.

In this case, use numerical codes for individual options, as given in Standard DHCPv4 Server Options table on page 47 (from RFC 2132) and Wind River-Specific DHCPv4 Server Configuration Options table on page 54.

You can also set DHCP options through shell commands. In this case, you enter the option code as required by the shell command. For example, for subnet mask (option code 1), you need to enter 1 as the option code (see Standard DHCPv4 Server Options table on page 47).

Finally, if you set DHCP option values through DHCPv4 server API routines, instead of directly entering a numerical option value, enter a macro value for the code. For option code 1, use **IPDHCPS_OPTCODE_SUBNET_MASK** (see Standard DHCPv4 Server Options table on page 47 and Wind River-Specific DHCPv4 Server Configuration Options table on page 54 for lists of these macros).

Standard DHCP Options

Standard DHCPv4 Server Options table on page 47 lists the standard DHCPv4 server options available in Wind River DHCP. The options are ordered by code number, and the table gives their macro names and definitions. When using shell commands to invoke options, use the code number to refer to the server option.

Table 1. Standard DHCPv4 Server Options

Code	Macro Name and Definition	Option Type
1	**IPDHCPS_OPTCODE_SUBNET_MASK** subnet mask	IPv4 address
2	**IPDHCPS_OPTCODE_TIME_OFFSET** time offset	time offset 32-bits
3	**IPDHCPS_OPTCODE_ROUTERS** router(s)	list of IPv4 addresses

Code	Macro Name and Definition	Option Type
4	**IPDHCPS_OPTCODE_TIME_SERVERS** time server(s)	list of IPv4 addresses
5	**IPDHCPS_OPTCODE_NAME_SERVERS** name server(s)	list of IPv4 addresses
6	**IPDHCPS_OPTCODE_DOMAIN_NAME_SERVERS** domain name server(s)	list of IPv4 addresses
7	**IPDHCPS_OPTCODE_LOG_SERVERS** log server(s)	list of IPv4 addresses
8	**IPDHCPS_OPTCODE_COOKIE_SERVERS** cookie server(s)	list of IPv4 addresses
9	**IPDHCPS_OPTCODE_LPR_SERVERS** lpr server(s)	list of IPv4 addresses
10	**IPDHCPS_OPTCODE_IMPRESS_SERVERS** impress server(s)p	list of IPv4 addresses
11	**IPDHCPS_OPTCODE_RESOURCE_LOCATION_SERVERS** resource location server(s)	list of IPv4 addresses
12	**IPDHCPS_OPTCODE_HOST_NAME** host name	string
13	**IPDHCPS_OPTCODE_BOOT_SIZE** boot file size	16-bits

Code	Macro Name and Definition	Option Type
14	**IPDHCPS_OPTCODE_MERIT_DUMP** merit dump path	string
15	**IPDHCPS_OPTCODE_DOMAIN_NAME** domain name	string
16	**IPDHCPS_OPTCODE_SWAP_SERVER** swap server(s) I	Pv4 address
17	**IPDHCPS_OPTCODE_ROOT_PATH** root path	string
18	**IPDHCPS_OPTCODE_EXTENSIONS_PATH** extensions path	string
19	**IPDHCPS_OPTCODE_IP_FORWARDING** ip forwarding	boolean
20	**IPDHCPS_OPTCODE_NON_LOCAL_SOURCE_ROUTING** non local source routing	boolean
21	**IPDHCPS_OPTCODE_POLICY_FILTER** policy filter	list of IPv4 address pairs
22	**IPDHCPS_OPTCODE_MAX_DGRAM_REASSEMBLY** max datagram reassembly	16-bits
23	**IPDHCPS_OPTCODE_DEFAULT_IP_TTL** ip time to live	8-bits

Code	Macro Name and Definition	Option Type
24	**IPDHCPS_OPTCODE_PATH_MTU_AGING_TIMEOUT** path mtu aging timeout	32-bits
25	**IPDHCPS_OPTCODE_PATH_MTU_PLATEAU_TABLE** path mtu plateau table	list of 16-bit values
26	**IPDHCPS_OPTCODE_INTERFACE_MTU** interface mtu	16-bits
27	**IPDHCPS_OPTCODE_ALL_SUBNETS_LOCAL** all subnets local	boolean
28	**IPDHCPS_OPTCODE_BROADCAST_ADDRESS** broadcast address	IPv4 address
29	**IPDHCPS_OPTCODE_PERFORM_MASK_DISCOVERY** perform mask discovery	boolean
30	**IPDHCPS_OPTCODE_MASK_SUPPLIER** subnet mask supplier	boolean
31	**IPDHCPS_OPTCODE_ROUTER_DISCOVERY** perform router discovery	boolean
32	**IPDHCPS_OPTCODE_ROUTER_SOLICITATION_ADDRESS** router solicitation address	IPv4 address
33	**IPDHCPS_OPTCODE_STATIC_ROUTES** static routes	list of IPv4 address pairs

Code	Macro Name and Definition	Option Type
34	**IPDHCPS_OPTCODE_TRAILER_ENCAPSULATION** trailer encapsulation	boolean
35	**IPDHCPS_OPTCODE_ARP_CACHE_TIMEOUT** arp cache timeout	32-bits
36	**IPDHCPS_OPTCODE_IEEE802_3_ENCAPSULATION** ethernet encapsulation	boolean
37	**IPDHCPS_OPTCODE_DEFAULT_TCP_TTL** default tcp time to live	8-bits
38	**IPDHCPS_OPTCODE_TCP_KEEPALIVE_INTERVAL** tcp keep alive interval	32-bits
39	**IPDHCPS_OPTCODE_TCP_KEEPALIVE_GARBAGE** tcp keep alive garbage	boolean
40	**IPDHCPS_OPTCODE_NIS_DOMAIN** nis domain	string
41	**IPDHCPS_OPTCODE_NIS_SERVERS** nis server(s)	list of IPv4 addresses
42	**IPDHCPS_OPTCODE_NTP_SERVERS** ntp server(s)	list of IPv4 addresses
43	**IPDHCPS_OPTCODE_VENDOR_ENCAPSULATED_OPTIONS** vendor encapsulated options	string

Code	Macro Name and Definition	Option Type
44	**IPDHCPS_OPTCODE_NETBIOS_NAME_SERVERS** netbios name server(s)	list of IPv4 addresses
45	**IPDHCPS_OPTCODE_NETBIOS_DD_SERVER** netbios dgram distr server(s)	list of IPv4 addresses
46	**IPDHCPS_OPTCODE_NETBIOS_NODE_TYPE** netbios node type	8-bits
47	**IPDHCPS_OPTCODE_NETBIOS_SCOPE** netbios scope	string
48	**IPDHCPS_OPTCODE_FONT_SERVERS** X font server(s)	list of IPv4 addresses
49	**IPDHCPS_OPTCODE_X_DISPLAY_MANAGER** X display manager	list of IPv4 addresses
60	**IPDHCPS_OPTCODE_VENDOR_CLASS** vendor class	string
61	**IPDHCPS_OPTCODE_CLIENT_IDENTIFIER** client identifier (to configure this parameter from the shell use the host add command with the -i modifier)	string
64	**IPDHCPS_OPTCODE_NISPLUS_DOMAIN** nis+ domain	string
65	**IPDHCPS_OPTCODE_NISPLUS_SERVERS** nis+ server(s)	list of IPv4 addresses

Code	Macro Name and Definition	Option Type
68	**IPDHCPS_OPTCODE_HOME_AGENTS** mobile ip home agent	list of IPv4 addresses
69	**IPDHCPS_OPTCODE_SMTP_SERVERS** smtp server(s)	list of IPv4 addresses
70	**IPDHCPS_OPTCODE_POP3_SERVERS** pop3 server(s)	list of IPv4 addresses
71	**IPDHCPS_OPTCODE_NNTP_SERVERS** nntp server(s)	list of IPv4 addresses
72	**IPDHCPS_OPTCODE_WWW_SERVERS** www server(s)	list of IPv4 addresses
73	**IPDHCPS_OPTCODE_FINGER_SERVERS** finger server(s)	list of IPv4 addresses
74	**IPDHCPS_OPTCODE_IRC_SERVERS** irc server(s)	list of IPv4 addresses
75	**IPDHCPS_OPTCODE_STREETTALK_SERVERS** street talk server(s)	list of IPv4 addresses
76	**IPDHCPS_OPTCODE_STDA_SERVERS** stda server(s)	list of IPv4 addresses
77	**IPDHCPS_OPTCODE_USER_CLASS** user class	string

Code	Macro Name and Definition	Option Type
82	**IPDHCPS_OPTCODE_RELAY_AGENT_INFO** relay agent information option Sub-options: **IPDHCPS_SUBOPTCODE_RA_CIRCUIT_ID** circuit identifier suboption 1 **IPDHCPS_SUBOPTCODE_RA_REMOTE_ID** remote identifier suboption 2 **IPDHCPS_SUBOPTCODE_RA_AUTHENTICATION** authentication suboption 8 string string string	string string string
255	**IPDHCPS_OPTCODE_END** (End option for the array)	byte

Wind River Specific Option

Wind River-Specific DHCPv4 Server Configuration Options table on page 54 lists the configuration options specific to the Wind River implementation of the DHCPv4 server, and gives their the API-related macro names and shell command option names.

Table 2. Wind River-Specific DHCPv4 Server Configuration Options

Code	Macro Name and Shell Command Option Name	Description	Option Typ
0x8001	**IPDHCPS_CONFCODE_LEASE_TIME_DFLT** **default-lease-time**	Set the default lease time for clients, in seconds. You can also set this option as a build parameter. Give the option in host byte order.	integer
0x8002	**IPDHCPS_CONFCODE_LEASE_TIME_MAX** **max-lease-time**	Set the maximum lease time for clients, in seconds. You can also set this option as a build parameter. Give the option in host byte order.	integer
0x8003	**IPDHCPS_CONFCODE_LEASE_TIME_MIN** **min-lease-time**	Set the minimum lease time for clients, in seconds. You can also set this option as a build parameter. Give the option in host byte order.	integer
0x8004	**IPDHCPS_CONFCODE_RENEWAL_TIME** **renewal-time**	Set the lease renewal time for clients, in seconds. You can also set this option as a build parameter. Give the option in host byte order.	integer
0x8005	**IPDHCPS_CONFCODE_REBINDING_TIME** **rebinding-time**	Set the lease rebinding time for clients, in seconds. You can also set this option as a build parameter. Give this option in host byte order.	integer

Code	Macro Name and Shell Command Option Name	Description	Option Typ
0x8006	**IPDHCPS_CONFCODE_ALLOW_BOOTP** **allow-bootp**	Enable the server to accept requests from BOOTP clients. You can also set this option as a build parameter.	boolean
0x8007	**IPDHCPS_CONFCODE_ALLOW_DYNAMIC_BOOTP** **allow-dynamic-bootp**	Enable the server to accept dynamic leases for BOOTP. You can also set this option as a build parameter.	boolean
0x8008	**IPDHCPS_CONFCODE_NEXT_SERVER_NAME** **next-server-name**	Give the name of the next server in the boot chain.	string
0x8009	**IPDHCPS_CONFCODE_NEXT_SERVER_IP** **next-server-ip**	Give the IPv4 address of the next server in the boot chain, in network byte order.	four bytes
0x800a	**IPDHCPS_CONFCODE_BOOT_FILE** **boot-file**	Set the name of the boot file.	string
0x800b	**IPDHCPS_CONFCODE_DO_ICMP_ADDRESS_CHECK** **do-icmp-address-check**	Indicates if IPDHCPS is going to perform an ICMP ping check on each address it is offering to clients.	Boolean
0x800c	**IPDHCPS_CONFCODE_AUTHORIZED_AGENTS** **authorized-agents**	Address(es) of DHCP relay agents authorized to access the DHCP server.	list of IPv4 addresses
0x800d	**IPDHCPS_CONFCODE_RA_AUTHENTICATION** **option82-auth**	The shared key information for the authentication suboption (suboption 8) of the Relay Agent Information Option (option 82).	string
0x800e	**IPDHCPS_CONFCODE_RA_DROP** **option82-drop**	The drop policy enforcement when the received packet has no authentication suboption (suboption 8) as part of the Relay Agent Information Option (option 82).	boolean
0x80ff	**IPDHCPS_CONFCODE_END**	End option for the array.	8-bits

6. WIND RIVER DHCPV4: RELAY AGENT

1. Introduction

This chapter describes the Wind River DHCPv4 relay agent. For a general overview of DHCP, see DHCP Overview on page 3. For a list of implemented RFCs, see Supported RFCs on page 2.

A *relay agent* is a switch or router that passes DHCP messages between DHCP clients and DHCP servers. There can be multiple relay agents between a client and a server. A DHCP relay agent is also a BOOTP relay agent, as specified in RFCs 951 and 1542. The Wind River DHCP relay agent runs as a daemon process that acts as a stateless message forwarder between a DHCP client and a DHCP server. The relay agent examines each packet in a DHCP or BOOTP message.

Packets Sent to a DHCP Server

When the relay agent receives a message from a DHCP client, it forwards the message on to all the DHCP servers that the relay agent knows about.

The DHCP/BOOTP header in DHCP packets contains an *agent* field that is meant to contain the IP address of the first relay agent that handles a request by a client. The address indicates the network segment on which the client is located. Before forwarding a DHCP packet to a server, the relay agent checks the DHCP/BOOTP header to see if it already contains an agent address. If it does not, the relay agent assumes that it is the first agent to see the message and it fills the field with the address of the interface on which it received the message, before passing the message on.

Packets Sent to the Client

If a relay agent receives a packet intended for a DHCP client, the relay *agent* checks the agent field and takes one of the following actions:

- If the agent field contains the address of one of its own interfaces, the agent relays the packet directly to the client.
- If the agent field contains an address that does not belong to one of its interfaces but matches the subnet of an other relay agent, the relay agent forwards the packet to the next agent.
- If the address in the agent field does not match a known agent or one of its own interfaces, the relay agent broadcasts the message on the subnet identified in the address.

DHCPv4 Relay Agent Information Option (option 82)

The relay agent information option (option 82) feature enables DHCPv4 servers to implement security and IP address assignment policies based on information received by a relay agent or a bridge. The DHCPv4 server can also use this information to perform access control and set quality of service (QoS) for each client.

Security is enhanced with a shared key between the relay agent and the server. IP address assignment can be based on a user-specified circuit identifier or a user-specified remote identifier residing on the relay agent.

The DHCPv4 server echoes the option 82 information back to the relay agent in server-to-client replies. The relay agent strips the option 82 information before forwarding the reply to the client.

The relay agent information option is organized as a single DHCP option that contains suboptions. The option and suboptions are as follows:

- Relay Agent Information Option (option 82) with the following suboptions:
  - A circuit identifier for the incoming circuit (suboption 1).

- A remote identifier of a trusted remote device (suboption 2).
- An authentication suboption containing a shared key used for data integrity and replay protection for relayed DHCP messages (suboption 8).

If the relay agent determines that the authentication suboption is present and that it should process the suboption, the specified algorithm, key identifier, and replay detection method are used to validate the message. The Wind River DHCP relay agent supports the following parameter values:

- Algorithm: HMAC-SHA1 keyed hash
- Key identifier: 0 (use only one shared secret among a network's relay agent and DHCP server pair)
- Replay detection method: 1 (use of a monotonically increasing counter value)

To configure the DHCP relay agent's option 82 parameters from the command shell, see Using DHCP Relay Agent Shell Commands on page 59.

To configure the DHCP relay agent's option 82 parameters from an application, see API Functions on page 61.

Using the Circuit identifier Sub-option Parameter

The circuit identifier is intended for use by relay agents in relaying DHCP responses back to the proper circuit (client-side). The circuit identifier must be unique for a given relay agent.

DHCPv4 servers can use the circuit identifier for IP and other parameter assignment policies as they relate to a DHCP client.

Since the circuit identifier is local only to a particular DHCP relay agent, a circuit identifier can be qualified with the **giaddr** value that identifies the DHCP relay agent.

Using the Remote identifier Sub-option Parameter

You can add the remote identifier suboption to identify the remote host end of the circuit (server-side).

DHCPv4 servers can use this option to select parameters specific to particular users, hosts, or subscriber modems. A remote identifier must be unique for a network.

Using the Authentication Sub-option Parameter

You can add the authentication suboption (a shared-key) to relay agents that seek to ensure the integrity of the data they include in the relay agent option. The relay agents are configured with the parameters necessary for generating cryptographic checksums of the data in the DHCP messages that they forward to DHCPv4 servers. A DHCPv4 server configured to process the authentication suboption uses the information in the suboption to verify the checksum in the suboption and continues processing the relay agent information option only if the checksum is valid. If the DHCPv4 server sends a response, it includes an authentication suboption in its response message. Relay agents test the checksums in DHCPv4 server responses to decide whether to forward the responses.

If a received packet has option 82 parameters but excludes the option 82 authentication suboption parameter, the DHCP relay agent does not add the authentication suboption to the packet.

A relay agent and a server can only share a single shared-key (authentication suboption value).

Drop Policy

Enable the drop policy to force option 82 authentication of incoming packets using the shared key. When the drop policy is disabled, option 82 authentication is not implemented by the relay agent.

Option 82 Support for the Bridge and DHCP Server

For information on the Relay Agent Information Option for the Wind River DHCP server, see Wind River DHCPv4: Server on page 32.

Relay Agent Information Option Tutorial

For information on how to configure, test, and troubleshoot a DHCPv4 environment with the relay agent information option, see DHCPv4 Relay Agent Information Option Tutorial on page 96.

2. Configuring VxWorks for the DHCP Relay Agent

To add the DHCP relay agent to your VSB project, see Source Build Configuration Options (VSB) on page 4.

To add the DHCP relay agent to your VIP, see Kernel Configuration Components (VIP) on page 4.

The **INCLUDE_IPDHCPR** component includes DHCP relay agent functionality to to pass DHCP messages between DHCP clients and DHCP servers. The component also includes the DHCP relay agent shell commands (dhcpr).

To add optional components, see Optional Kernel Configuration Components on page 5.

DHCP Relay-Agent Configuration Parameters table on page 58 lists the DHCP relay parameters and their default values.

DHCP Relay API Function table on page 61 lists the API routines available to DHCP relay agent applications.

DHCP Relay Agent Configuration Parameters

DHCP Relay-Agent Configuration Parameters table on page 58 lists the DHCP relay agent configuration parameters and system variables.

Table 1. DHCP Relay-Agent Configuration Parameters

Parameter Name, System Variable Name, and Description	Type
DHCPR_NETCONF_SYSVAR Either "true" or "false". If "true", you can statically configure the relay agent by entering configuration values in the **ipdhcpr_netconf_sysvar**array in the **ipdhcpr_config.c** file. For information on entering values into the array, see Configuring with the ipdhcpr_netconf_sysvar Array on page 61.	**NULL** void *
DHCPR_START_CALLBACK_HOOK If you set this function pointer and set the Install dhcp relay callback routine parameter to TRUE, the DHCP relay agent calls this routine—your implementation of the **ipdhcpr_start_hook( )** function—when it starts. For information on **ipdhcpr_start_hook( )**, see Implementing ipdhcpr_start_hook( ) on page 63.	**NULL** funcptr
DHCPR_INSTALL_CALLBACK_HOOK Set this to TRUE if you implement the **ipdhcpr_start_hook( )** function (and also set the following parameter to point to your implementation of that routine). For information on **ipdhcpr_start_hook( )**, see Implementing ipdhcpr_start_hook( ) on page 63.	**FALSE** BOOL

Parameter Name, System Variable Name, and Description	Type
DHCPR_MAX_PKT_SIZE ipdhcpr.PacketSize The maximum packet size, in bytes, the DHCP relay agent will use when sending and receiving packets. The size may never be less than 576 bytes, which is the standard packet size for DHCP.	**"576"** char *
DHCPR_HOPS_THRESHOLD ipdhcpr.HopsThreshold The maximum number of relay agents allowed between a DHCP client and a DHCP server. The agent silently drops packets with a hop count greater than this setting.	**"3"** char *
DHCPR_SERVER_PORT ipdhcpr.ServerPort The UDP port on which the DHCP server receives messages. Enter the value as a quoted string.	**"67"** char *
DHCPR_CLIENT_PORT ipdhcpr.ClientPort The UDP port on which the DHCP client receives messages. Enter the value as a quoted string.	**"68"** char *

3. Shell Commands

You can use the dhcpr shell command to manage the DHCP relay agent. The dhcpr shell command is included with the **INCLUDE_DHCPR** component.

You can use the ipd shell command to start and stop the DHCP relay agent daemon.

You can use the syslog shell command to set the priority for the DHCP relay agent syslog debug messages.

To run shell commands, you must access the command-interpreter mode. To access the command-interpreter mode from the kernel shell, enter the following command:

```
-> cmd

[vxWorks *} #
```

To return to the kernel shell, enter the following command:

```
# C

->
```

Using DHCP Relay Agent Shell Commands

To obtain a list of dhcpr shell commands and *help* for the commands, enter the following command:

```
# dhcpr
```

The available commands are as follows:

```
dhcpr server    {list | add {addr} | delete {addr} |
                option82 {addr} {{enable auth {in raw hex format}} |
                {disable auth}}}
dhcpr interface {list | enable {name} | disable {name} |
                 option82 {name} {{circuit-id {in raw hex format}} |
                 {remote-id {in raw hex format}} | disable}}
dhcpr option82   {list | enable {auth | drop} | disable {auth | drop}}
```

Use the dhcpr server shell command to list all DHCP servers known to the relay agent, to add a server to the relay agent's list of servers, or to delete a server from this list.

Use the dhcpr interface shell command to list all interfaces enabled for DHCP, to enable or disable individual interfaces for DHCP, or to configure the DHCP relay agent information parameters for option 82.

Use the dhcpr option82 shell command to enable or disable the relay agent information option (option 82) authentication and the drop policy.

Stopping and Starting the DHCP Relay Agent Daemon

Use the ipd command to stop or start the DHCP relay agent daemon (**ipdhcpr**). For example:

```
# ipd list
Services:
ipdhcpr               started

# ipd kill ipdhcpr
ipd: kill ipdhcpr ok

# ipd list
Services:
ipdhcpr               killed

# ipd start ipdhcpr
ipd: start ipdhcpr ok
```

To view the task information for the DHCP relay agent, run the task command and locate the **ipdhcpr** task. For example:

```
# task
NAME    ENTRY    TID       PRI  STATUS  PC        SP        ERRNO  DELAY
...
ipdhcpr ipdhcpr  105765d0  50   PEND    102b87eb  10a2fcf0  31     0
```

Setting the DHCP Relay Agent system Log Priority

Run the syslog list command to get the current priority level for all modules:

```
# syslog list ipdhcpr
    syslog facility priority
    ipdhcpr Error
```

To list the available syslog priority levels, use the following command:

```
# syslog priority *
syslog: missing <prio>
        priorities: Emerg Crit Error Warning Notice Info Debug Debug2
```

Use the following shell command syntax to change this setting:

```
# syslog priority ipdhcpr priority
```

For more information on the syslog shell command, see the *VxWorks 7 Network Stack Programmer's Guide*.

4. API Functions

. For more information, see the reference entries for the DHCP relay agent (**ipdhcpr**).

DHCP Relay API Function table on page 61 lists the API routines available to DHCP relay agent applications and gives a brief description of each routineThe DHCP relay agent public API routines are defined in **ipdhcpr.c**.

Table 1. DHCP Relay API Function

Functions	Description
ipdhcpr_interface_option82( )	Set the circuit identifier and remote identifier parameters for the relay agent information option (option 82).
ipdhcpr_option82_drop_set( )	Set the drop policy mode for the relay agent information option (option 82).
ipdhcpr_option82_auth_set( )	Set the relay agent authentication mode for the relay agent information option (option 82).
ipdhcpr_server_option82( )	Set the option 82 authentication mode and shared key for the specified server.
ipdhcpr_interface_status_set( )	Set the status of an interface on the relay agent.
ipdhcpr_server_add( )	Add a new server address to the DHCP relay agent.
ipdhcpr_server_delete( )	Delete a server address.

5. Configuring with the ipdhcpr_netconf_sysvar Array

You can statically configure the relay agent by entering commands and parameter values in the **ipdhcpr_netconf_sysvar** array in the **ipdhcpr_config.c file**.

For a list of the commands, see Setting Commands in the ipdhcpr_netconf_sysvar Array on page 62

With the exception of the last entry in the array, entries have the following form:

```
{"ipdhcpr.netconf.index", "command action"}
```

In these entries, replace *index*, *command*, and *action* as follows:

index is an index into the **ipdhcpr_netconf_sysvar** array, and

command is an expression followed by an action, such as add, enable, or disable, as in the command server add, which is used to add a DHCP server to the relay agent's list of DHCP servers.

The last entry in the array must always be:

```
{IP_NULL, IP_NULL}
```

In the following example the first element in the **ipdhcpr_netconf_sysvar array** adds DHCP server 10.1.0.2 to the relay agent's list of DHCPv4 servers:

```
IP_STATIC IP_CONST Ipcom_sysvar ipdhcpr_netconf_sysvar[] =
  {
    {"ipdhcpr.netconf.00", "server add 10.1.0.2"},
    ...
    {IP_NULL, IP_NULL}
};
```

Setting Commands in the ipdhcpr_netconf_sysvar Array

You can use the following three commands in the DHCP relay agent configuration array:

server add *address*

Add a DHCP server to the relay agent's list of DHCP servers.

interface enable *interfaceName*

Enable an interface for DHCP.

interface disable *interfaceName*

Disable an interface for DHCP. By default, all interfaces are enabled for DHCP.

Sample ipdhcpr_netconf_sysvar Array

The following example shows sample entries in an **ipdhcpr_netconf_sysvar** array:

```
IP_STATIC IP_CONST Ipcom_sysvar ipdhcpr_netconf_sysvar[] =
  {
    /* Add DHCP server 10.1.0.2 */
    {"ipdhcpr.netconf.00", "server add 10.1.0.2"},

    /* Add DHCP server 192.168.1.1 */
    {"ipdhcpr.netconf.00", "server add 192.168.1.1"},

    /* Enable interface eth1 for DHCP */
```

```
    {"ipdhcpr.netconf.00", "interface enable eth1"},

    /* Mark the end of the array */
    {IP_NULL, IP_NULL}
 };
```

6. Implementing ipdhcpr_start_hook( )

If you want to do some custom initialization at DHCP relay-agent startup, you can implement this in the form of an ipdhcpr_start_hook( ) initialization routine that the relay-agent will call when it starts. For instance, you can implement ipdhcpr_start_hook( ) to configure the relay agent by reading a configuration file at startup. The following section includes an example of this.

The signature of ipdhcpr_start_hook( ) is defined in **ipdhcpr.h**.

This routine returns zero to indicate that initialization has succeeded (or one to indicate a failure)

You can implement ipdhcpr_start_hook( ) to call any DHCP relay-agent API routine. For a list and brief descriptions of the API routines, see API Functions on page 61.

To enable your implementation of the ipdhcpr_start_hook( ) function, you must set the **Install dhcp relay callback routine** and **DHCP relay callback startup routine** configuration parameters appropriately. See DHCP Relay-Agent Configuration Parameters table on page 58.

Reading Configuration Values from a File

The following is a sample implementation of the ipdhcpr_start_hook( ) routine that configures the relay agent based on the contents of a configuration file. The routine and the configuration file are as follows:

```
IP_GLOBAL int ipdhcpr_start_hook (void)
  {
    IP_FILE *     fd          = IP_NULL;
    char *        cmdbuf      = IP_NULL;
    char          args[100];
    const char * cfgfile     = "/home/dhcpRelay/dhcpr.conf";
    const char * cmd         = "dhcpr ";
    int           argc;
    char *        tempargv    = [106 + 2];
    char **       argv        = tempargv;
    int           ret         = -1;
    if ((fd = fopen (cfgfile,"r")) == 0)
        {
        logMsg ("DHCP_RELAY ERROR: Failed to open configuration file %s\n",
               cfgfile, 0, 0, 0, 0, 0);
        goto leave;
        }
   /*
    * The config file consists of command line arguments that are
    * understood by the relay agent's shell tool
    */

   /* Read the command lines one by one and execute */
   while (fgets (args, sizeof (args), fd) != IP_NULL)
       {
```

```
       /* Remove the trailing EOLN */
       args [strlen (args) - 1] = 0;
       /* Compose a complete command args */
       cmdbuf = malloc (strlen (args) + strlen (cmd) + 1);

       if (cmdbuf == IP_NULL)
           {
           logMsg ("DHCP_RELAY ERROR: failed to allocate buffer\n",
                   0, 0, 0, 0, 0, 0);
           goto leave;
           }

       strcpy (cmdbuf, cmd);
       strcat (cmdbuf, args);

       if (getOptServ (cmdbuf, "ipdhcpr_start_hook", &argc, argv, 106 + 2)
           != OK)
          {
          logMsg ("DHCP_RELAY ERROR: failed to parse command '%s'\n",
                  args, 0, 0, 0, 0, 0);
          goto leave;
          }

      /* Let the relay agent's command tool execute */
      if (ipdhcpr_cmd_dhcpr (argc, argv) != OK)
         {
         logMsg ("DHCP_RELAY ERROR: failed to execute command '%s'\n",
                 args, 0, 0, 0, 0, 0);
          goto leave;
         }

      free (cmdbuf);
      cmdbuf = NULL;
      }

   ret = OK;

leave:

    if (fd)
       fclose (fd);

    if (cmdbuf)
       free (cmdbuf);

    return ret;
}
```

Sample Configuration File

For this example, the file path and file name are as follows:

```
/home/dhcpRelay/dhcpr.conf
```

The file content is as follows:

```
server add 10.1.0.2
   server add 192.168.1.1
   interface enable eth1
```

7. WIND RIVER DHCPV6: CLIENT

1. Introduction

This chapter describes the Wind River DHCPv6 client.

For a general overview of DHCP, see DHCP Overview on page 3. For a list of implemented RFCs, see Supported RFCs on page 2.

The Wind River DHCPv6 client runs as a single process that listens to both ICMPv6 router advertisements and routing socket messages. The DHCPv6 client sends queries to DHCPv6 servers to ask for either stateless or stateful information, depending on how you configure the client. In the case of stateful information, the client periodically communicates with the server to update configuration information and extend any existing leases.

DHCPv6 supports the DHCPv6 Client Fully Qualified Domain Name (FQDN) option (RFC 4704), which can be used by DHCPv6 clients and servers to exchange information about the client's fully qualified domain name and about which service has the responsibility for updating the DNS with the associated AAAA. The server always updates the pointer (PTR) RR.

The DHCPv6 client supports the Session Initiation Protocol (SIP) that allows SIP clients to locate SIP servers for outbound SIP requests. On the client side, you specify the SIP-enabled interfaces. On the server side, you specify the SIP server addresses and domain names.

Duplicate Address Detection

The client performs duplicate address detection on each of the addresses in any Identity Associations it receives in the Reply message before using that address for traffic. If any of the addresses are found to be in use on the link, the client sends a Decline message to the server.

2. Configuring VxWorks for the DHCPv6 Client

To add the DHCPv6 client to your VSB project, see Source Build Configuration Options (VSB) on page 4.

To add the DHCPv6 client to your VIP, see Kernel Configuration Components (VIP) on page 4.

The **INCLUDE_IPDHCPC6** component includes DHCPv6 client functionality to interface with DHCPv6 servers using the Dynamic Host Configuration Protocol.

To add optional components, see Optional Kernel Configuration Components on page 5.

DHCPv6 Client Configuration Parameters table on page 67 lists the DHCPv6 client parameters and their default values.

DHCPv6 Client API Function table on page 79 lists the API routines available to DHCPv6 client application.

> Note: For IPv6, include either the router advertisement component (**INCLUDE_IPRADVD**) or the DHCPv6 client component (**INCLUDE_IPDHCPC6)**, but not both.

3. DHCPv6 Client Configuration Parameters

DHCPv6 Client Configuration Parameters table on page 67 lists the DHCPv6 client configuration parameters and system variables.

Table 1. DHCPv6 Client Configuration Parameters

Parameter Name, System Variable Name, and Description	Default and Type
DHCPC6_CFGFILE The full path of the configuration file that VxWorks will use to configure the DHCPv6 client parameters. When no file is specified (default setting), the DHCPv6 client parameter values are based on the VIP configuration. For more information, see File-Based Configuration for the DHCP Clients on page 8.	"" char *
DHCPC6_IF_IA_PD_HINTS_DEFAULT_REBIND_LIST **ipdhcpc6.if.***interfaceName* **.ia_pd.default.hints.rebind** A list of interface Default IA_PD Rebind Hints pairs, each separated from the other by a semi-colon as follows: "eth0=8000;vlan21=4000"	"" char *

DHCPC6_IF_HINTS_DEFAULT_REBIND_LIST
ipdhcpc6.if.*interfaceName* **.ia_na.default.hints.rebind=***"time"*
For interfaces that can send hints (see the table entry for **Interface Default Hints Status List**), a desired rebind time for individual interfaces. The rebind time is similar to the renewal extension time (see the table entry for **Interface Renew List**), but applies only when the client has sent a Renew message without receiving a response (see RFC 3315).
For each interface, enter the interface name and a rebind time, in seconds. The following are configuration parameter examples:
`"eth0=7000"`
`"eth0=7000;eth1=6000;vlan21=8000"`
The following are the equivalent sysvar examples:
`sysvar set ipdhcpc6.if.eth0.ia_na.default.hints.rebind 7000`
`sysvar set ipdhcpc6.if.eth0.ia_na.default.hints.rebind 7000`
`sysvar set ipdhcpc6.if.eth1.ia_na.default.hints.rebind 6000`
`sysvar set ipdhcpc6.if.vlan21.ia_na.default.hints.rebind 8000`
	""
	char *

DHCPC6_HMAC_MD5_SECRET **ipdhcpc6.authkey** The default authentication key that the client will use for message authentication, if you configure the client for authenticated DHCPv6 operation and it receives a message from a server for which you have not configured a specific key. (To learn how to assign a key to a server, see Assigning Server-specific Authentication Keys on page 80) Specify the key as an ASCII string, starting with the prefix "0x" and followed by an even number of hexadecimal digits, for example: **"0x0123456789abcdef"**. The string length that you can give in the kernel configurator is limited to 130 bytes, which translates to a key length of up to 64 bytes. If you leave this parameter at its default value of "", the client will use only those replies from servers to which the client has a specific key mapped for DHCPv6 exchanges.	"" char *

Parameter Name, System Variable Name, and Description	Default and Type
DHCPC6_AUTHENTICATION_REALM **ipdhcpc6.authrealm** The Authentication Realm as described in section 21.4.1 of RFC 3315. This parameter, in addition to setting the DHCPv6 authentication realm, also controls whether the DHCPv6 client will generate DHCPv6 requests asking for authenticated DHCPv6 service. If you leave this parameter in its default value of `""` the server will not do authenticated exchanges. Set this parameter to anything other than `""` to turn on the authentication capability. The authentication realm is size limited to 128 bytes.	**""** char *
DHCPC6_DUID_EN_NUM **ipdhcpc6.duid.en.number** For en DUIDs, the enterprise number to use when the client generates a DUID. If you set this value, you must make sure that no value is set for the DUID EN Value (DHCPC6_DUID_EN_VAL) parameter (see the next table entry).	**"10000"** char *

DHCPC6_DUID_EN_VAL
ipdhcpc6.duid.en.value
For en DUIDs, the enterprise value to use when the client generates a DUID. If you set this value, you must make sure that no value is set for the DUID EN Number (DHCPC6_DUID_EN_NUM) parameter (see the preceding table entry).
The value entered for this parameter must be a string, but the string can contain a hexadecimal value. Each of the following are valid entries:
`"0xabcdef"`
`"duid123"`
	"0xabcdef"
	char *

DHCPC6_DUID_IF **ipdhcpc6.duid.if** For ll and llt DUIDs, the interface whose link-layer address the client is to use, for example: **"eth0"**. If you do not specify an interface, the client selects the first DHCP-enabled interface it finds.	**""** char *
DHCPC6_DUID_TYPE **ipdhcpc6.duid.type** The type of DHCP Unique ID (DUID) the client uses. The possible values are: **"ll" (link-layer)** **"llt" (link-layer plus timestamp)** **"en" (enterprise)**	**"ll"** char *

Parameter Name, System Variable Name, and Description	Default and Type
DHCPC6_INSTALL_REPLY_CALLBACK_HOOK Determine if the user-defined DHCP6 client callback routine as specified by the macro **DHCPC6_REPLY_CALLBACK_HOOK**. If FALSE, the **DHCPC6_REPLY_CALLBACK_HOOK**configuration is not applicable.	**FALSE** BOOL

DHCPC6_IF_DNS_LIST
ipdhcpc6.if.*interfaceName* **.dns**
For interfaces that are able to receive stateless information, whether or not an individual interface can request information about DNS servers and Domain Search Lists. For each interface, enter either **"enable"** or **"disable"**. The following are configuration parameter examples:
`"eth0=enable"`
`"eth0=enable;eth1=enable"`
The following are the equivalent sysvar examples:
`sysvar set ipdhcpc6.if.eth0.dns enable`
 	
`sysvar set ipdhcpc6.if.eth0.dns enable`
`sysvar set ipdhcpc6.if.eth1.dns enable`
	""
	char *

DHCPC6_IF_HINTS_DEFAULT_ENUM_LIST
ipdhcpc6.if.*interfaceName* **.ia_na.default.hints.enum.default**
For interfaces belonging to the default IANA, which interfaces can send hints to the DHCP server for preferred prefixes and timeout intervals.
For each interface, enter either **"enable"** or **"disable"**. The following are configuration parameter examples:
`"eth0=enable"`
`"eth0=enable;vlan2=disable"`
The following are the equivalent sysvar examples:
`sysvar set ipdhcpc6.if.eth0.ia_na.default.hints.enum.default enable`
 	
`sysvar set ipdhcpc6.if.eth0.ia_na.default.hints.enum.default enable`
`sysvar set ipdhcpc6.if.vlan2.ia_na.default.hints.enum.default disable`
	""
	char *

DHCPC6_IF_HINTS_DEFAULT_VALID_LIST ipdhcpc6.if.*interfaceName* .ia_na.default.hints.default.valid For interfaces that can send hints (see the table entry for **Interface Default Hints Status List**), the *valid lifetime* (see RFC 2462) that an individual interface wants the server to assign to IPv6 addresses. For each interface, enter the interface name and a lifetime. The lifetime specified for an interface must be greater than or equal to the interface's *preferred lifetime* (see the table entry for **Interface Preferred List**). The following are configuration parameter examples:	**""** char *

Parameter Name, System Variable Name, and Description	Default and Type

"eth0=6000"
"eth0=6000;eth1=5000;vlan21=7000"
 	
The following are the equivalent sysvar examples:
`sysvar set ipdhcpc6.if.eth0.ia_na.default.hints.default.valid 6000`
 	
`sysvar set ipdhcpc6.if.eth0.ia_na.default.hints.default.valid 6000`
`sysvar set ipdhcpc6.if.eth1.ia_na.default.hints.default.valid 5000`
`sysvar set ipdhcpc6.if.vlan21.ia_na.default.hints.default.valid 7000`	

DHCPC6_IF_IA_NA_DEFAULT_IAID_LIST
ipdhcpc6.if.*interfaceName* **.ia_na.default.iaid**
This parameter assigns an Identity Association ID (IAID) to individual interfaces that are part of the default IANA. The IAID for each interface must be unique among all IAIDs for IANAs. For each interface, enter the interface name and IAID. The following are configuration parameter examples:
"eth0=1"
"eth0=1;eth1=2;vlan21=3"
 	
The following are the equivalent sysvar examples:
`sysvar set ipdhcpc6.if.eth0.ia_na.default.iaid 1`
 	
`sysvar set ipdhcpc6.if.eth0.ia_na.default.iaid 1`
`sysvar set ipdhcpc6.if.eth1.ia_na.default.iaid 2`
`sysvar set ipdhcpc6.if.vlan21.ia_na.default.iaid 3`
	""
	char *

DHCPC6_IF_IA_NA_DEFAULT_LIST
ipdhcpc6.if.*interfaceName* **.ia_na.enum.default**
For each interface, enter either "**enable**" (meaning that the interface uses the default IANA) or "**disable**" (meaning that the default IANA is disabled, though you may still retrieve stateless information). The following are configuration parameter examples:
"eth0=enable"
"eth0=enable;eth1=enable"
 	
The following are the equivalent sysvar examples:
`sysvar set ipdhcpc6.if.eth0.ia_na.enum.default enable`
`sysvar set ipdhcpc6.if.eth0.ia_na.enum.default enable`
`sysvar set ipdhcpc6.if.eth1.ia_na.enum.default enable`
 	
For interfaces that are able to receive stateful information, whether or not an individual interface is part of the default IANA.
The Wind River Network Stack allows only one default Identity Association for Non-temporary Addresses (IANA) when you configure it statically. You can dynamically configure other IANAs.
	""
	char *

Parameter Name, System Variable Name, and Description	Default and Type
DHCPC6_IF_IA_PD_HINTS_DEFAULT_ENUM_LIST **ipdhcpc6.if.***interfaceName* **.ia_pd.default.hints.enum.default** A list of interface Default IA_PD Hints Status pairs, each separated by a semi-colon as follows: "eth0=enable;vlan21=automatic"	"" char *
DHCPC6_IF_IA_PD_DEFAULT_LIST **ipdhcpc6.if.***interfaceName* **.ia_pd.enum.default** A list of interface IA_PD Default Status pairs, each separated by a semi-colon as follows: "eth0=enable;vlan21=automatic"	"" char *
DHCPC6_IF_IA_PD_HINTS_DEFAULT_VALID_LIST **ipdhcpc6.if.***interfaceName* **.ia_pd.default.hints.default.valid** A list of interface Default IA_PD Valid Hints pairs, each separated by a semi-colon as follows: "eth0=8000;vlan21=7000"	"" char *
DHCPC6_IF_IA_PD_DEFAULT_IAID_LIST **ipdhcpc6.if.***interfaceName* **.ia_pd.default.iaid** A list of interface IA_PD Default IAID pairs, each separated by a semi-colon as follows: "eth0=1;vlan21=2"	"" char *
DHCPC6_IF_IA_PD_HINTS_DEFAULT_PREFERRED_LIST **ipdhcpc6.if.***interfaceName* **.ia_pd.default.hints.default.preferred** A list of interface Default IA_PD Preferred Hints pairs, each separated by a semi-colon as follows: "eth0=8000;vlan21=7000"	"" char *
DHCPC6_IF_IA_PD_HINTS_DEFAULT_PREFIX_LIST **ipdhcpc6.if.***interfaceName* **.ia_pd.default.hints.default.prefix** A list of interface Default IA_PD Prefix Hints pairs, each separated from the other by a semi-colon as follows: "eth0=3FFE:01;vlan21=3FFE:02"	"" char *
DHCPC6_IF_IA_PD_HINTS_DEFAULT_RENEW_LIST **ipdhcpc6.if.***interfaceName* **.ia_pd.default.hints.renew** A list of interface Default IA_PD Renew Hints pairs, each separated from the other by a semi-colon as follows: "eth0=8000;vlan21=4000"	"" char *

Parameter Name, System Variable Name, and Description	Default and Type

DHCPC6_IF_INFORMATION_ONLY_LIST
ipdhcpc6.if.*interfaceName* **.information_only**
Whether or not an interface only requests stateless information. For each interface, enter either **"enable"** or **"disable"**. The following are configuration parameter examples:
`"eth0=enable"`
`"eth0=enable;eth1=enable"`
The following are the equivalent sysvar examples:
`sysvar set ipdhcpc6.if.eth0.information_only enable`
 	
`sysvar set ipdhcpc6.if.eth0.information_only enable`
`sysvar set ipdhcpc6.if.eth1.information_only enable`
	""
	char *

DHCPC6_IF_INFO_REFRESH_DEFAULT_LIST
ipdhcpc6.if.*interfaceName* **.information_refresh.default**
For interfaces that are able to receive stateless information, the default time interval, in seconds, for a client to use in refreshing stateless information. The following are configuration parameter examples:
`"eth0=43200"`
`"eth0=43200;eth1=129600"`
The following are the equivalent sysvar examples:
`sysvar set ipdhcpc6.if.eth0.information_refresh.default 43200`
 	
`sysvar set ipdhcpc6.if.eth0.information_refresh.default 43200`
`sysvar set ipdhcpc6.if.eth1.information_refresh.default 129600`
	""
	char *

DHCPC6_IF_INFO_REFRESH_MAX_LIST
ipdhcpc6.if.*interfaceName* **.information_refresh.maximum**
For interfaces that are able to receive stateless information, the maximum time interval, in seconds, for a client to wait before refreshing stateless information. If no value is entered for an interface, there is no time limit on the interface. The following are configuration parameter examples:
`"eth0=1728400"`
`"eth0=1728400;eth1=1728400"`
The following are the equivalent sysvar examples:
`sysvar set ipdhcpc6.if.eth0.information_refresh.maximum 1728400`
 	
`sysvar set ipdhcpc6.if.eth0.information_refresh.maximum 1728400`
`sysvar set ipdhcpc6.if.eth1.information_refresh.maximum 1728400`
	""
	char *

Parameter Name, System Variable Name, and Description	Default and Type

DHCPC6_IF_INFO_REFRESH_MIN_LIST
ipdhcpc6.if.*interfaceName* **.information_refresh.minimum**
For interfaces that are able to receive stateless information, the minimum time interval, in seconds, for a client to use in refreshing stateless information. The following are configuration parameter examples:
`"eth0=900"`
`"eth0=900;vlan21=1200"`
The following are the equivalent sysvar examples:
`sysvar set ipdhcpc6.if.eth0.information_refresh.minimum 900`
`sysvar set ipdhcpc6.if.eth0.information_refresh.minimum 900`
`sysvar set ipdhcpc6.if.vlan21.information_refresh.minimum 1200`
	""
	char *

DHCPC6_IF_INFO_REFRESH_LIST
ipdhcpc6.if.*interfaceName* **.information_refresh**
For interfaces that are able to receive stateless information, whether or not an individual interface requests a refresh time-interval from the DHCP server. The client uses the time-interval to determine how often it needs to refresh its stateless information. For each interface, enter either "**enable**" or "**disable**". The following are configuration parameter examples:
`"eth0=enable"`
`"eth0=enable;eth1=enable"`
The following are the equivalent sysvar examples:
`sysvar set ipdhcpc6.if.eth0.information_refresh enable`
 	
`sysvar set ipdhcpc6.if.eth0.information_refresh enable`
`sysvar set ipdhcpc6.if.eth1.information_refresh enable`
	""
	char *

DHCPC6_IF_HINTS_DEFAULT_PREFERRED_LIST
ipdhcpc6.if.*interfaceName* **.ia_na.default.hints.default.preferred**
For interfaces that can send hints (see the table entry for **Interface Default Hints Status List**), the *preferred lifetime* (see RFC 2462) that an individual interface wants the server to assign to IPv6 addresses. For each interface, enter the interface name and a preferred lifetime, in seconds. The preferred lifetime specified for an interface must be less than or equal to the interface's valid lifetime (see the table entry for **Interface Hints Valid List**). The following are configuration parameter examples:
`"eth0=7000"`
`"eth0=7000;eth1=6000;vlan21=8000"`
The following are the equivalent sysvar examples:
`sysvar set ipdhcpc6.if.eth0.ia_na.default.hints.default.preferred 7000`
 	
`sysvar set ipdhcpc6.if.eth0.ia_na.default.hints.default.preferred 7000`
	""
	char *

Parameter Name, System Variable Name, and Description	Default and Type
`sysvar set ipdhcpc6.if.eth1.ia_na.default.hints.default.preferred 5000` `sysvar set ipdhcpc6.if.vlan21.ia_na.default.hints.default.preferred 8000`	

DHCPC6_IF_HINTS_DEFAULT_PREFIX_LIST
ipdhcpc6.if.*interfaceName* **.ia_na.default.hints.default.prefix**
For interfaces that can send hints (see the table entry for **Interface Default Hints Status List**), the preferred prefix for individual interfaces. For each interface, enter the interface name and the preferred prefix. The following are configuration parameter examples:
`"eth0=2001:DB8::"`
`"eth0=2001:DB8::;eth1=2001:DB8:02::"`
The following are the equivalent sysvar examples:
`sysvar set ipdhcpc6.if.eth0.ia_na.default.hints.default.prefix 2001:DB8::`
 	
`sysvar set ipdhcpc6.if.eth0.ia_na.default.hints.default.prefix 2001:DB8::`
`sysvar set ipdhcpc6.if.eth1.ia_na.default.hints.default.prefix 2001:DB8:02::`
	""
	char *

DHCPC6_IF_RAPID_COMMIT_LIST
ipdhcpc6.if.*interfaceName* **.rapid_commit**
Determines whether individual interfaces can use the DHCPv6 Rapid Commit option. For each interface, enter either "enable" or "disable". The following are configuration parameter examples:
`"eth0=enable"`
`"eth0=enable;eth1=enable"`
The following are the equivalent sysvar examples:
`sysvar set ipdhcpc6.if.eth0.rapid_commit enable`
 	
`sysvar set ipdhcpc6.if.eth0.rapid_commit enable`
`sysvar set ipdhcpc6.if.eth1.rapid_commit enable`
	""
	char *

DHCPC6_IF_HINTS_DEFAULT_RENEW_LIST
ipdhcpc6.if.*interfaceName* **.ia_na.default.hints.renew**
For interfaces that can send hints (see the table entry for **Interface Default Hints Status List**), the amount of time individual interfaces want the server to add to their valid and preferred lifetimes before the server requires address renewal. For each interface, enter the interface name and a renewal extension time, in seconds. The following are configuration parameter examples:
`"eth0=7000"`
`"eth0=7000;eth1=6000;vlan21=8000"`
The following are the equivalent sysvar examples:
	""
	char *

Parameter Name, System Variable Name, and Description	Default and Type

`sysvar set ipdhcpc6.if.eth0.ia_na.default.hints.renew 7000`
 	
`sysvar set ipdhcpc6.if.eth0.ia_na.default.hints.renew 7000`
`sysvar set ipdhcpc6.if.eth1.ia_na.default.hints.renew 6000`
`sysvar set ipdhcpc6.if.vlan21.ia_na.default.hints.renew 8000`	

DHCPC6_IF_SIP_LIST
ipdhcpc6.if.*interfaceName* **.sip**
Determines whether a client can locate a Session Interface Protocol (SIP) server to send SIP requests (see RFC 3319). For each interface, enter either "enable" or "disable". The following are the equivalent sysvar examples:
`sysvar set -c ipdhcpc6.if.eth0.sip enable`
 	
`sysvar set -o ipdhcpc6.if.eth0.sip disable`
`sysvar set -c ipdhcpc6.if.eth1.sip enable`
The **ipsip.server.address** sysvar holds the IP addresses of the valid SIP servers.
The **ipsip.server.domain** sysvar holds the domain names of the valid SIP servers.
	""
	char *

DHCPC6_IF_SNTP_LIST
ipdhcpc6.if.*interfaceName* **.sntp**
For interfaces that are able to receive stateless information, whether or not an individual interface can request information about available SNTP (Simple Network Time Protocol) servers. For each interface, enter either "enable" or "disable". The following are configuration parameter examples:
`"eth0=enable"`
`"eth0=enable;eth1=enable"`
The following are the equivalent sysvar examples:
`sysvar set ipdhcpc6.if.eth0.sntp enable`
 	
`sysvar set ipdhcpc6.if.eth0.sntp enable`
`sysvar set ipdhcpc6.if.eth1.sntp enable`
	""
	char *

DHCPC6_IF_ENUM_LIST **ipdhcpc6.if.enum.***interfaceName* Whether or not individual interfaces are enabled for DHCP. For each interface, enter one of the following values: "**enable**", "**disable**", or "**automatic**". For the interpretation of "automatic", see Automatic Mode on page 79. The following are configuration parameter examples: `"eth0=automatic" "eth0=automatic;eth1=enable"` The following are the equivalent sysvar examples:	"" char *

Parameter Name, System Variable Name, and Description	Default and Type
`sysvar set ipdhcpc6.if.enum.eth0 automatic sysvar set ipdhcpc6.if.enum.eth0 automatic sysvar set ipdhcpc6.if.enum.eth1 enable`	
DHCPC6_REPLY_CALLBACK_HOOK This callback can be used to change the contents of the DHCPc6 client reply. If not specified, the internal reply callback routine will be used. The DHCPC6_INSTALL_REPLY_CALLBACK_HOOK must also be set to **TRUE** in order to install this callback routine.	**ipdhcpc6_reply_default_callback** funcptr

4. Shell Commands

You can use the dhcpc6 shell command to manage a DHCPv6 client.

You can use the ipd shell command to start and stop the DHCPv6 client daemon.

You can use the syslog shell command to set the priority for the DHCPv6 client syslog debug messages.

To run shell commands, you must access the command-interpreter mode. To access the command-interpreter mode from the kernel shell, enter the following command:

```
-> cmd
[vxWorks *} #
```

To return to the kernel shell, enter the following command:

```
# C
->
```

Using DHCPv6 Client Shell Commands

To obtain a list of dhcpc6 shell commands and *help* for the commands, enter the following command:

```
# dhcpc6
```

The available commands are as follows:

```
dhcpc6 show
dhcpc6 history [show|flush]
dhcpc6 release
dhcpc6 refresh
dhcpc6 bind add|delete
```

You can use shell commands to do the following:

- View the current configuration of client interfaces.
- View or delete the history of the client interfaces.

- Release the current configuration of client interfaces.
- Reinitialize or confirm the current bindings.
- Configure the downlink interfaces that are bound to an uplink interface.

Example commands and the resulting output follow:

dhcpc6 show

Show the current configuration of all DHCPv6 interfaces or of a specified interface. For example, to show the configuration of interface eth0, with verbose reporting turned on, enter:

```
# dhcpc6 -v show eth0
```

The following is sample output from a dhcpc6 show command:

```
[vxWorks *]# dhcpc6 show vlan100 ia_na default
  Interpeak DHCPv6 Client:
  DUID     : 06:30:00:10:00:00:00:00:00:02
  Interface: vlan100
    Mode             : enabled
    Link State       : up
    Non-Temporary Identity Association default, 1:
      Status : bound/idle
      Binding:
        DHCP Server : 2001:DB8::200:FF:FE00:3
        Renew/Rebind: 10/16
        Lease       : 2001:DB8:2::D/64 preferred 20 valid 40
```

dhcpc6 history show

Show the history of all DHCPv6 interfaces or of a specified interface. For example, to show the history of all client interfaces, enter:

```
# dhcpc6 history show
```

The following is sample output from this shell command:

```
[vxWorks *]# dhcpc6 history show vlan100 ia_na default
  Interpeak DHCPv6 Client:
  DUID     : 06:30:00:10:00:00:00:00:00:02
  Interface: vlan100
    Non-Temporary Identity Association default, 1:
      History(6):
        Sun Feb 18 21:32:11 2007: solicit/idle
        Sun Feb 18 21:32:11 2007: solicit/solicit
        Sun Feb 18 21:32:11 2007: solicit/soliciting
        Sun Feb 18 21:32:14 2007: solicit/request
        Sun Feb 18 21:32:14 2007: solicit/requesting
        Sun Feb 18 21:32:14 2007: bound/idle
```

dhcpc6 history flush

Delete the history of all DHCPv6 interfaces or of a specified interface. The following example deletes the history of the default IANA on interface vlan100:

```
# dhcpc6 history flush vlan100 ia_na default
```

dhcpc6 refresh

Request confirmation of current configuration information. If confirmation is not received, the command tries to acquire information from the network. The following example requests confirmation of non-temporary information on interface eth0, restricted to the default IANA.

```
# dhcpc6 refresh eth0 ia_na default
```

dhcpc6 release

Releases current configuration information for all interfaces or for a specified interface. For example, to release all configuration information, enter:

```
# dhcpc6 release
```

Stopping and starting the DHCPv6 Client Daemon

Use the ipd command to stop or start the DHCPv6 client daemon (**ipdhcpc6**). For example:

```
# ipd list
Services:
ipdhcpc6             started
```

```
# ipd kill ipdhcpc6

ipd: kill ipdhcpc6 ok
```

```
# ipd list

Services:
ipdhcpc6             killed
```

```
# ipd start ipdhcpc6

ipd: start ipdhcpc6 ok
```

To view the task information for the DHCPv6 client, run the task command and locate the **ipdhcpc6** task. For example:

```
# task

NAME      ENTRY         TID      PRI  STATUS  PC        SP        ERRNO  DELAY
...
ipdhcpc6  ipdhcpc6_pr>  105707d0 50   PEND    102b87eb  109efb20  16     0
```

Setting the DHCPv6 Client System Log Priority

Run the syslog list command to get the current priority level for all modules:

```
# syslog list ipdhcpc6
    syslog facility priority
    ...
    ipdhcpc6 Error
```

To list the available syslog priority levels, use the following command:

```
# syslog priority *
syslog: missing <prio>
        priorities: Emerg Crit Error Warning Notice Info Debug Debug2
```

Use the following command syntax to change the priority level for the DHCPv6 client:

```
# syslog priority ipdhcpc6 priority
```

For more information on the syslog shell command, see the *VxWorks 7 Network Stack Programmer's Guide*.

5. API Function

DHCPv6 Client API Functions table on page 79 lists the API functions available to DHCPv6 client applications and gives a brief description of each routine. For more information, see the reference entries for the DHCPv6 client (**ipdhcpc6**).

The DHCPv6 client public API routines are defined in **ipdpcpc6_cmd.c**.

Table 1. DHCPv6 Client API Functions

Function	Description
ipdhcpc6_iapd_bind_add( )	Add the downstream interface(s) to the binding configuration for an IA_PD. This binding associates a distinct IA_PD with each of its downstream network interfaces and uses that IA_PD to obtain a prefix for the interface(s) from the delegating router interface.
ipdhcpc6_iapd_bind_delete( )	Remove the downstream interface(s) from a binding configuration for an IA_PD.
ipdhcpc6_refresh( )	Request confirmation of the current configuration.
ipdhcpc6_release( )	Release the current interface association.
ipdhcpc6_reply_callback( )	Examine each DHCPv6 reply returned by a DHCPv6 server and execute user-defined operations for specific replies.

6. Automatic Mode

Router advertisement messages contain an **M** field, for a **Managed address configuration** flag, and an **O** field, for an **Other stateful configuration** flag (see RFC 2461: *Neighbor Discovery for IP Version 6*). When you set a static build parameter to automatic, the affected interfaces operate in accord with the **M** and **O** settings in the most recently received router advertisement message. The settings are interpreted as follows:

- If the **Managed address configuration** flag (**M**) is set, interfaces configure IPv6 addresses using stateful autoconfiguration.
- If the **Other stateful configuration** flag (**O**) is set, interfaces use stateful autoconfiguration to obtain information other than IPv6 addresses.

If the **Managed address configuration** flag is set, the **Other stateful configuration** flag must also be set. The client cannot use stateful address autoconfiguration without accepting additional stateful information from a DHCPv6 server.

7. Assigning Server-specific Authentication Keys

To assign an authentication key to a specific DHCPv6 server, based on its server DUID, use the following routines:

ipdhcpc6_authdb_create_ll_duid( )

```
/* create a DUID based on link-layer address: section 9.4 RFC3315 */
Ipdhcpc6_duid *
    ipdhcpc6_authdb_create_ll_duid (int len, Ip_u16 hwtype, Ip_u8 * macaddr)
```

The parameters to this routine are as follows:

len – the length of the hardware address (mac address)

hwtype – the hardware type code, usually 0x1 (ethernet)

macaddr – a pointer to a byte string containing the hardware address

Note: The len parameter is currently ignored; ipdhcpc6_authdb_create_ll_duid( ) uses only the first six bytes of **macaddr** to identify the server.

ipdhcpc6_authdb_create_ll_duid( ) returns a pointer to the newly created DUID structure, or **IP_NULL** if the routine fails.

ipdhcps6_authdb_add_server_authkey( )

```
/* associate an authentication key with a specific server */
Ip_err ipdhcps6_authdb_add_server_authkey (Ipdhcpc6_duid * server_duid, Ip_u8 * key, int keylen)
```

Given a previously created server DUID structure, and a key/key length pair, this routine creates an entry in the internal authentication database assigning the key to the given server.

This routine returns **IPCOM_SUCCESS** if successful, or **IPCOM_ERR_FAILED** if not.

Assigning an Authentication Key to a Server

In this example, the server has an ethernet mac address of 01:02:03:04:05:06, and we want to assign a specific authentication key of **"this is a secret"**.

```
void assign_example (void)
{
     Ipdhcpc6_duid * server_duid;

     Ip_u8 macaddr[] = { 0x1, 0x2, 0x3, 0x4, 0x5, 0x6 };
```

```
    char * key      = "this is a secret";
    int keylen      = strlen (key);

    server_duid = ipdhcpc6_authdb_create_ll_duid (6, 0x1, macaddr);
    if (IP_NULL == client_duid)
        {
        LOG_ERROR_SOMEHOW ("could not create server duid");
        return;
        }
    if (IPCOM_SUCCESS !=
        ipdhcpc6_authdb_add_server_authkey (server_duid, (IP_u8 *) key, keylen))
    {
        LOG_ERROR_SOMEHOW ("couldnt associate key with server duid");
    }
```

The return value from ipdhcpc6_authdb_create_ll_duid( ) is a pointer to a memory area obtained with a call to ipcom_malloc( ). After you call ipdhcps6_authdb_add_server_authkey( ) to associate the key with the server DUID, you may release the memory region containing the DUID by calling ipcom_free( ). However, you may wish to keep it around for a later call to ipdhcps6_authdb_server_no_auth( ).

Note: This assignment procedure is sufficient for servers that send DUIDs of type LL (link-layer-address) or LLT (link-layer-address + time). Assigning a server-specific key to servers that send an EN (enterprise number) type DUID is currently not supported.

ipdhcps6_authdb_server_no_auth

```
/* To delete a specific servers auth key entry */
    Ip_err ipdhcps6_authdb_server_no_auth (Ipdhcps6_duid * client_duid);
```

This call deletes any server-specific key associated with the given server DUID, then returns **IPCOM_SUCCESS** if successful, or **IPCOM_ERR_FAILED** otherwise.

Run-time Configuration of the DHCPv6 Authentication Parameters

A DHCPv6-enabled device will probably need to configure itself in the field, after manufacture. If you set the **DHCPC6_AUTHENTICATION_REALM** configuration parameter to anything other than the default setting (**""**), the DHCPv6 client task will block on startup, waiting for this run-time configuration to take place. This blocking behavior does not occur if you set **DHCPC6_AUTHENTICATION_REALM** to the default (**""**).

After a device completes such configuration, it must call ipdhcpc6_user_authdb_config_finished( ) to notify the DHCPv6 client that configuration is complete. Only then will the DHCPv6 client become operational.

Write a routine to do this run-time configuration, and invoke this routine at device startup, for instance, from usrAppInit( ).

The simplest run-time configuration routine (which does nothing except to notify the DHCPv6 client that it can go ahead and start up) would be the following:

```
extern void ipdhcpc6_user_authdb_config_finished (void);
runtimeConfigureDHCPV6auth (void)
     {
     ipdhcpc6_user_authdb_config_finished ();
     }
```

A device that needs to configure authentication parameters, but does not have any information about its configuration available at kernel configuration time, must set the **DHCPC6_AUTHENTICATION_REALM** kernel configuration parameter to some non-empty-

string value, such as **"yes"**, and then provide a runtime configuration routine that configures the device at startup using values obtained from the end user, NVRAM storage, and so forth.

Call the following routines to configure the authentication parameters:

```
Ip_err ipdhcpc6_set_auth_realm (int len, Ip_u8 * realm);
```

Set the authentication realm for the DHCPv6 client. This overwrites the **DHCPC6_AUTHENTICATION_REALM** configuration parameter.Set the authentication realm for the DHCPv6 client. This overwrites the **DHCPC6_AUTHENTICATION_REALM** configuration parameter.

Parameters:

len – the length of the byte string pointed to by the realm parameter.

realm – a pointer to the realm string. It does not need to be null-terminated.

This routine returns **IPCOM_SUCCESS** if successful, or **IPCOM_ERR_FAILED** if the supplied realm length is too large.

The **DHCPC6_AUTHENTICATION_REALM** parameter is limited to 128 bytes.

```
Ip_err ipdhcpc6_set_default_auth_key (int len, Ip_u8 * key);
```

Set the default authentication key for the DHCPv6 client. This overwrites the **DHCPC6_HMAC_MD5_SECRET** configuration parameter.

Parameters:

len – the length of the byte string pointed to by the realm parameter. The key length is limited to 64 bytes.

key – a pointer to the authentication key, supplied as a raw byte sequence. It may contain null bytes.

This routine returns IPCOM_SUCCESS if successful, or **IPCOM_ERR_FAILED** if the supplied key is too large.

```
Ipdhcpc6_duid * ipdhcpc6_authdb_create_ll_duid
                         (int len, Ip_u16 hwtype, Ip_u8 * macaddr)

Ip_err ipdhcps6_authdb_add_server_authkey
                         (Ipdhcpc6_duid * server_duid, Ip_u8 * key, int keylen))
```

These two routines are described in Assigning Server-specific Authentication Keys on page 80.

```
void ipdhcpc6_user_authdb_config_finished (void);
```

Call this routine to notify the DHCPv6 client that initial run-time configuration is complete, and so the DHCPv6 client may start.

Initializing the client replay-detection counters

DHCPv6 authentication incorporates an anti-replay mechanism to protect against replay attacks.

Each client/server pair maintains an 8-byte counter which must monotonically increase with each DHCPv6 message sent. This can be problematic if a client has previously established a session with a given server, and then experiences a reboot. Unless the client has some way of initializing the starting value of this counter to a value greater than its value before it rebooted, it may be refused authenticated service with that server.

Wind River recommends that you tie the replay counter to the time in some fashion. Some DHCPv6 clients and servers use an NTP format timestamp as their anti-replay counter. If your target environment has a time/date reference available to it, or a number-of-

reboots counter, or something of this sort, Wind River recommends that you use that information at start up to initialize the anti-replay counter. Use the following routine to set the anti-replay counter:

```
IP_GLOBAL void ipdhcpc6_set_replay_counters (Ip_u32 high, Ip_u32 low);
```

- **high** –upper 4 bytes of the anti-replay counter
- **low** – lower 4 bytes of the anti-replay counter

You can obtain the current value of the replay counter at any time by calling the following routine:

```
IP_GLOBAL void ipdhcpc6_get_replay_counters (Ip_u32 *high, Ip_u32 *low);
```

- **high** – pointer to location where upper 4 bytes of the anti-replay counter will be written
- **low** – pointer to location where lower 4 bytes of the anti-replay counter will be written

8. Implementing FQDN

The Domain Name System (DNS) maintains mapping between a host's Fully Qualified Domain Name (FQDN) and an IPv6 address. The information is maintained in two types of Resource Records (RRs); AAAA and a pointer resource record (PTR).

DHCPv6 provides a mechanism whereby a DHCPv6 client and server can to exchange information about the client's fully qualified domain name and about who has the responsibility for updating the DNS with the associated AAAA resource record (RR). An AAAA record associates a domain name to an IPv6 address whereas a PTR resource record associates an IPv6 address to a domain name. The server always updates the PTR RR.

The primary options for updating the resource records are as follows:

1. The DHCPv6 client updates the AAAA RR and the DHCPv6 server updates the PTR RR.
   The client sends the FQDN option containing the domain name to the server and the server includes the FQDN option in the response message. Once the DHCPv6 configuration is completed, the client gets the complete fully qualified domain name from the server, then updates the AAAA RR. If the update fails, the client informs the administrator.
2. The DHCPv6 server updates both the AAAA and the PTR RRs.
   The client sends the FQDN option to the server indicating that the server must perform the AAAA RR update. The server sends a reply message to the client then updates the AAAA and PTR RRs. If the update fails, the server does not send any indication to the client but informs the administrator.

Note: Only one IPv6 address is assigned to a FQDN.

Controlling the FQDN Flags with System Variables

The Client FQDN option includes system variables that DHCPv6 clients and servers use to negotiate the updates of the AAAA RR. Each FQDN system variable includes the name of the interface (*ifName*). The format of the FQDN system variables is as follows.

For the client we have:

- **ipdhcpc6.if***ifName***.fqdn.fqdn**: Set to **enable** to enable the FQDN option for the client. Otherwise, set to **disable**. The default setting is *disable*.
- **ipdhcpc6.if***ifName***.fqdn.request**: Set to **enable** to inform the server that FQDN is a valid option. The information is sent as part of an Option Request Option (ORO). Otherwise, set to **disable**. The default setting is **enable**.

- **ipdhcpc6.if**.*ifName*.**fqdn.update**: Set to enable to indicate that the client is responsible for updating the AAAA RR. Set to **disable** to indicate that the server is responsible for updating the AAAA RR. The default setting is **disable**. This system variable controls the client-side S bit described in RFC 4704, *The Dynamic Host Configuration Protocol for IPv6 (DHCPv6) Client Fully Qualified Domain Name (FQDN) Option*.

For the server we have:

- **ipdhcpc6.if**.*ifName*.**fqdn.allow.update**: Set to **enable** to indicate that the server is responsible for updating the AAAA RR. Otherwise, set to **disable**. The default setting is **enable**. This system variable controls the server-side S bit described in RFC 4704, *The Dynamic Host Configuration Protocol for IPv6 (DHCPv6) Client Fully Qualified Domain Name (FQDN) Option*.

The N and O bits described in RFC 4704 are preset to 0 and cannot be changed. Their description in RFC 4704 is as follows:

The O bit indicates whether the server has overridden the client's preference for the S bit. A client must set this bit to 0. A server must set this bit to 1 if the S bit in its reply to the client does not match the S bit received from the client.

The N bit indicates whether the server should not perform any DNS updates. A client sets this bit to 0 to request that the server should perform updates (the PTR RR and possibly the AAAA RR based on the S bit) or to 1 to request that the server should not perform any DNS updates. A server sets the N bit to indicate whether the server shall (0) or shall not (1) perform DNS updates. If the N bit is 1, the S bit must be 0.

Configuring the Client to Update the AAAA RR

The following system variable settings cause the client to control the updates to the AAAA RR:

Client:

```
sysvar set -co ipdhcpc6.if.ifName.fqdn.fqdn enable
sysvar set -co ipdhcpc6.if.ifName.fqdn.update enable
sysvar set -co ipdhcpc6.if.ifName.fqdn.request enable
```

Server:

```
sysvar set -co ipdhcps6.if.ifName.fqdn.allow.update disable
```

The server always updates the PTR RR.

For example usage of this configuration in a vxsim environment, see Use Case - Allowing the Client to Update the AAAA RR on page 85.

Configuring the Server to Update the AAAA RR

The following system variable settings cause the server to control the updates to the AAAA RR:

Client:

```
sysvar set -co ipdhcpc6.if.ifName.fqdn.fqdn enable
sysvar set -co ipdhcpc6.if.ifName.fqdn.update disable
sysvar set -co ipdhcpc6.if.ifName.fqdn.request enable
```

Server:

```
sysvar set -co ipdhcps6.if.ifName.fqdn.allow.update enable
```

The server always updates the PTR RR.

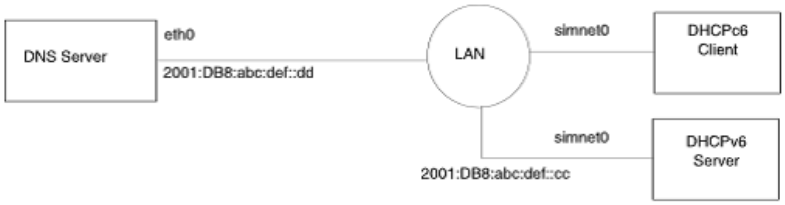
For example usage of this configuration in a vxsim environment, see Use Case - Allowing the Server to Update the AAAA RR on page 87.

9. Use Case - Allowing the Client to Update the AAAA RR

This scenario specifies how to use the FQDN system variables to allow the DHCPv6 client to update the AAAA resource record (RRs). The pointer record (PTR) is always updated by the server.

Topology for the FQDN Scenario on page 85 shows the topology for the scenario. The DNS server operates on a separate host whereas the DHCPv6 client and server operate from VxSim.

Figure 1. Topology for the FQDN Scenario



Configuring the VSB

Enable the following components in the VSB:

- **IPNET_DHCPS6**
- **IPNET_DHCPC6**
- **FEATURE_IPNET_INET6**

Configuring the VIP

Include the following components in the VIP:

- **INCLUDE_IFCONFIG**
- **INCLUDE_IPCOM_SYSVAR_CMD**
- **INCLUDE_IPD_CMD**
- **INCLUDE_IPDHCPC6**
- **INCLUDE_IPDHCPS6**
- **INCLUDE_IPDHCPC6_CMD**
- **INCLUDE_IPDHCPS6_CMD**
- **INCLUDE_IPIFCONFIG_CMD**
- **INCLUDE_PING6**
- **INCLUDE_SHELL**
- **INCLUDE_SHELL_INTERP_CMD**

Configuring the DNS Server

Configure a Linux DNS Server with static address 2001:DB8:abc:def::dd which is able to accept AAAA RR and PTR RR updates.

Configuring the DHCPv6 Server

Execute the following DHCPv6 server shell commands to configure the DHCPv6 server interface and the FQDN option:

```
# ifconfig simnet0 inet6 add 2001:DB8:abc:def::cc up
# sysvar set -o ipdhcps6.duid.ifname simnet0
# dhcps6 start
# dhcps6 subnet simnet0 pool add 2001:DB8:abc:def::1 2001:DB8:abc:def::10 100 200
# dhcps6 option add dns 2001:DB8:abc:def::dd primary
# sysvar set -co ipdhcps6.fqdn.allow.update disable
# sysvar set -co ipdnsc.primaryns 2001:DB8:abc:def::dd
# ipd reconfigure ipdhcps6
```

Configuring the DHCPv6 Client

Execute the following DHCPv6 client shell commands to configure the DHCPv6 client interface and the FQDN option to allow the client to update the AAAA RR:

```
# sysvar set -o ipdhcpc6.duid.if simnet0
# sysvar set -co ipdhcpc6.if.enum.simnet0 enable
# sysvar set -co ipdhcpc6.if.simnet0.ia_na.enum.default enable
# sysvar set -co ipdhcpc6.if.simnet0.fqdn.fqdn enable
# sysvar set -co ipdhcpc6.if.simnet0.fqdn.update enable
# sysvar set -co ipdhcpc6.if.simnet0.fqdn.request enable
# sysvar set -co ipdnsc.primaryns 2001:DB8:abc:def::dd
```

Initiating the Negotiation Proces

Execute the following DHCPv6 client shell command to reconfigure the client and initiate the DHCPv6 client to DHCPv6 server negotiation process.

```
# ipd reconfigure ipdhcpc6
```

Verifying the Results

After the client is reconfigured, the client is assigned an IPv6 address and an associated domain name that is maintained by the DHCPv6 client. If the update fails, the client informs the administrator.

Verifying the Results on the DHCPv6 Server Side

From the DHCPv6 server, check the server configuration:

```
# dhcps6 show
```

The output should be similar to the following:

```
Global option(s)
Option(s) on link simnet0
Address pool on link simnet0
   from 2001:db8:abc:def::1 to 2001:db8:abc:def::10 preferred: 100s, valid: 200s
Defined prefix(es)
Known user(s)
Known host(s)
OPTION_CLIENTID
        00:03:00:01:7A:7A:C0:A8:C8:01
        addr: fe80::787a:c0ff:fea8:c801%simnet0
```

```
        FQDN: VxSim1.windriver.com
        last served 18 seconds ago
        lease addr: 2001:db8:abc:def::c, preferred: 100s, valid: 200s
```

Verifying the Results on the DHCPv6 Client Side

From the DHCPv6 client, check the client configuration:

```
# dhcpc6 -v show
Interpeak DHCPv6 Client:
   DUID        : 00:03:00:01:7a:7a:c0:a8:c8:01
   Interface   : simnet0
   Mode        : enabled
   Link State  : up
Configuration:
  FQDN         : enabled
  DHCP Server  : fe80::787a:c0ff:fea8:c805
  FQDN         : VxSim1.windriver.com
  Non-Temporary Identity Association default, 46138085:
  Status       : bound/idle
Binding:
  DHCP Server  : fe80::787a:c0ff:fea8:c805
  Renew/Rebind : 50/80
  Lease        : 2001:db8:abc:def::c/64 preferred 100 valid 200
```

10. Use Case - Allowing the Server to Update the AAAA RR

This scenario specifies how to use the FQDN system variables to allow the DHCPv6 server to update the AAAA resource record (RR).

The only differences between this scenario and the scenario in Use Case - Allowing the Client to Update the AAAA RR on page 85 are as follows.

In section Configuring the DHCPv6 Client on page 85 change:

```
# sysvar set -co ipdhcps6.fqdn.allow.update disable
```

to

```
# sysvar set -co ipdhcps6.fqdn.allow.update enable
```

In section Configuring the DHCPv6 Client on page 86 change:

```
# sysvar set -co ipdhcpc6.if.simnet0.fqdn.update enable
```

to

```
# sysvar set -co ipdhcpc6.if.simnet0.fqdn.update disable
```

The expected results from the show commands are the same.

If the update fails, the server does not send any indication to the client but informs the administrator.

8. WIND RIVER DHCPV6: SERVER AND RELAY AGENT

1. Introduction

This chapter describes the Wind River DHCPv6 server implementation.

For a general overview of DHCP, see DHCP Overview on page 3. For a list of implemented RFCs, see Supported RFCs on page 2.

The Wind River DHCPv6 *server* provides IP addresses and option settings to DHCPv6 clients.

DHCPv6 Server Enhancements

Support for multicast and the creation of link-local addresses during initialization allow a client to use its link-local address and a well-known multicast address to discover and communicate with DHCP servers or relay agents on its link

A node may autoconfigure addresses based on router advertisements and make use of a valid lifetime period to support renumbering of addresses on the Internet. In addition, the protocol interaction by which a node begins stateless or stateful autoconfiguration is specified. DHCP is one vehicle to perform stateful autoconfiguration. Compatibility with stateless address autoconfiguration is a design requirement of DHCP.

IPv6 Neighbor Discovery is the node discovery protocol in IPv6 which replaces and enhances the functions of ARP.

Dynamic Updates to DNS supports the dynamic update of DNS records for both IPv4 and IPv6. DHCP can use the dynamic updates to DNS to integrate addresses and *name space* to not only support autoconfiguration, but also for autoregistration in IPv6.

DHCPv6 supports the DHCPv6 Client Fully Qualified Domain Name (FQDN) option (RFC 4704), which can be used by DHCPv6 clients and servers to exchange information about the client's fully qualified domain name and about which service has the responsibility for updating the DNS with the associated AAAA resource record (RR). The server always updates the pointer (PTR) RR.

The DHCPv6 server supports the Session Initiation Protocol (SIP) that allows SIP clients to locate SIP servers for outbound SIP requests (see RFC 3319). On the server side, you specify the SIP server addresses and domain names. On the client side, you specify the SIP-enabled interfaces.

Relay Agents in a DHCPv6

The relay agent may be configured to use a list of destination addresses, which may include unicast addresses, the All_DHCP_Servers multicast address, or other addresses selected by the network administrator. If the relay agent has not been explicitly configured, it must use the All_DHCP_Servers multicast address as the default.

If the relay agent relays messages to the All_DHCP_Servers multicast address or other multicast addresses, it sets the hop count to 32 (**DHCPS6_MAXHOP_COUNT**).

2. Configuring VxWorks for the DHCPv6 Server

To add the DHCPv6 server to your VSB project, see Source Build Configuration Options (VSB) on page 4.

To add the DHCPv6 server to your VIP, see Kernel Configuration Components (VIP) on page 4.

The **INCLUDE_IPDHCPS6** component includes DHCPv6 server functionality to interface with DHCPv6 clientss using the Dynamic Host Configuration Protocol.

To add optional components, see Optional Kernel Configuration Components on page 5.

DHCPv6 Server Configuration Parameters on page 89 lists the DHCPv6 server parameters and their default values.

DHCPv6 Server API Routines table on page 92 lists the API routines available to DHCPv6 server application.

DHCPv6 Server Configuration Parameters

DHCPv6 Server Configuration Parameters on page 89 lists the DHCPv6 server configuration parameters and system variables.

Table 1. DHCPv6 Server Configuration Parameters

Parameter Name, System Variable Name, and Description	Default and Type
DHCPS6_ALLOW_RAPID_COMMIT **ipdhcps6.server.allow_rapid_commit** If set to "yes", the server allows rapid commits.	**"yes"** char *
DHCPS6_HMAC_MD5_SECRET **ipdhcps6.authkey** The default authentication key that the server will use for message authentication, if a client requests authentication but the server does not know of a client-specific key for that client. (To set a client-specific authentication key for a client, see Assigning Client-Specific Authentication Keys on page 93.) Specify the key as an ASCII string, starting with the prefix "**0x**" and followed by an even number of hexadecimal digits, for example: **"0x0123456789abcdef"**. The string length that you can give in the kernel configurator is limited to 130 bytes, which translates to a key length of up to 64 bytes. If you leave this parameter at its default value of **""**, only clients that have had a specific key assigned to them will be able to use authenticated DHCPv6 communications.	**""** char *
DHCPS6_AUTHENTICATION_REALM **ipdhcps6.authrealm** The Authentication Realm as described in section 21.4.1 of RFC 3315. This parameter, in addition to setting the DHCPv6 authentication realm, also controls whether the DHCPv6 server will generate DHCPv6 replies using authentication at all. If you leave this parameter in its default value of "" the server will not do authenticated exchanges. Set this parameter to anything other than "" to turn on the authentication capability.	**""** char *
DHCPS6_MAXHOP_COUNT **ipdhcps6.relay.hop_count_limit** (Relay agents only) The maximum number of hops allowed when forwarding DHCPv6 requests.	**"32"** char *

Parameter Name, System Variable Name, and Description	Default and Type
DHCPS6_DUID_IFNAME **ipdhcps6.duid.ifname** The name of the interface whose link-layer address the server is to use as its DHCPv6 unique ID (DUID), for example: "**eth0**". You must enter a value for this parameter or the DHCP server will not start. When the server starts, the DUID is written to the system log.	**""** char *

DHCPS6_IF_RELAY_MAP_LIST
ipdhcps6.relay.map.in*InterfaceName*
(Relay agents only) For each interface that receives DHCP messages, this parameter specifies either an outgoing interface to use for multicasting messages or a unicast destination address.
The format for a configuration parameter entry is:
`"inInterfaceName=[outInterfaceName|destinationAddress]"`
Enter interface pairs as a string, with individual entries separated by semicolons. The following are configuration parameter examples:
`"eth0=eth1"`
`"eth0=eth1;eth2=2001:DB8:123::2:1"`
The following are the equivalent sysvar examples:
`sysvar set ipdhcps6.relay.map.eth0 eth1`
`sysvar set ipdhcps6.relay.map.eth0 eth1`
`sysvar set ipdhcps6.relay.map.eth2 2001:DB8:123::2:1`
	""
	char *

DHCPS6_MODE **ipdhcps6.mode** Determines whether the node runs as a DHCPv6 server ("server") or relay agent ("relay").	**"server"** char *

3. Shell Commands

.You can use the dhcps6 shell command to manage the DHCPv6 server.

You can use the ipd shell command to start and stop the DHCPv6 server daemon.

You can use the syslog shell command to set the priority for the DHCPv6 server syslog debug messages.

To run shell commands, you must access the command-interpreter mode. To access the command-interpreter mode from the kernel shell, enter the following command:

```
-> cmd
[vxWorks *} #
```

To return to the kernel shell, enter the following command:

```
# C
->
```

Using DHCPv6 Server Shell Comands

To obtain a list of dhcps6 shell commands and *help* for the commands, enter the following command:

```
# dhcps6
```

The available commands are as follows:

```
dhcps6 { start | stop | show }
dhcps6 { subnet | prefix | option | host } list
dhcps6 option { add | delete } <opt_param>
dhcps6 pool { add | delete } <pool_param>
dhcps6 prefix delete <id>
dhcps6 prefix add <id> <prefix_spec> [ preferred <sec> ] [ valid <sec> ]
dhcps6 subnet { add | delete } <addr6>
dhcps6 subnet <addr6 | ifname> { option | pool | prefix } list
dhcps6 subnet <addr6 | ifname> { option | pool | prefix } { add | delete }
              <opt_param | pool_param>
dhcps6 user <username> prefix <id>
```

You can use the dhcps6 option shell command to configure global options that apply across all subnets in the configuration database. You can use the dhcps6 subnet shell command to configure options that apply to a specific subnet and override global option settings.

Stopping and Starting the DHCPv6 Server Daemon

Use the ipd command to stop or start the DHCPv6 server daemon (ipdhcps6). For example:

```
# ipd list
Services:
ipdhcps6              started

# ipd kill ipdhcps6
ipd: kill ipdhcps6 ok

# ipd list
Services:
ipdhcps6              killed

# ipd start ipdhcps6
ipd: start ipdhcps6 ok
```

To view the task information for the DHCPv6 server, run the task command and locate the ipdhcps6 task. For example:

```
# task
NAME     ENTRY     TID      PRI STATUS PC       SP       ERRNO  DELAY
...
ipdhcps6 100376d0  1090f648 50  PEND+T 102b87eb 109afb20 3d0004 151
```

Setting the DHCPv6 Server System Log Priority

Run the syslog list command to get the current priority level for all modules:

```
# syslog list ipdhcps6
    syslog facility priority
    ...
    ipdhcps6 Error
```

To list the available syslog priority levels, use the following command:

```
# syslog priority *

syslog: missing <prio>
        priorities: Emerg Crit Error Warning Notice Info Debug Debug2
```

Use the following shell command syntax to change this setting:

```
# syslog priority ipdhcps6 priority
```

For more information on the syslog shell command, see the *VxWorks 7 Network Stack Programmer's Guide*.

4. API Functions

For more information, see the reference entries for the DHCPv6 server (**ipdhcps6**).

DHCPv6 Server API Routines table on page 92 lists the API routines available to DHCPv6 server applications and gives a brief description of each routine. The DHCPv6 server public API routines are defined in the files found in *installDir* **/vxworks-7/pkgs_v2/net/ipnet/app/dhcp-2.0.0.3/dhcps6/src/**.

Table 1. DHCPv6 Server API Routines

Function	Description
ipdhcps6_subnet_from_ifname( )	Creates the subnet information that represents the specified link.
ipdhcps6_subnet_add( )	Adds a new subnet to the DHCP server configuration.
ipdhcps6_subnet_delete( )	Deletes a subnet from the DHCP server configuration. All options and address pools associated with the subnet are also deleted.
ipdhcps6_subnet_list( )	Lists all added subnets.
ipdhcps6_subnet_to_string( )	Returns a string representation of the specified subnet.
ipdhcps6_pool_add( )	Adds a pool of addresses to the specified subnet.
ipdhcps6_pool_delete( )	Deletes a DHCPv6 pool from a specified subnet.

Function	Description
ipdhcps6_pool_get( )	Gets the address pool specification on a specific subnet.
ipdhcps6_option_add( )	Adds a DHCPv6 option either globally or to a specified subnet.
ipdhcps6_option_delete( )	Deletes a DHCPv6 option from a specified subnet.
ipdhcps6_option_list( )	Lists all DHCPv6 options assigned to the specified subnet.
ipdhcps6_option_to_string( )	Returns a string representation of the specified option.
ipdhcps6_prefix_add( )	Adds a prefix that this DHCP server can distribute to routers.
ipdhcps6_prefix_delete( )	Deletes a prefix.
ipdhcps6_prefix_get( )	Fetches the prefix that matches a specified key.
ipdhcps6_prefix_list( )	Lists all DHCPv6 prefixes.
ipdhcps6_host_add( )	Adds a DHCPv6 host to the database.
ipdhcps6_host_delete( )	Deletes a DHCPv6 host from the database.
ipdhcps6_host_list( )	List all DHCPv6 hosts in the database.
ipdhcps6_host_to_string( )	Creates a string representation of the host entry.
ipdhcps6_host_get_user( )	Gets the username that the host used when it authenticated.
ipdhcps6_user_add( )	Adds a new user to the DHCPv6 server.
ipdhcps6_user_delete( )	Deletes a user from the DHCPv6 server.
ipdhcps6_user_list( )	Lists all DHCPv6 users in the database.
ipdhcps6_version( )	Returns the version string of the server.

5. Assigning Client-Specific Authentication Keys

To assign a separate authentication key for a specific client, based on the client DUID, use the following routines:

ipdhcps6_authdb_create_ll_duid( )

```
/* create a DUID based on link-layer address: section 9.4 RFC3315*/
IP_GLOBAL Ipdhcps6_duid *
    ipdhcps6_authdb_create_ll_duid (int len, Ip_u16 hwtype, Ip_u8 * macaddr)
```

The parameters to this routine are as follows:

len – length of the hardware address (mac address)

hwtype – the hardware type code. Most likely to be 0x1 (ethernet)

macaddr – pointer to a byte string containing the hardware address.

Note: Currently, len is ignored. ipdhcps6_authdb_create_ll_duid( ) uses only the first six bytes of macaddr to identify the client.

ipdhcps6_authdb_add_client_authkey( )

```
/* associate an authentication key with a specific client */
IP_GLOBAL IP_err
    ipdhcps6_authdb_add_client_authkey (Ipdhcps6_duid * client_duid,
                                        Ip_u8 * key, int keylen)
```

Given a previously created client DUID structure, and a key/key length pair, this routine creates an entry in the internal authentication database assigning the key to the given client.

This routine returns **IPCOM_SUCCESS** if successful, or **IPCOM_ERR_FAILED** if not.

Example Code: Assigning an Authentication Key to a Client

The client has an ethernet mac address of 01:02:03:04:05:06, and we wish to assign a specific authentication key of **"this is a secret"**.

```
void assign_example (void)
    {
    Ipdhcps6_duid * client_duid;

    Ip_u8 macaddr[] = { 0x1, 0x2, 0x3, 0x4, 0x5, 0x6 };
    char * key = "this is a secret";
    int keylen = strlen (key);

    client_duid = ipdhcps6_authdb_create_ll_duid (6, 0x1, macaddr);
    if (IP_NULL == client_duid)
        {
        LOG_ERROR_SOMEHOW ("couldnt create client duid");
        return;
        }

    if (IPCOM_SUCCESS !=
    ipdhcps6_authdb_add_client_authkey (client_duid, (IP_u8 *) key, keylen))
        {
        LOG_ERROR_SOMEHOW ("could not associate key with client duid");
        }
}
```

The return value from ipdhcps6_authdb_create_ll_duid( ) is a pointer to a memory area obtained by a call to ipcom_malloc( ). Once you have called ipdhcps6_authdb_add_client_authkey( ) to associate the key with the client DUID in that memory region, you may release this memory area by calling ipcom_free( ). However, you may wish to keep this DUID (and the memory region that contains it) for a later call to ipdhcps6_authdb_delete_client_authkey( ).

Note: This assignment procedure is sufficient for clients that send DUID's of type LL (link-layer-address) or LLT (link-layer-address + time). Assigning a client-specific key to clients which send an EN (enterprise number) type DUID is currently unsupported (but is planned in a future release).

ipdhcps6_authdb_delete_client_authkey( )

```
/* To delete a specific clients auth key entry */
IP_GLOBAL IP_err
    ipdhcps6_authdb_delete_client_authkey (Ipdhcps6_duid *client_duid);
```

This call deletes any client-specific key associated with the given client DUID. Returns **IPCOM_SUCCESS** if successful, **IPCOM_ERR_FAILED** on failure.

6. Fully Qualified Domain Name

The Domain Name System (DNS) maintains mapping between a host's Fully Qualified Domain Name (FQDN) and an IPv6 address. The information is maintained in two types of Resource Records (RRs); AAAA and a pointer (PTR) record.

DHCPv6 provides a mechanism whereby a DHCPv6 client and server can exchange information about the client's fully qualified domain name and about who has the responsibility for updating the DNS with the associated AAAA resource record (RR). An AAAA record associates a domain name to an IPv6 address whereas a PTR resource record associates an IPv6 address to a domain name. The server always updates the PTR RR.

For additional information on FQDN, including use cases, see Implementing FQDN on page 83.

9. DHCP SETUP TUTORIAL

1. Network Requirements and Configuring the Targets

This tutorial guides you through the setup, configuration, testing, and troubleshooting of a DHCP environment with IPv4 addressing. The tutorial describes a simulated implementation of a network consisting of a DHCPv4 server with relay agent information option 82 capability, a DHCP relay agent with relay agent information option 82 capability, and three clients, one in each of the simulated client subnets.

For information on the relay agent information option (option 82), see DHCPv4 Relay Agent Information Option (option 82) on page 56.

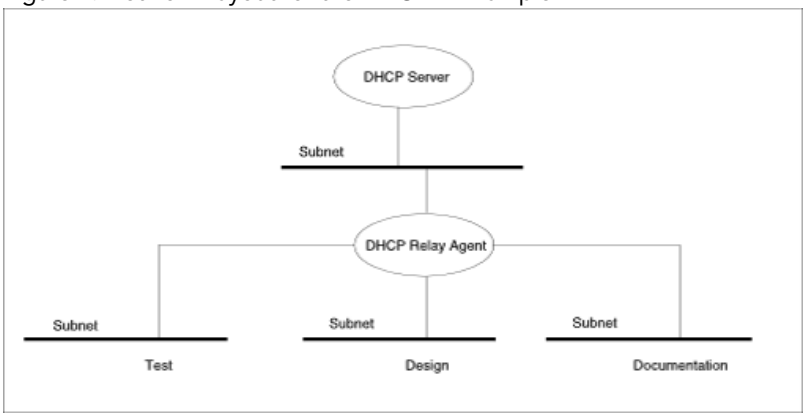
Operating Environment

The example scenario is executed on a PC using VxSim for VxWorks 7 (**vxsim**).

Networking Layout

Network Layout for the DHCPv4 Example on page 96 details the network layout where the DHCPv4 server connects to the relay agent over a subnet and the relay agent connects to three subnets; one for each department. The department names are labelled as **test**, **design**, and **documentation**.

Figure 2. Network Layout for the DHCPv4 Example



Network Configuration

Network IP Configuration for the Example on page 97 details the network IP configuration for the example. The network configuration is as follows:

- a DHCPv4 server with relay agent information option capability
- a DHCP relay agent with agent information option capability
- a 172.16.1.0 subnet interconnecting the DHCPv4 server and the DHCP relay agent
  - the DHCP relay agent is at 172.16.1.1
  - the DHCPv4 server is at 172.16.1.2
- three subnets interconnecting the DHCP relay agent to each of the three clients:

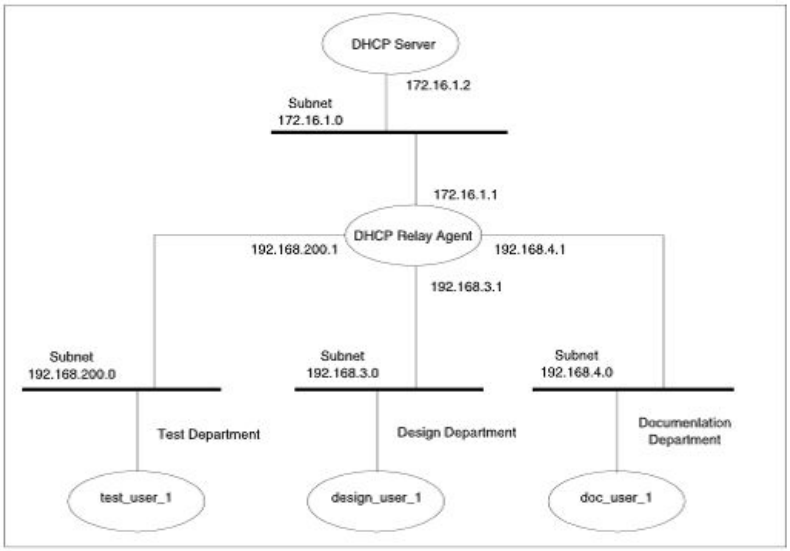
  - a 192.168.200.0 subnet for the **test** department
  - a 192.168.3.0 subnet for the **design** department
  - a 192.168.4.0 subnet for the **documentation** department

- a connection from the DHCP relay agent to each subnet:
  - 192.168.200.1 for the **test** subnet
  - 192.168.3.1 for the **design** subnet
  - 192.168.4.1 for the **documentation** subnet

- a DHCPv4 client in each subnet:
  - **test_user_1**
  - **design_user_1**
  - **doc_user_1**

The example network involves three DHCPv4 clients assigned to different subnets The relay agent relays the DHCP messages between the DHCP clients and the DHCPv4 server. The DHCP relay agent adds relay agent information to enhance security and allow the DHCPv4 server to assign IP addresses based on the class of the client as specified by option 82 parameters.

Figure 3. Network IP Configuration for the Example



Network Requirements

For this example, the DHCPv4 server has three classes; one for each department. The classes are as follows:

- one class for the testers: **option82_agent1_test**
- one class for the designers: **option82_agent1_design**
- one class for the writers: **option82_agent1_doc**

For this example, the DHCPv4 server receives information from the DHCP relay agent that allows the server to assign IP addresses based on the requesting subnet.

The required distribution of IP addresses for the example are as follows:

- Test users are limited to addresses 192.168.200.10 to 192.168.200.19.
- Design users are limited to addresses 192.168.3.10 to 192.168.3.19.

WNDRVR

- Documentation users are limited to addresses 192.168.4.10 to 192.168.4.19.

As part of the relay agent information circuit-identifier suboption, the DHCP relay agent supplies the name of the users associated with each subnet. The relay agent information option parameters for the example are as follows:

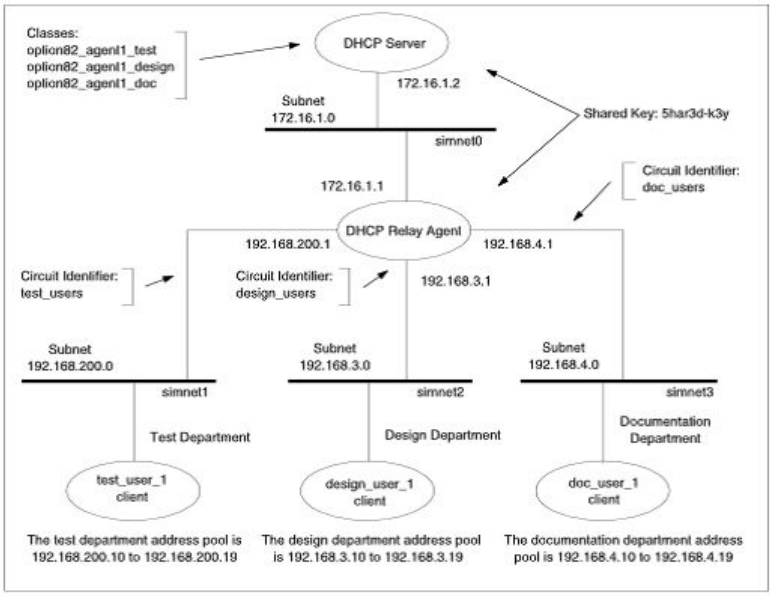
- circuit identifier for the test department users: **test_users**
- circuit identifier for the design department users: **design_users**
- circuit identifier for the documentation department users: **doc_users**
- server and relay agent shared key: **5har3d-k3y**

For **vxsim** simulation purposes, the subnet naming is as follows:

- **test_users** are on **simnet1**
- **design_users** are on **simnet2**
- **doc_users** are on **simnet3**

Detailed Network Configuration for the Example on page 98 details the network requirements for the example.

Figure 4. Detailed Network Configuration for the Example



Configuring the Target for the DHCPv4 Server

To meet the requirements set in Network Requirements on page 97 configure the DHCPv4 server as follows:

1. Configure the three subnet address ranges.
2. Create a class for each subnet and configure the circuit identifier for each subnet.
3. Configure the address pool for each class.
4. Configure the shared key for each class. Only one shared key is allowed for a given DHCP relay agent.
5. Enable the interface between the DHCPv4 server and the DHCP relay agent. In this case we use the VxSim-assigned subnet name of **simnet0**.
6. Define the three routes connecting the DHCPv4 server to the three client subnets.
7. Set the drop policy to true for all subnets.

The following shell commands configure the DHCPv4 server for the example:

```
dhcps subnet add 192.168.200.0 255.255.255.0
dhcps subnet add 192.168.3.0 255.255.255.0
dhcps subnet add 192.168.4.0 255.255.255.0

dhcps class add 192.168.200.0 option82_agent1_test circuit-id test_users
dhcps class add 192.168.3.0 option82_agent1_design circuit-id design_users
dhcps class add 192.168.4.0 option82_agent1_doc circuit-id doc_users

dhcps pool add 192.168.200.0 -c option82_agent1_test 192.168.200.10 192.168.200.19
dhcps pool add 192.168.3.0 -c option82_agent1_design 192.168.3.10 192.168.3.19
dhcps pool add 192.168.4.0 -c option82_agent1_doc 192.168.4.10 192.168.4.19
dhcps config set 192.168.200.0 -c option82_agent1_test option82-auth 5har3d-k3y
dhcps config set 192.168.3.0 -c option82_agent1_design option82-auth 5har3d-k3y
dhcps config set 192.168.4.0 -c option82_agent1_doc option82-auth 5har3d-k3y
dhcps interface enable simnet0
route add -net 192.168.200.0/24 172.16.1.1
route add -net 192.168.3.0/24 172.16.1.1
route add -net 192.168.4.0/24 172.16.1.1
dhcps config set 192.168.200.0 -c option82_agent1_test option82-drop true
dhcps config set 192.168.3.0 -c option82_agent1_design option82-drop true
dhcps config set 192.168.4.0 -c option82_agent1_doc option82-drop true
```

Configuring the Target for the DHCP Relay Agent

To meet the requirements set in Network Requirements on page 97 configure the DHCP relay as follows:

1. Enable the relay agent information option for the DHCP relay agent.
2. Enable the drop policy for the DHCP relay agent.
3. Configure the circuit identifier for each of the three subnets.
4. Add the server to the list of valid DHCP servers.
5. Define the relay agent information option for the server and define the shared secret.

The following shell commands configure the DHCP relay agent for the example:

```
dhcpr option82 enable auth

dhcpr option82 enable drop

dhcpr interface option82 simnet1 circuit-id test_users
dhcpr interface option82 simnet2 circuit-id design_users
dhcpr interface option82 simnet3 circuit-id doc_users

dhcpr server add 172.16.1.2

dhcpr server option82 172.16.1.2 enable auth 5har3d-k3y
```

2. Configuring and Building the VSB

The VSB is targeted for VxSim. You can use either Workbench 4 or the **wrtool** command line method. Proceed as follows to create the VSB:

WNDRVR

1. Create a VSB and base it on the **simpc** BSP. Label the VSB as **Option82_DHCP**.

   2From the default simpc VSB, enable the following components:
   - For the DHCPv4 server, set **IPNET_DHCPS** to yes.
   - For the DHCP relay agent, set **IPNET_DHCPR** to yes.
   - For the DHCPv4 clients, set **IPNET_DHCPC** to yes.

3. Build the VSB.

3. Configuring and Building the VIPs

This example requires three VIPs; one for the DHCPv4 server, one for the DHCP relay agent, and one for the DHCPv4 clients. You must base the VIPs on the VSB created in Configuring and Building the VSB on page 99.

DHCPv4 Server VIP

Proceed as follows to create the DHCPv4 server VIP:

1. Create a base VIP and base it on the Option82_DHCP VSB. Label the VIP as **dhcp_server**.

   2Include the following components:
   - **INCLUDE_IPCOM_SHELL_CMD**
   - **INCLUDE_IPD_CMD**
   - **INCLUDE_IPDHCPS**
   - **INCLUDE_IPROUTE_CMD**
   - **INCLUDE_PING**
   - **INCLUDE_SHELL_INTERP_CMD**
   - **INCLUDE_STARTUP_SCRIPT**

3. Build the VIP.

DHCP Relay Agent VIP

Proceed as follows to create the DHCP relay agent VIP:

1. Create a base VIP and base it on the Option82_DHCP VSB. Label the VIP as **dhcp_relay_agent**.

   2Include the following components:
   - **INCLUDE_IPCOM_SHELL_CMD**
   - **INCLUDE_IPD_CMD**
   - **INCLUDE_IPDHCPR**
   - **INCLUDE_PING**
   - **INCLUDE_SHELL_INTERP_CMD**
   - **INCLUDE_STARTUP_SCRIPT**

3. Build the VIP

DHCPv4 Client Build

Proceed as follows to create the DHCPv4 client VIP:

1. Create a base VIP and base it on the Option82_DHCP VSB. Label the VIP as dhcp_client.

2Include the following components:
   - **INCLUDE_IPCOM_SHELL_CMD**
   - **INCLUDE_IPD_CMD**
   - **INCLUDE_IPDHCPC**
   - **INCLUDE_PING**
   - **INCLUDE_SHELL_INTERP_CMD**
   - **INCLUDE_STARTUP_SCRIPT**

3. Build the VIP.

4. Creating the Target Startup Scripts

To automate the target startup and configuration process, this example includes one startup script for each target type. The startup script for the DHCP server is based on the information in Configuring the Target for the DHCP Relay Agent on page 99. The startup script for the DHCP relay agent is based on the information in Configuring the Target for the DHCPv4 Server on page 98.

Create a folder to store the startup scripts and name the folder **option82_startup_scripts**.

DHCPv4 Server Statup Script

Create a text file named **server_startup.txt** in the **option82_startup_scripts** folder and include the following shell commands in the file:

```
cmd
dhcps subnet add 192.168.200.0 255.255.255.0
dhcps subnet add 192.168.3.0 255.255.255.0
dhcps subnet add 192.168.4.0 255.255.255.0
dhcps class add 192.168.200.0 option82_agent1_test circuit-id test_users
dhcps class add 192.168.3.0 option82_agent1_design circuit-id design_users
dhcps class add 192.168.4.0 option82_agent1_doc circuit-id doc_users
dhcps pool add 192.168.200.0 -c option82_agent1_test 192.168.200.10 192.168.200.19
dhcps pool add 192.168.3.0 -c option82_agent1_design 192.168.3.10 192.168.3.19
dhcps pool add 192.168.4.0 -c option82_agent1_doc 192.168.4.10 192.168.4.19
dhcps config set 192.168.200.0 -c option82_agent1_test option82-auth 5har3d-k3y
dhcps config set 192.168.3.0 -c option82_agent1_design option82-auth 5har3d-k3y
dhcps config set 192.168.4.0 -c option82_agent1_doc option82-auth 5har3d-k3y
dhcps interface enable simnet0
route add -net 192.168.200.0/24 172.16.1.1
route add -net 192.168.3.0/24 172.16.1.1
route add -net 192.168.4.0/24 172.16.1.1
dhcps config set 192.168.200.0 -c option82_agent1_test option82-drop true
dhcps config set 192.168.3.0 -c option82_agent1_design option82-drop true
dhcps config set 192.168.4.0 -c option82_agent1_doc option82-drop true
```


WNDRVR

DHCP Relay Agent Startup Script

Create a text file named **relay_agent_startup.txt** in the **option82_startup_scripts** folder and include the following shell commands in the file:

```
cmd
dhcpr option82 enable auth
dhcpr option82 enable drop
dhcpr interface option82 simnet1 circuit-id test_users
dhcpr interface option82 simnet2 circuit-id design_users
dhcpr interface option82 simnet3 circuit-id doc_users
dhcpr server add 172.16.1.2
dhcpr server option82 172.16.1.2 enable auth 5har3d-k3y
```

DHCPv4 Client Startup Script

Create a text file named **client_startup.txt** in the **option82_startup_scripts** folder and include the following shell command in the file:

```
cmd ifconfig simnet0 dhcp
```

The ifconfig command initiates the DHCP auto negotiation process.

5. Configuring VxSim

The **vxsim** simulation contains four subnets as follows:

- 172.16.1.0
- 192.168.200.0
- 192.168.3.0
- 192.168.4.0

To access the subnets from a network protocol analyser such as Wireshark, you must configure the subnets to be externally accessible. This example configures the following subnets as external:

- 172.16.1.0
- 192.168.200.0

To access the subnets you must also create a WRTAP network connection for each subnet. Refer to the *VxWorks 7 VxSim User's Guide* for information on how to create these network connections.

Configuration File for the Simulation

Create a text file named **vxsim_option82.conf** in the *installDir* **/vxworks-7/host/x86-win32/bin** folder and include the following configuration in the file:

```
SUBNET_START sub2 {
SUBNET_ADDRESS = "192.168.200.0";
SUBNET_EXTERNAL = yes;
SUBNET_EXTPROMISC = yes;
```

```
};
SUBNET_START sub3 {
SUBNET_ADDRESS = "192.168.3.0";
SUBNET_EXTERNAL = no;
SUBNET_EXTPROMISC = no;
};
SUBNET_START sub4 {
SUBNET_ADDRESS = "192.168.4.0";
SUBNET_EXTERNAL = no;
SUBNET_EXTPROMISC = no;
};
SUBNET_START sub5 {
SUBNET_ADDRESS = "172.16.1.0";
SUBNET_EXTERNAL = yes;
SUBNET_EXTPROMISC = yes;
};
```

6. Launching VxSim

To launch VxSim, start a Windows command shell and enter the following command from the *installDir* **/vxworks-7/host/x86-win32/bin** directory:

```
C:\installDir\vxworks-7\host\x86-win32\bin> vxsimnetd -f vxsim_option82.conf -s

vxsimnetd Debug Shell
vxsimnetd>
```

You can verify the VxSim configuration with the following commands:

```
vxsimnetd> subnet
vxsimnetd> node sub2
vxsimnetd> node sub3
vxsimnetd> node sub4
vxsimnetd> node sub5
```

Getting Help

To get help on VxSim, use the following command from the *installDir* **/vxworks-7/host/x86-win32/bin** directory:

```
C:\installDir\vxworks-7\host\x86-win32\bin> vxsim -help
```

To get help on the vxsimnet daemon startup options, use the following command:

```
C:\installDir\vxworks-7\host\x86-win32\bin> vxsimnetd -help
```

To get help on the vxsimnet daemon, use the following command:

```
vxsimnetd> help
```

See also the *VxWorks 7 VxSim User's Guide*.

WNDRVR

7. Launching the Targets

To launch the DHCP targets, launch a VxWorks development shell for each target and execute the target startup command from the target default directory.

Procedure

1. Launch the DHCP Server.

   To launch the DHCPv4 server, the vxsim command must initiate the following actions:
   - Create a network interface on simnet0 with IP address 172.16.1.2.
   - Assign process number 1 to the DHCP server simulation.
   - Assign a label of Server to the simulation.
   - Execute the server_startup.txt startup script.

   The vxsim command for the DHCP server is as follows:

```
C:\WindRiver\workspace\dhcp_server\default> vxsim -ni simnet0=172.16.1.2 -p 1
-tn "Server" -s c:/dhcp/dhcp_option82/server_startup.txt
```

2. Observe the Execution of the Startup Script.

   From the target window, observe that the startup script is executed successfully. The output should be as follows:

```
...
Executing startup script ...

cmd
dhcps subnet add 192.168.200.0 255.255.255.0
dhcps subnet add 192.168.3.0 255.255.255.0
dhcps subnet add 192.168.4.0 255.255.255.0
dhcps class add 192.168.200.0 option82_agent1_test circuit-id test_users
dhcps class add 192.168.3.0 option82_agent1_design circuit-id design_users
dhcps class add 192.168.4.0 option82_agent1_doc circuit-id doc_users
dhcps pool add 192.168.200.0 -c option82_agent1_test 192.168.200.10 192.168.200.19
dhcps pool add 192.168.3.0 -c option82_agent1_design 192.168.3.10 192.168.3.19
dhcps pool add 192.168.4.0 -c option82_agent1_doc 192.168.4.10 192.168.4.19
dhcps config set 192.168.200.0 -c option82_agent1_test option82-auth 5har3d-k3y
dhcps config set 192.168.3.0 -c option82_agent1_design option82-auth 5har3d-k3y
dhcps config set 192.168.4.0 -c option82_agent1_doc option82-auth 5har3d-k3y
dhcps interface enable simnet0
route add -net 192.168.200.0/24 172.16.1.1
add net 192.168.200.0: netmask 255.255.255.0: gateway 172.16.1.1
route add -net 192.168.3.0/24 172.16.1.1
add net 192.168.3.0: netmask 255.255.255.0: gateway 172.16.1.1
route add -net 192.168.4.0/24 172.16.1.1
add net 192.168.4.0: netmask 255.255.255.0: gateway 172.16.1.1
dhcps config set 192.168.200.0 -c option82_agent1_test option82-drop true
dhcps config set 192.168.3.0 -c option82_agent1_design option82-drop true
dhcps config set 192.168.4.0 -c option82_agent1_doc option82-drop true

Done executing startup script ...
->
```

3. Launch the DHCP Relay Agent.

To launch the DHCP relay agent, the vxsim command must initiate the following actions:

- Assign IP address 172.16.1.1 to the DHCP relay agent on simnet0.
- Create a network interface on simnet1 with IP address 192.168.200.1.
- Create a network interface on simnet2 with IP address 192.168.3.1.
- Create a network interface on simnet3 with IP address 192.168.4.1.
- Assign process number 2 to the DHCP relay agent simulation.
- Assign the label of Relay_Agent to the simulation.
- Execute the relay_agent_startup.txt startup script.

The vxsim command for the DHCP relay agent is as follows:

```
C:\WindRiver\workspace\dhcp_relay_agent\default> vxsim -d simnet -e 172.16.1.1 -ni
simnet1=192.168.200.1;simnet2=192.168.3.1;simnet3=192.168.4.1 -p 2 -tn "Relay_Agent"
-s c:/dhcp/dhcp_option82/relay_agent_startup.txt
```

4. Observe the Execution of the Startup Script.

   From the target window, observe that the startup script is executed successfully. The output should be as follows:

```
...
Executing startup script ...

cmd
dhcpr option82 enable auth
dhcpr option82 enable drop
dhcpr interface option82 simnet1 circuit-id test_users
dhcpr interface option82 simnet2 circuit-id design_users
dhcpr interface option82 simnet3 circuit-id doc_users
dhcpr server add 172.16.1.2
dhcpr server option82 172.16.1.2 enable auth 5har3d-k3y

Done executing startup script ...
->
```

5. Launch DHCPv4 Client 1 (test_user_1).

   To launch DHCPv4 client number one, enter the following command:

```
C:\WindRiver\workspace\dhcp_client\default>vxsim -d simnet -e 192.168.200.2 -p 3
-tn "Client1" -s c:/dhcp/dhcp_option82/client_startup.txt
```

   The startup script initiates a DHCP request for an IP address from the DHCP server.

6. Observe the Execution of the Startup Script.

   From the target window, observe that the startup script is executed successfully.

   The output should be as follows:

```
Executing startup script ...
cmd ifconfig simnet0 dhcp
Done executing startup script ...
->
```

7. Launch DHCP Client 2 (design_user_1).

To launch DHCP client number two, enter the following command:

```
C:\WindRiver\workspace\dhcp_client\default>vxsim -d simnet -e 192.168.3.2 -p 4 -tn
"Client2" -s c:/dhcp/dhcp_option82/client_startup.txt
```

The startup script initiates a DHCP request for an IP address from the DHCP server.

8. Observe the Execution of the Startup Script.

   From the target window, observe that the startup script is executed successfully. The output should be as follows:

```
Executing startup script ...Executing startup script ...
cmd ifconfig simnet0 dhcp
Done executing startup script ...
->
```

9. Launch DHCP Client 3 (doc_user_1).

   To launch DHCP client number three, enter the following command:

```
C:\WindRiver\workspace\dhcp_client\default>vxsim -d simnet -e 192.168.4.2 -p 5 -tn
"Client3" -s c:/dhcp/dhcp_option82/client_startup.txt
```

   The startup script initiates a DHCP request for an IP address from the DHCP server.

10. Observe the Execution of the Startup Script.

    From the target window, observe that the startup script is executed successfully. The output should be as follows:

```
Executing startup script ...
cmd ifconfig simnet0 dhcp
Done executing startup script ...
->
```

8. Verifying the DHCP Service

Once a client is launched and a request for an IP address is made through a DHCP Discover message from the client, circuit identifier and shared key parameters are exchanged between the DHCP relay agent and the DHCPv4 server. The DHCP process ensures that an exact match of the parameters is found before an address is assigned to the client.

The DHCPv4 server provides an address taken from the configured address pool assigned to the class of the client.

To verify the successful completion of the DHCP Discover, DHCP Offer, DHCP Request, and DHCP Ack process, simply query the current IP address of each target. The expected output for each of the three clients is as follows:

- Client 1 (test_user_1)

  Client 1 should have an IP address in the 192.168.200.10 to 192.168.200.19 range. Use the ifconfig command to display the current configuration.

```
-> ifconfig
...
simnet0Link type:Ethernet HWaddr 7a:7a:c0:a8:c8:02 Queue:none
inet 192.168.200.10 mask 255.255.255.0 broadcast 192.168.200.255
UP RUNNING SIMPLEX BROADCAST MULTICAST DHCP
```

```
MTU:1500 metric:1 VR:0 ifindex:2
RX packets:6 mcast:0 errors:0 dropped:6
TX packets:6 mcast:0 errors:0
collisions:0 unsupported proto:0
RX bytes:2052 TX bytes:1152
value = 0 = 0x0
->
```

- Client 2 (design_user_1)

  Client 2 should have an IP address in the 192.168.3.10 to 192.168.3.19 range. Use the ifconfig command to display the current configuration.

```
-> ifconfig
...
simnet0Link type:Ethernet HWaddr 7a:7a:c0:a8:03:02 Queue:none
inet 192.168.3.10 mask 255.255.255.0 broadcast 192.168.3.255
UP RUNNING SIMPLEX BROADCAST MULTICAST DHCP
MTU:1500 metric:1 VR:0 ifindex:2
RX packets:4 mcast:0 errors:0 dropped:4
TX packets:7 mcast:0 errors:0
collisions:0 unsupported proto:0
RX bytes:1368 TX bytes:1494
value = 0 = 0x0
->
```

- Client 3 (doc_user_1)

  Client 3 should have an IP address in the 192.168.4.10 to 192.168.4.19 range. Use the ifconfig command to display the current configuration.

```
-> ifconfig
...
simnet0Link type:Ethernet HWaddr 7a:7a:c0:a8:04:02 Queue:none
inet 192.168.4.10 mask 255.255.255.0 broadcast 192.168.4.255
UP RUNNING SIMPLEX BROADCAST MULTICAST DHCP
MTU:1500 metric:1 VR:0 ifindex:2
RX packets:4 mcast:0 errors:0 dropped:4
TX packets:10 mcast:0 errors:0
collisions:0 unsupported proto:0
RX bytes:1368 TX bytes:2520
value = 0 = 0x0
->
```

9. Troubleshooting with Shell Commands

You can use shell commands to verify the configuration and operation of the DHCP relay agent and the DHCPv4 server. You can also use the ping command to verify connectivity between targets.

Verifying the DHCP Server Functionality

Use the following commands to view the configuration and operational parameters of the DHCPv4 server.

Subnets

View the DHCP subnet list:

WNDRVR

```
# dhcps subnet list
Subnet: default
Subnet: 192.168.200.0 netmask: 255.255.255.0
Subnet: 192.168.4.0 netmask: 255.255.255.0
Subnet: 192.168.3.0 netmask: 255.255.255.0
```

All three subnets must be listed.

Interfaces

View the interface list:

```
# dhcps interface list
Interface simnet0(172.16.1.2) :: enabled
```

The interface to the DHCP relay agent must be set to enabled.

Pool Address Assignments and Classes

View the pool lists:

```
# dhcps pool list 192.168.200.0
Pool: 192.168.200.10 -> 192.168.200.19 (9 entries)
bound to class: option82_agent1_test

# dhcps pool list 192.168.3.0
Pool: 192.168.3.10 -> 192.168.3.19 (9 entries)
bound to class: option82_agent1_design

# dhcps pool list 192.168.4.0

Pool: 192.168.4.10 -> 192.168.4.19 (9 entries)
bound to class: option82_agent1_doc
```

The three class names and their address range must match the scenario requirements.

Class Identifiers

View the class lists:

```
# dhcps class list 192.168.200.0
Class :: name:option82_agent1_test, type:circuit-id, id:"test_users"
# dhcps class list 192.168.3.0
Class :: name:option82_agent1_design, type:circuit-id, id:"design_users"
# dhcps class list 192.168.4.0
Class :: name:option82_agent1_doc, type:circuit-id, id:"doc_users"
```

The class identifiers must match the scenario requirements.

Subnet Masks

View the option lists:

```
# dhcps option list 192.168.200.0
Code Description Value
```

```
1 subnet mask 255.255.255.0
# dhcps option list 192.168.3.0
Code Description Value
1 subnet mask 255.255.255.0
# dhcps option list 192.168.4.0
Code Description Value
1 subnet mask 255.255.255.0
```

Subnet and Class Configuration

View the class configuration, including the authentication and drop policy configuration:

```
# dhcps config list 192.168.200.0 -c option82_agent1_test
Configuration Value
default-lease-time 864000
max-lease-time 8640000
min-lease-time 60
renewal-time 0
rebinding-time 0
allow-bootp true
allow-dynamic-bootp false
next-server-name
next-server-ip 0.0.0.0
boot-file
do-icmp-address-check false
authorized-agents any
option82-auth 5har3d-k3y
option82-drop true

# dhcps config list 192.168.3.0 -c option82_agent1_design
Configuration Value
default-lease-time 864000
max-lease-time 8640000
min-lease-time 60
renewal-time 0
rebinding-time 0
allow-bootp true
allow-dynamic-bootp false
next-server-name
next-server-ip 0.0.0.0
boot-file
do-icmp-address-check false
authorized-agents any
option82-auth 5har3d-k3y
option82-drop true

# dhcps config list 192.168.4.0 -c option82_agent1_doc
Configuration Value
default-lease-time 864000
max-lease-time 8640000
min-lease-time 60
renewal-time 0
rebinding-time 0
allow-bootp true
allow-dynamic-bootp false
next-server-name y
next-server-ip 0.0.0.0
```

```
boot-file
do-icmp-address-check false
authorized-agents any
option82-auth 5har3d-k3y
option82-drop true
```

DHCP leases

```
# dhcps lease list
IP Address : 192.168.4.10
Binding state: active
Bound to :
MAC: 7a:7a:c0:a8:04:02
ID : 0x01 'z' 'z' 0xc0 0xa8 0x04 0x02
Lease time : 864000 seconds
Expires at : 2010/12/03 04:36:21
IP Address : 192.168.200.10
Binding state: active
Bound to :
MAC: 7a:7a:c0:a8:c8:02
ID : 0x01 'z' 'z' 0xc0 0xa8 0xc8 0x02
Lease time : 864000 seconds
Expires at : 2010/12/03 04:33:08
IP Address : 192.168.3.10
Binding state: active
Bound to :
MAC: 7a:7a:c0:a8:03:02
ID : 0x01 'z' 'z' 0xc0 0xa8 0x03 0x02
Lease time : 864000 seconds
Expires at : 2010/12/03 04:33:53
```

116 Pinging the Network Interfaces of the DHCP Relay Agent:

```
# ping 172.16.1.1
# ping 192.168.200.1
# ping 192.168.3.1
# ping 192.168.4.1
```

All of the above pings should be successful.

Verifying the DHCP Relay Agent Functionality

Use the following commands to view the configuration and the operational parameters of the DHCP relay agent.

Interfaces

Viewing the interface list:

```
# dhcpr interface list
Interface simnet3 (192.168.4.1) :: enabled, circuit-ID : doc_users,
remote-ID :
Interface simnet2 (192.168.3.1) :: enabled, circuit-ID : design_users,
remote-ID :
Interface simnet1 (192.168.200.1) :: enabled, circuit-ID : test_users,
```

```
remote-ID :
Interface simnet0 (172.16.1.1) :: enabled, circuit-ID : ,
remote-ID :
```

The interfaces to the client subnets and the DHCP server must be set to **enabled** and the identifiers must match the scenario requirements.

Servers

Viewing the server list:

```
# dhcpr server list
Registered servers
172.16.1.2 :: shareKey : 5har3d-k3y, firstPacket : TRUE, curSendRDCount : 0,
lastRecvRDCount : 0
```

The server address and the shared key must be match the scenario requirements.

Option 82 Authentication and Drop Policy Configuration

Viewing the option 82 configuration:

```
# dhcpr option82 list
DHCP relay agent authentication is enabled
DHCP relay agent drop policy is enabled
```

The authentication and drop policy must be set to enabled.

Pinging the Network Interfaces of the DHCP Server and the Clients:

```
# ping 172.16.1.2
# ping 192.168.200.10
# ping 192.168.3.10
# ping 192.168.4.10
```

All of the above pings should be successful.

Troubleshooting with a Network Protocol Analyser

To troubleshoot this scenario, you can launch a network protocol analyser such as Wireshark and capture the DHCP traffic between a client and the DHCP relay agent, and between the DHCP relay agent and the DHCPv4 server. Proceed as follows:

Procedure

1. Install a network protocol analyser such as Wireshark (used in this example).
2. Start the DHCP Server.

   If not already running, start the DHCP server, see Launching the Targets on page 104.

3. Start the DHCP Relay Agent.

   If not already running, start the DHCP relay agent, see Launching the Targets on page 104.

4. Terminate any Client Processes.

   Close all client target windows.

5. Start Wireshark and Enable the Capture of Packets on 172.16.1.0.
   1. Start an instance of Wireshark and navigate to the **Capture > Interfaces...** dialog box.
   2. Click the Start button for the **Wind River Tap Adapter 172.16.1.254** subnet.
6. Start Wireshark and Enable the Capture of Packets on 192.168.200.0.
   1. Start an instance of Wireshark and navigate to the **Capture > Interfaces...** dialog box.
   2. Click the Start button for the **Wind River Tap Adapter 192.168.200.254** subnet.
7. Start Client 1.

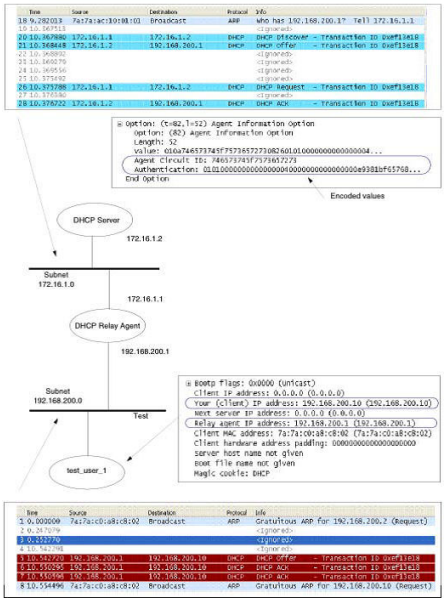
   Start client 1, see Launching the Targets on page 104.

8. Verify the Analyser Output.

   Check that the your output is similar to the one in Analyser Output for Client 1 - DHCP Messaging on page 112.

   Check the DHCP message sequence between the client and the relay agent, and between the relay agent and the server.

   Figure 5. Analyser Output for Client 1 - DHCP Messaging



10. Troubleshooting with the Console Debug Output

You can enable console debug output for the clients, the DHCP relay agent, and the DHCP server.

About This Task

To do so, proceed as follows:

Procedure

1. Enable debug mode in the VSB by enabling the **BUILD_MODE_debug** component.
2. In the **IPNET_DEFAULT_SYSLOG_PRIORITY_CHOICE** component, select **Information level**.
3. Rebuild the VSB.
4. Create a client, DHCP relay agent, or DHCP server VIP based on the debug-enabled VSB:
   1. Add the necessary components as described in Configuring and Building the VIPs on page 100.
   2. Add the **INCLUDE_IPCOM_SYSLOGD_CMD**) component to allow you to control the debug messages displayed on the console.
   3. For the **IPCOM_SYSLOGD_DEFAULT_PRIORITY** component, set it to **IPCOM_LOG_INFO**.
   4. Add the **INCLUDE_IPCOM_SYSLOGD_USE_LOG_FILE**) component to log the debug output. To display the log file contents use:

      ```
      # cat /ram/syslog
      ```

   5. Build the VIP. For this scenario, the Client1 is built with the debug-enabled VSB.
5. Launch VxSim.

   Launch VxSim, see Launching VxSim on page 103.

6. Launch the DHCP Server.

   Launch the DHCP server, see Launching the Targets on page 104.

7. Launch the DHCP Relay Agent.

   Launch the DHCP relay agent, see Launching the Targets on page 104.

8. Modify the Client1 Startup Script.

   To prevent Client1 from initiating the DHCP process upon startup, comment out the shell command in the startup script. Change the startup script as follows:

   from

   ```
   cmd ifconfig simnet0 dhcp
   ```

   to

   ```
   'cmd ifconfig simnet0 dhcp
   ```

9. Launch Client1.

   Launch Client1, see Launching the Targets on page 104.

10. Task step.
    1. From the Client1 C shell, run the following command:

       ```
       -> cmd syslog priority ipdhcpc debug
       ```

    2. Verify the setting with:

       ```
       -> cmd syslog list
       syslog facility priority
       ipcom Info
       ```

```
ipdhcpc Debug

ipnet Info
ipcom_drv_eth Info
iptcp Inf
```

11. Enable the DHCP Process from **Client1**.

    From the Client1 C shell, run the following command:

```
-> cmd ifconfig simnet0 dhcp
```

12. Observe the Debug Output and Troubleshoot as Required.

    Example output for the scenario follows. Compare this output with your output and troubleshoot as required. The DHCP message flow is highlighted in the output.

    The **ifconfig** command confirms the assignment of the correct IP address to **Client1**.

```
-> cmd ifconfig simnet0 dhcp
TUE NOV 23 17:36:11 2010: ipdhcpc[108df960]: Info: ipdhcpc[simnet0] :: Disabling IPv4 address a
uto
configuration
TUE NOV 23 17:36:11 2010: ipnet[1048bff0]: Info: simnet0 is now STOPPED
TUE NOV 23 17:36:11 2010: ipnet[1048bff0]: Info: simnet0 is now DOWN
TUE NOV 23 17:36:11 2010: ipnet[1048bff0]: Info: IPv4: removed 224.0.0.1 from simnet0
TUE NOV 23 17:36:11 2010: ipnet[1048bff0]: Info: simnet0 is now RUNNING
TUE NOV 23 17:36:11 2010: ipnet[1048bff0]: Info: simnet0 is now UP
TUE NOV 23 17:36:11 2010: ipnet[1048bff0]: Info: IPv4: added 224.0.0.1 to simnet0
TUE NOV 23 17:36:11 2010: ipnet[1048bff0]: Info: IPv4: removed 192.168.200.2 from simnet0
*******************

ipdhcpc[simnet0] :: Sending DHCP Request in REBOOTING state (len 300)
[ DHCPDISCOVER ] [ xid: 0x8ec1a1bc ] [ secs: 0 ] [ flags: 0 ]
[ ciaddr: 0.0.0.0 ] [ yiaddr: 0.0.0.0 ]
[ siaddr: 0.0.0.0 ] [ giaddr: 0.0.0.0 ]
[ Options (len 64): ]
[ Option # 53: Message Type: DISCOVER ]
[ Option # 61: Client Identifier: Ethernet: 7a:7a:c0:a8:c8:02: ]
[ Option # 57: Maximum DHCP Message Size: 576 ]
*******************

*******************
ipdhcpc[simnet0] :: Sending DHCP Request in REQUESTING state (len 300)
[ DHCPREQUEST ] [ xid: 0x8ec1a1bc ] [ secs: 0 ] [ flags: 0 ]
[ ciaddr: 0.0.0.0 ] [ yiaddr: 0.0.0.0 ]
[ siaddr: 0.0.0.0 ] [ giaddr: 0.0.0.0 ]
[ Options (len 64): ]
[ Option # 53: Message Type: REQUEST ]
[ Option # 61: Client Identifier: Ethernet: 7a:7a:c0:a8:c8:02: ]
[ Option # 54: Server Identifier: 172.16.1.2 ]
[ Option # 50: Requested IP Address: 192.168.200.10 ]
[ Option # 57: Maximum DHCP Message Size: 576 ]
*******************

*******************
ipdhcpc[simnet0] :: Receiving DHCP Reply in REQUESTING state (len 300)
```

```
[ DHCPOFFER ] [ xid: 0x8ec1a1bc ] [ secs: 0 ] [ flags: 0 ]
[ ciaddr: 0.0.0.0 ] [ yiaddr: 192.168.200.10 ]
[ siaddr: 0.0.0.0 ] [ giaddr: 192.168.200.1 ]
[ Options (len 64): ]
[ Option # 53: Message Type: OFFER ]
[ Option # 51: IP Address Lease Time: 864000 ]
[ Option # 54: Server Identifier: 172.16.1.2 ]
[ Option # 1: Subnet Mask: 255.255.255.0 ]
*******************
*******************
ipdhcpc[simnet0] :: Receiving DHCP Reply in REQUESTING state (len 300)
[ DHCPACK ] [ xid: 0x8ec1a1bc ] [ secs: 0 ] [ flags: 0 ]
[ ciaddr: 0.0.0.0 ] [ yiaddr: 192.168.200.10 ]
[ siaddr: 0.0.0.0 ] [ giaddr: 192.168.200.1 ]
[ Options (len 64): ]
[ Option # 53: Message Type: ACK ]
[ Option # 51: IP Address Lease Time: 864000 ]
[ Option # 54: Server Identifier: 172.16.1.2 ]
[ Option # 1: Subnet Mask: 255.255.255.0 ]
*******************

TUE NOV 23 17:36:23 2010: ipnet[1048bff0]: Info: IPv4: added 192.168.200.10 to simnet0

-> ifconfig "simnet0"

simnet0Link type:Ethernet HWaddr 7a:7a:c0:a8:c8:02 Queue:none
inet 192.168.200.10 mask 255.255.255.0 broadcast 192.168.200.255
UP RUNNING SIMPLEX BROADCAST MULTICAST DHCP
MTU:1500 metric:1 VR:0 ifindex:2
RX packets:6 mcast:0 errors:0 dropped:6
TX packets:6 mcast:0 errors:0
collisions:0 unsupported proto:0
RX bytes:2052 TX bytes:1152
value = 0 = 0x0
->
```

13. Create Debug-Enabled VIPs for the Relay Agent and Server.

    To further debug the DHCP process, you can also create debug-enabled VIPs for the DHCP relay agent and the DHCPv4 server.